

C++ Introduction

What is C++

C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.

C++ is a middle-level language, as it encapsulates both high and low level language features.

Object-Oriented Programming (OOPs)

C++ supports the object-oriented programming, the four major pillar of object-oriented programming (OOPs) used in C++ are:

1. Inheritance
2. Polymorphism
3. Encapsulation
4. Abstraction

C++ Standard Libraries

Standard C++ programming is divided into three important parts:

- The core library includes the data types, variables and literals, etc.
- The standard library includes the set of functions manipulating strings, files, etc.

- The Standard Template Library (STL) includes the set of methods manipulating a data structure.

Usage of C++

By the help of C++ programming language, we can develop different types of secured and robust applications:

- Window application
- Client-Server application
- Device drivers
- Embedded firmware etc

C++ Program

In this tutorial, all C++ programs are given with C++ compiler so that you can easily change the C++ program code.

File: main.cpp

```
#include <iostream>

using namespace std;

int main() {

    cout << "Hello C++ Programming";

    return 0;
```

}

C vs C++

What is C?

C is a structural or procedural oriented programming language which is machine-independent and extensively used in various applications.

C is the basic programming language that can be used to develop from the operating systems (like Windows) to complex programs like Oracle database, Git, Python interpreter, and many more. C programming language can be called a god's programming language as it forms the base for other programming languages. If we know the C language, then we can easily learn other programming languages. C language was developed by the great computer scientist Dennis Ritchie at the Bell Laboratories. It contains some additional features that make it unique from other programming languages.

What is C++?

C++ is a special-purpose programming language developed by **Bjarne Stroustrup** at Bell Labs circa 1980. C++ language is very similar to C language, and it is so compatible with C that it can run 99% of C programs without changing any source of code though C++ is an object-oriented programming language, so it is safer and well-structured programming language than C.

Let's understand the differences between C and C++.

The following are the differences between C and C++:

Definition

C is a structural programming language, and it does not support classes and objects, while C++ is an object-oriented programming language that supports the concept of classes and objects.

Type of programming language

C supports the structural programming language where the code is checked line by line, while C++ is an object-oriented programming language that supports the concept of classes and objects.

Developer of the language

Dennis Ritchie developed C language at Bell Laboratories while Bjarne Stroustrup developed the C++ language at Bell Labs circa 1980.

Subset

C++ is a superset of C programming language. C++ can run 99% of C code but C language cannot run C++ code.

Type of approach

C follows the top-down approach, while C++ follows the bottom-up approach. The top-down approach breaks the main modules into tasks; these tasks are broken into sub-tasks, and soon. The bottom-down approach develops the lower level modules first and then the next level modules.

Security

In C, the data can be easily manipulated by the outsiders as it does not support the encapsulation and information hiding while C++ is a very secure language, i.e., no

outsiders can manipulate its data as it supports both encapsulation and data hiding. In C language, functions and data are the free entities, and in C++ language, all the functions and data are encapsulated in the form of objects.

Function Overloading

Function overloading is a feature that allows you to have more than one function with the same name but varies in the parameters. C does not support the function overloading, while C++ supports the function overloading.

Function Overriding

Function overriding is a feature that provides the specific implementation to the function, which is already defined in the base class. C does not support the function overriding, while C++ supports the function overriding.

Reference variables

C does not support the reference variables, while C++ supports the reference variables.

Keywords

C contains 32 keywords, and C++ supports 52 keywords.

Namespace feature

A namespace is a feature that groups the entities like classes, objects, and functions under some specific name. C does not contain the namespace feature, while C++ supports the namespace feature that avoids the name collisions.

Exception handling

C does not provide direct support to the exception handling; it needs to use functions that

support exception handling. C++ provides direct support to exception handling by using a try-catch block.

Input/Output functions

In C, scanf and printf functions are used for input and output operations, respectively, while in C++, cin and cout are used for input and output operations, respectively.

Memory allocation and de-allocation

C supports calloc() and malloc() functions for the memory allocation, and free() function for the memory de-allocation. C++ supports a new operator for the memory allocation and delete operator for the memory de-allocation.

Inheritance

Inheritance is a feature that allows the child class to reuse the properties of the parent class. C language does not support the inheritance while C++ supports the inheritance.

Header file

C program uses <stdio.h> header file while C++ program uses <iostream.h> header file.

Let's summarize the above differences in a tabular form.

No.	C	C++
1)	C follows the procedural style programming .	C++ is multi-paradigm. It supports both procedural and object oriented .
2)	Data is less secured in C.	In C++, you can use modifiers for

		class members to make it inaccessible for outside users.
3)	C follows the top-down approach .	C++ follows the bottom-up approach .
4)	C does not support function overloading.	C++ supports function overloading.
5)	In C, you can't use functions in structure.	In C++, you can use functions in structure.
6)	C does not support reference variables.	C++ supports reference variables.
7)	In C, scanf() and printf() are mainly used for input/output.	C++ mainly uses stream cin and cout to perform input and output operations.
8)	Operator overloading is not possible in C.	Operator overloading is possible in C++.
9)	C programs are divided into procedures and modules	C++ programs are divided into functions and classes .
10)	C does not provide the feature of	C++ supports the feature of

	namespace.	namespace.
11)	Exception handling is not easy in C. It has to perform using other functions.	C++ provides exception handling using Try and Catch block.
12)	C does not support the inheritance.	C++ supports inheritance.

C++ History

History of C++ language is interesting to know. Here we are going to discuss brief history of C++ language.

C++ programming language was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.

Bjarne Stroustrup is known as the **founder of C++ language**.



It was develop for adding a feature of **OOP (Object Oriented Programming)** in C without significantly changing the C component.

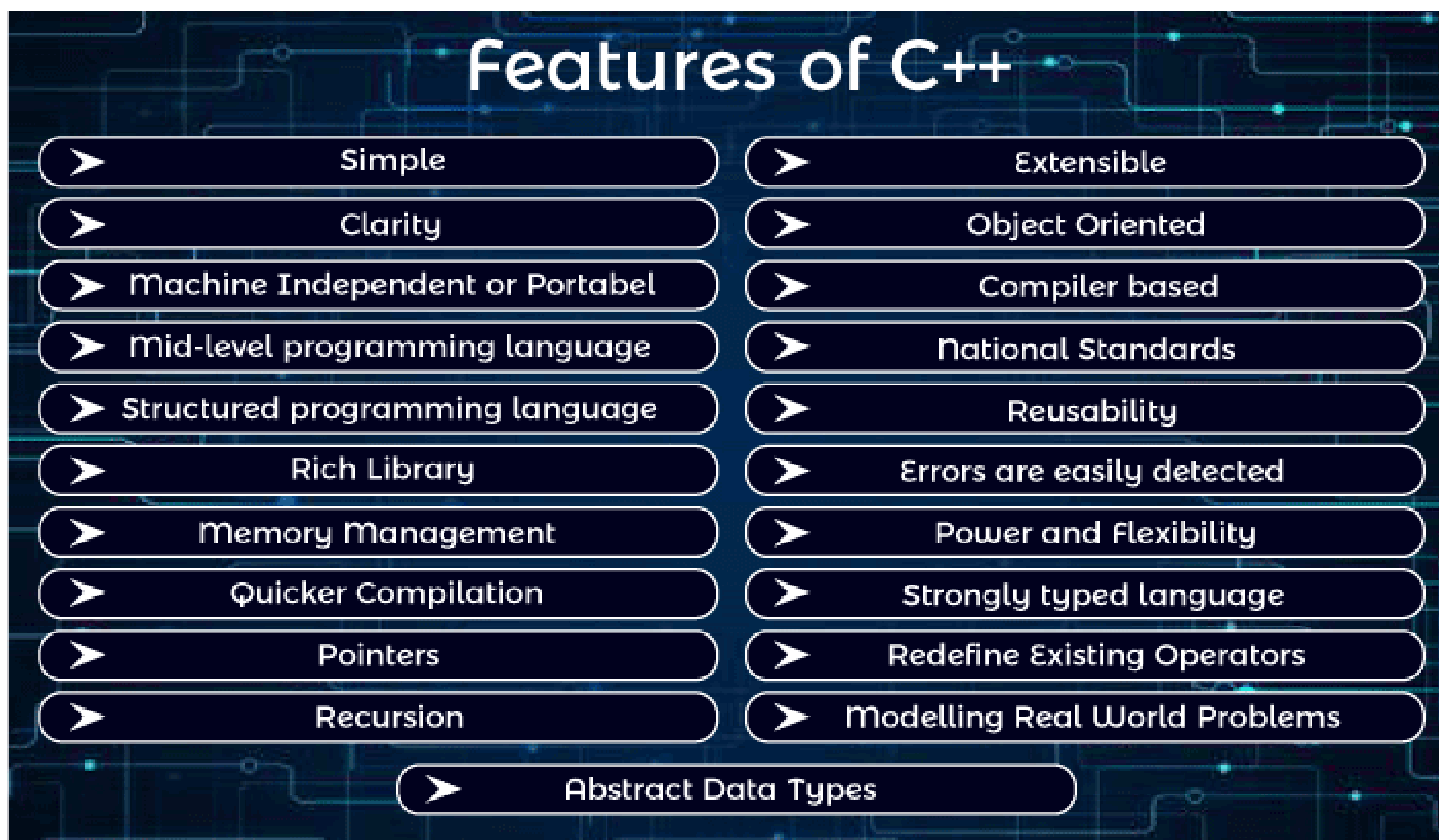
C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

Let's see the programming languages that were developed before C++ language.

Language	Year	Developed By
Algol	1960	International Group
BCPL	1967	Martin Richard
B	1970	Ken Thompson
Traditional C	1972	Dennis Ritchie
K & R C	1978	Kernighan & Dennis Ritchie
C++	1980	Bjarne Stroustrup

C++ Features

C++ is a widely used programming language.



It provides a lot of features that are given below.

1. Simple
2. Abstract Data types
3. Machine Independent or Portable
4. Mid-level programming language
5. Structured programming language
6. Rich Library
7. Memory Management
8. Quicker Compilation
9. Pointers

10. Recursion

11. Extensible

12. Object-Oriented

13. Compiler based

14. Reusability

15. National Standards

16. Errors are easily detected

17. Power and Flexibility

18. Strongly typed language

19. Redefine Existing Operators

20. Modeling Real-World Problems

21. Clarity

1) Simple

C++ is a simple language because it provides a structured approach (to break the problem into parts), a rich set of library functions, data types, etc.

2) Abstract Data types

In C++, complex data types called Abstract Data Types (ADT) can be created using classes.

3) Portable

C++ is a portable language and programs made in it can be run on different machines.

4) Mid-level / Intermediate programming language

C++ includes both low-level programming and high-level language so it is known as a mid-level and intermediate programming language. It is used to develop system applications such as kernel, driver, etc.

5) Structured programming language

C++ is a structured programming language. In this we can divide the program into several parts using functions.

6) Rich Library

C++ provides a lot of inbuilt functions that make the development fast. Following are the libraries used in C++ programming are:

- <iostream>
- <cmath>
- <cstdlib>
- <fstream>

7) Memory Management

C++ provides very efficient management techniques. The various memory management operators help save the memory and improve the program's efficiency. These operators allocate and deallocate memory at run time. Some common memory management operators available C++ are new, delete etc.

8) Quicker Compilation

C++ programs tend to be compact and run quickly. Hence the compilation and execution time of the C++ language is fast.

9) Pointer

C++ provides the feature of pointers. We can use pointers for memory, structures, functions, array, etc. We can directly interact with the memory by using the pointers.

10) Recursion

In C++, we can call the function within the function. It provides code reusability for every function.

11) Extensible

C++ programs can easily be extended as it is very easy to add new features into the existing program.

12) Object-Oriented

In C++, object-oriented concepts like data hiding, encapsulation, and data abstraction can easily be implemented using keyword class, private, public, and protected access specifiers. Object-oriented makes development and maintenance easier.

13) Compiler based

C++ is a compiler-based programming language, which means no C++ program can be executed without compilation. C++ compiler is easily available, and it requires very little space for storage. First, we need to compile our program using a compiler, and then we can execute our program.

14) Reusability

With the use of inheritance of functions programs written in C++ can be reused in any other program of C++. You can save program parts into library files and invoke them in your next programming projects simply by including the library files. New programs can be developed in lesser time as the existing code can be reused. It is also possible to define several functions with same name that perform different task. For Example: `abs()` is used to calculate the absolute value of integer, float and long integer.

15) National Standards

C++ has national standards such as ANSI.

16) Errors are easily detected

It is easier to maintain a C++ programs as errors can be easily located and rectified. It also provides a feature called exception handling to support error handling in your program.

17) Power and Flexibility

C++ is a powerful and flexible language because of most of the powerful flexible and modern UNIX operating system is written in C++. Many compilers and interpreters for

other languages such as FORTRAN, PERL, Python, PASCAL, BASIC, LISP, etc., have been written in C++. C++ programs have been used for solving physics and engineering problems and even for animated special effects for movies.

18) Strongly typed language

The list of arguments of every function call is typed checked during compilation. If there is a type mismatch between actual and formal arguments, implicit conversion is applied if possible. A compile-time occurs if an implicit conversion is not possible or if the number of arguments is incorrect.

19) Redefine Existing Operators

C++ allows the programmer to redefine the meaning of existing operators such as +, -.

For Example, The "+" operator can be used for adding two numbers and concatenating two strings.

20) Modelling real-world problems

The programs written in C++ are well suited for real-world modeling problems as close as possible to the user perspective.

21) Clarity

The keywords and library functions used in C++ resemble common English words.

C++ Program

Before starting the abcd of C++ language, you need to learn how to write, compile and run the first C++ program.

To write the first C++ program, open the C++ console and write the following code:

```
#include <iostream.h>

#include<conio.h>

void main() {

    clrscr();

    cout << "Welcome to C++ Programming.";

    getch();

}
```

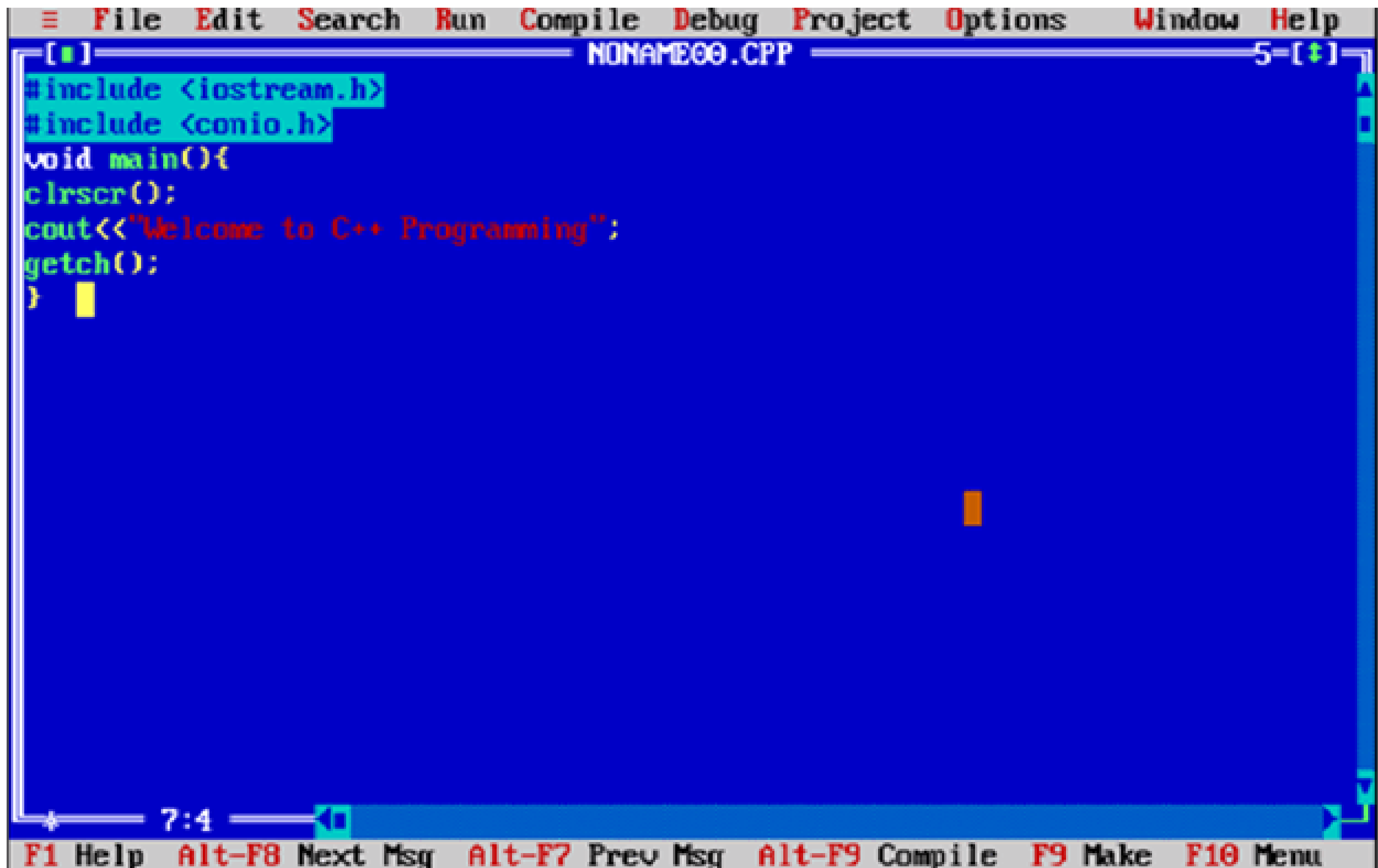
`#include<iostream.h>` includes the standard input output library functions. It provides `cin` and `cout` methods for reading from input and writing to output respectively.

`#include <conio.h>` includes the console input output library functions. The `getch()` function is defined in `conio.h` file.

`void main()` The `main()` function is the entry point of every program in C++ language. The `void` keyword specifies that it returns no value.

`cout << "Welcome to C++ Programming."` is used to print the data "Welcome to C++ Programming." on the console.

getch() The getch() function asks for a single character. Until you press any key, it blocks the screen.



```
File Edit Search Run Compile Debug Project Options Window Help
[ ] NONAME00.CPP 5-[+]
```

```
#include <iostream.h>
#include <conio.h>
void main(){
clrscr();
cout<<"Welcome to C++ Programming";
getch();
}
```

```
7:4
```

```
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu
```

How to compile and run the C++ program

There are 2 ways to compile and run the C++ program, by menu and by shortcut.

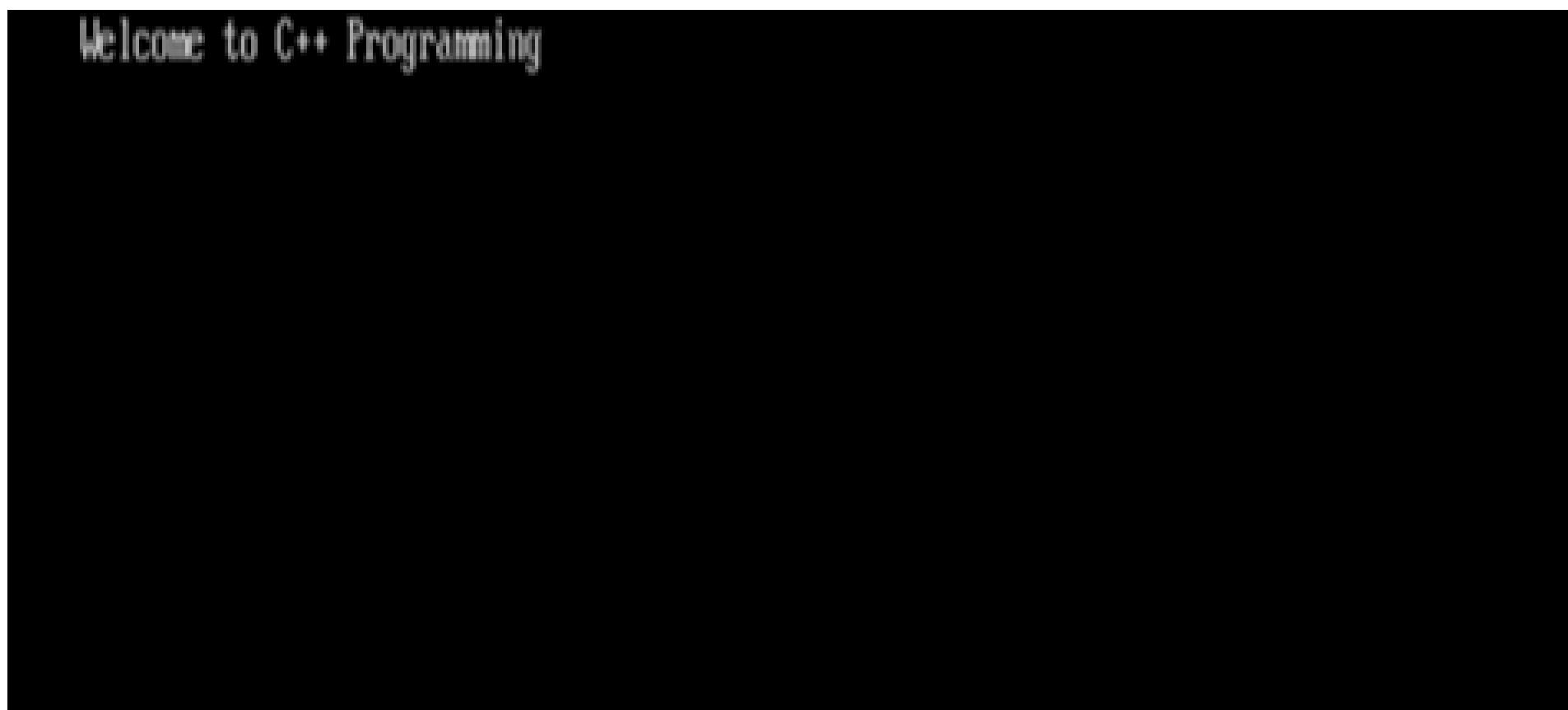
By menu

Now click on the compile menu then compile sub menu to compile the c++ program.

Then click on the run menu then run sub menu to run the c++ program.

By shortcut

Or, press **ctrl+f9** keys compile and run the program directly.



You will see the following output on user screen.

You can view the user screen any time by pressing the **alt+f5** keys.

Now press **Esc** to return to the turbo c++ console.

C++ Basic Input/Output

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.

If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation**.

If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation**.

I/O Library Header Files

Let us see the common header files used in C++ programming are:

Header File	Function and Description
<iostream>	It is used to define the <code>cout</code> , <code>cin</code> and <code>cerr</code> objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<iomanip>	It is used to declare services useful for performing formatted I/O, such as <code>setprecision</code> and <code>setw</code> .
<fstream>	It is used to declare services for user-controlled file processing.

Standard output stream (cout)

The `cout` is a predefined object of `ostream` class. It is connected with the standard output device, which is usually a display screen. The `cout` is used in conjunction with stream insertion operator (`<<`) to display the output on a console

Let's see the simple example of standard output stream (cout):

```
#include <iostream>

using namespace std;

int main( ) {

    char ary[] = "Welcome to C++ tutorial";

    cout << "Value of ary is: " << ary << endl;
```

```
}
```

Output:

```
Value of ary is: Welcome to C++ tutorial
```

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The **cin** is used in conjunction with stream extraction operator (**>>**) to read the input from a console.

Let's see the simple example of standard input stream (cin):

```
#include <iostream>

using namespace std;

int main( ) {

    int age;

    cout << "Enter your age: ";

    cin >> age;

    cout << "Your age is: " << age << endl;

}
```

Output:

```
Enter your age: 22
```

```
Your age is: 22
```

Standard end line (endl)

The `endl` is a predefined object of `ostream` class. It is used to insert a new line characters and flushes the stream.

Let's see the simple example of standard end line (`endl`):

```
#include <iostream>

using namespace std;

int main( ) {

    cout << "C++ Tutorial";

    cout << " Javatpoint" << endl;

    cout << "End of line" << endl;

}
```

Output:

```
C++ Tutorial Javatpoint

End of line
```

C++ Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

```
type variable_list;
```

The example of declaring variable is given below:

i. `int x;`

ii. `float y;`

iii. `char z;`

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below:

i. `int x=5,b=10; //declaring 2 variable of integer type`

ii. `float f=30.8;`

iii. `char c='A';`

Rules for defining variables

A variable can have alphabets, digits and underscore.

A variable name can start with alphabet and underscore only. It can't start with digit.

No white space is allowed within variable name.

A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

i. `int a;`

ii. `int _ab;`

iii. `int a30;`

Invalid variable names:

i. `int 4;`

ii. `int x y;`

iii. `int double;`

C++ Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.



Data Types in C++

There are 4 types of data types in C++ language.

Types	Data Types
Basic Data Type	int, char, float, double, etc
Derived Data Type	array, pointer, etc
Enumeration Data Type	enum
User Defined Data Type	structure

Basic Data Types

The basic data types are integer-based and floating-point based. C++ language supports both signed and unsigned literals.

The memory size of basic data types may change according to 32 or 64 bit operating system.

Let's see the basic data types. It size is given according to 32 bit OS.

Data Types	Memory Size	Range
Char	1 byte	-128 to 127
signed char	1 byte	-128 to 127

unsigned char	1 byte	0 to 127
Short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 32,767
Int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 32,767
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 32,767
long int	4 byte	
signed long int	4 byte	
unsigned long int	4 byte	

Float	4 byte	
Double	8 byte	
long double	10 byte	

C++ Keywords

A keyword is a reserved word. You cannot use it as a variable name, constant name etc. A list of 32 Keywords in C++ Language which are also available in C language are given below.

auto	break	case	char	const	continue	default	do
double	else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language which are not available in C language are given below.

asm	dynamic_cast	namespace	reinterpret_cast	bool
-----	--------------	-----------	------------------	------

explicit	new	static_cast	false	catch
operator	template	friend	private	class
this	inline	public	throw	const_cast
delete	mutable	protected	true	try
typeid	typename	using	virtual	wchar_t

C++ Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C language.

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Bitwise Operators
- Assignment Operator

- Unary operator
- Ternary or Conditional Operator
- Misc Operator

	Operator	Type
Binary Operator	+, -, *, /, %	Arithmetic Operators
	<, <=, >, >=, ==, !=	Relational Operators
	&&, , !	Logical Operators
	&, , <<, >>, ~, ^	Bitwise Operators
	=, +=, -=, *=, /=, %=	Assignment Operators
Unary Operator	→ ++, --	Unary Operator
Ternary Operator	→ ?:	Ternary or Conditional Operator

Precedence of Operators in C++

The precedence of operator specifies that which operator will be evaluated first and next.

The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below:

```
int data=5+10*10;
```

The "data" variable will contain 105 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C++ operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type) * & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Right to left
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Right to left
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right

Bitwise OR		Right to left
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

C++ Identifiers

C++ identifiers in a program are used to refer to the name of the variables, functions, arrays, or other user-defined data types created by the programmer. They are the basic requirement of any language. Every language has its own rules for naming the identifiers.

In short, we can say that the C++ identifiers represent the essential elements in a program which are given below:

- Constants
- Variables
- Functions

- Labels
- Defined data types

Some naming rules are common in both C and C++. They are as follows:

- Only alphabetic characters, digits, and underscores are allowed.
- The identifier name cannot start with a digit, i.e., the first letter should be alphabetical. After the first letter, we can use letters, digits, or underscores.
- In C++, uppercase and lowercase letters are distinct. Therefore, we can say that C++ identifiers are case-sensitive.
- A declared keyword cannot be used as a variable name.

For example, suppose we have two identifiers, named as 'FirstName', and 'Firstname'. Both the identifiers will be different as the letter 'N' in the first case is in uppercase while lowercase in second. Therefore, it proves that identifiers are case-sensitive.

Valid Identifiers

The following are the examples of valid identifiers:

- Result
- Test2
- _sum
- power

Invalid Identifiers

The following are the examples of invalid identifiers:

- i. `Sum-1` // containing special character '-'.
- ii. `2data` // the first letter is a digit.
- iii. `break` // use of a keyword.

Note: Identifiers cannot be used as the keywords. It may not conflict with the keywords, but it is highly recommended that the keywords should not be used as the identifier name. You should always use a consistent way to name the identifiers so that your code will be more readable and maintainable.

The major difference between C and C++ is the limit on the length of the name of the variable. ANSI C considers only the first 32 characters in a name while ANSI C++ imposes no limit on the length of the name.

Constants are the identifiers that refer to the fixed value, which do not change during the execution of a program. Both C and C++ support various kinds of literal constants, and they do have any memory location. For example, 123, 12.34, 037, 0X2, etc. are the literal constants.

Let's look at a simple example to understand the concept of identifiers.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```

int a;

int A;

cout<<"Enter the values of 'a' and 'A'";

cin>>a;

cin>>A;

cout<<"\nThe values that you have entered are : "<<a<<" , "<<A;

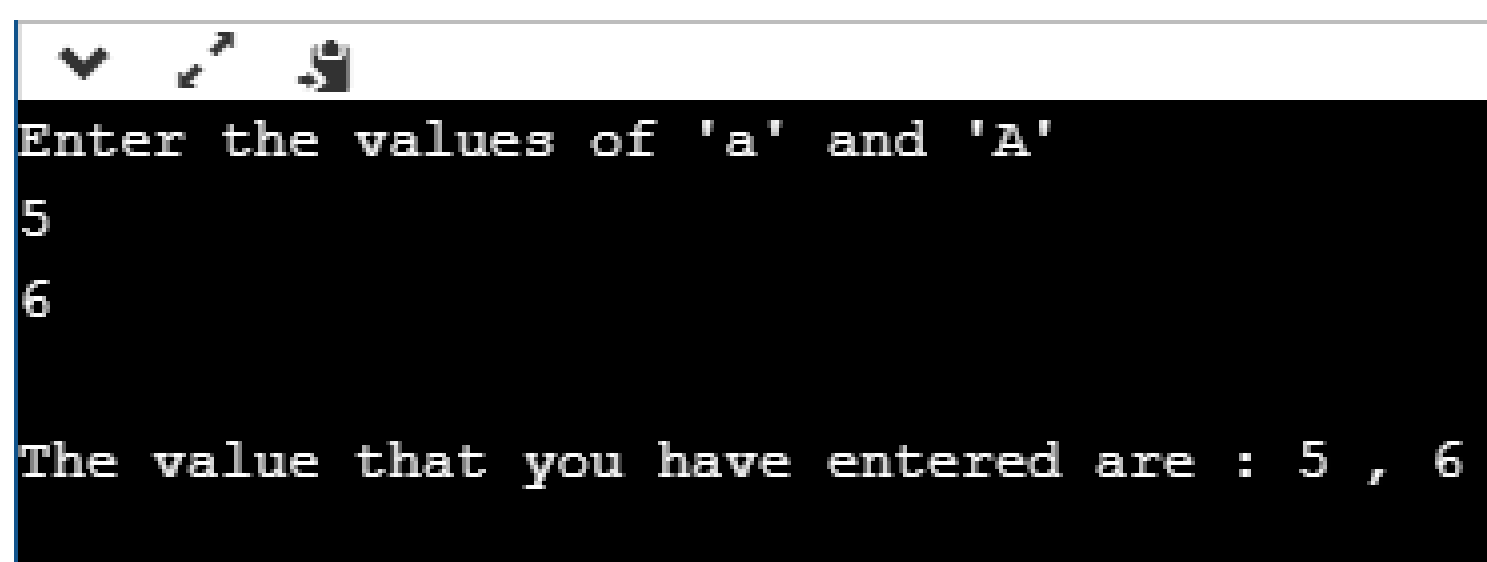
return 0;

}

```

In the above code, we declare two variables 'a' and 'A'. Both the letters are same but they will behave as different identifiers. As we know that the identifiers are the case-sensitive so both the identifiers will have different memory locations.

Output



```

Enter the values of 'a' and 'A'
5
6

The value that you have entered are : 5 , 6

```

What are the keywords?

Keywords are the reserved words that have a special meaning to the compiler. They are reserved for a special purpose, which cannot be used as the identifiers. For example, 'for', 'break', 'while', 'if', 'else', etc. are the predefined words where predefined words are those words whose meaning is already known by the compiler. Whereas, the identifiers are the names which are defined by the programmer to the program elements such as variables, functions, arrays, objects, classes.

Differences between Identifiers and Keywords

The following is the list of differences between identifiers and keywords:

Identifiers	Keywords
Identifiers are the names defined by the programmer to the basic elements of a program.	Keywords are the reserved words whose meaning is known by the compiler.
It is used to identify the name of the variable.	It is used to specify the type of entity.
It can consist of letters, digits, and underscore.	It contains only letters.
It can use both lowercase and uppercase letters.	It uses only lowercase letters.
No special character can be used except the underscore.	It cannot contain any special character.

The starting letter of identifiers can be lowercase, uppercase or underscore.	It can be started only with the lowercase letter.
It can be classified as internal and external identifiers.	It cannot be further classified.
Examples are test, result, sum, power, etc.	Examples are 'for', 'if', 'else', 'break', etc.

C++ Expression

C++ expression consists of operators, constants, and variables which are arranged according to the rules of the language. It can also contain function calls which return values.

An expression can consist of one or more operands, zero or more operators to compute a value. Every expression produces some value which is assigned to the variable with the help of an assignment operator.

Examples of C++ expression:

i. $(a+b) - c$

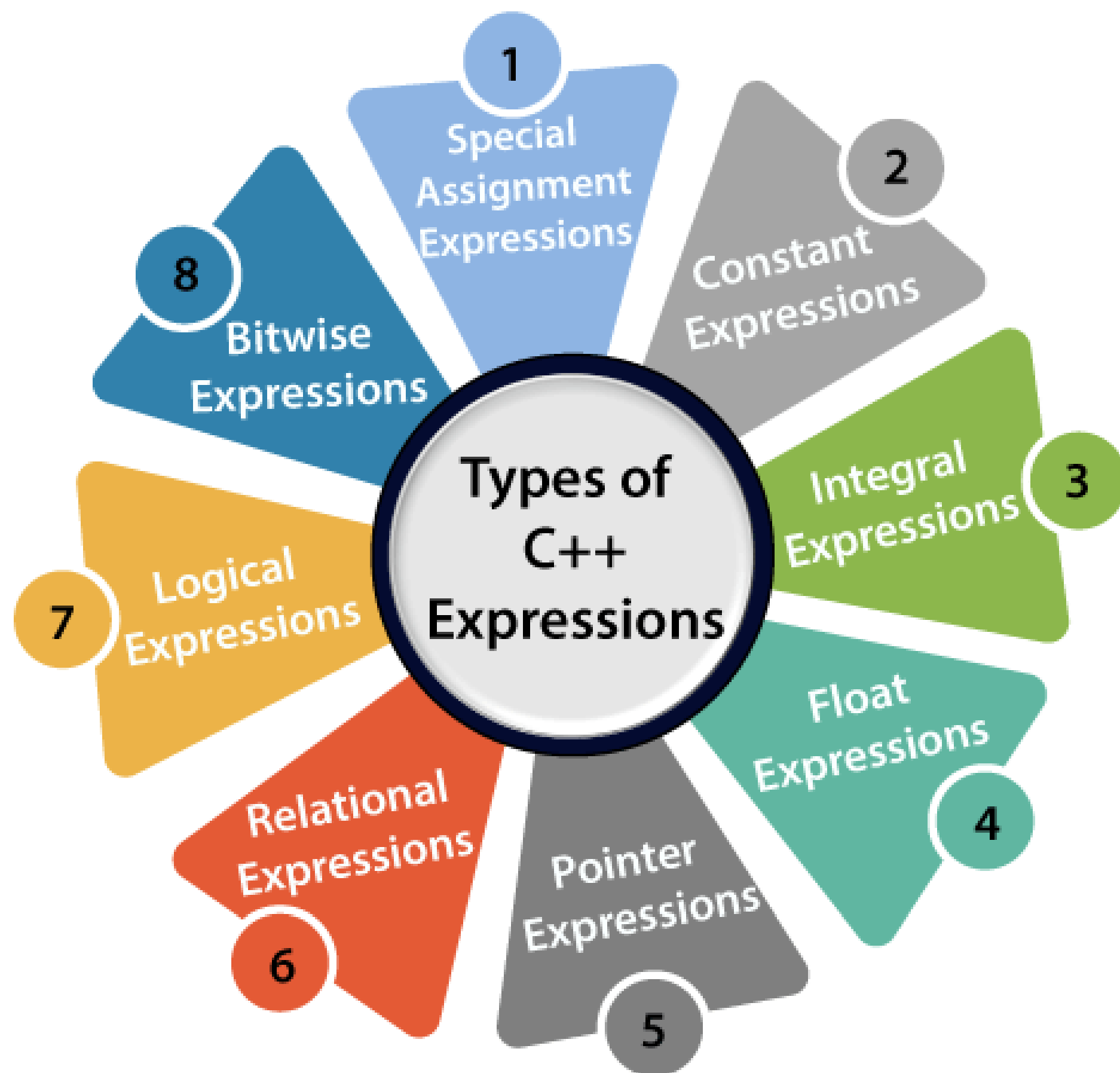
ii. $(x/y) - z$

iii. $4a^2 - 5b + c$

iv. $(a+b) * (x+y)$

An expression can be of following types:

- Constant expressions
- Integral expressions
- Float expressions
- Pointer expressions
- Relational expressions
- Logical expressions
- Bitwise expressions
- Special assignment expressions



If the expression is a combination of the above expressions, such expressions are known as compound expressions.

Constant expressions

A constant expression is an expression that consists of only constant values. It is an expression whose value is determined at the compile-time but evaluated at the run-time. It can be composed of integer, character, floating-point, and enumeration constants.

Constants are used in the following situations:

- It is used in the subscript declarator to describe the array bound.
- It is used after the case keyword in the switch statement.

- It is used as a numeric value in an **enum**
- It specifies a bit-field width.
- It is used in the pre-processor **#if**

In the above scenarios, the constant expression can have integer, character, and enumeration constants. We can use the static and extern keyword with the constants to define the function-scope.

The following table shows the expression containing constant value:

Expression containing constant	Constant value
<code>x = (2/3) * 4</code>	<code>(2/3) * 4</code>
<code>extern int y = 67</code>	<code>67</code>
<code>int z = 43</code>	<code>43</code>
<code>static int a = 56</code>	<code>56</code>

Let's see a simple program containing constant expression:

```
#include <iostream>

using namespace std;

int main()
```

```

{

    int x;    // variable declaration.

    x=(3/2) + 2; // constant expression

    cout<<"Value of x is : "<<x; // displaying the value of x.

    return 0;

}

```

In the above code, we have first declared the 'x' variable of integer type. After declaration, we assign the simple constant expression to the 'x' variable.

Output

Value of x is : 3

Integral Expressions

An integer expression is an expression that produces the integer value as output after performing all the explicit and implicit conversions.

Following are the examples of integral expression:

- i. $(x * y) - 5$
- ii. $x + \text{int}(9.0)$

where x and y are the integers.

Let's see a simple example of integral expression:

```

#include <iostream>

using namespace std;

int main()

{

    int x; // variable declaration.

    int y; // variable declaration

    int z; // variable declaration

    cout<<"Enter the values of x and y";

    cin>>x>>y;

    z=x+y;

    cout<<"\n"<<"Value of z is :"<<z; // displaying the value of z.

    return 0;

}

```

In the above code, we have declared three variables, i.e., x, y, and z. After declaration, we take the user input for the values of 'x' and 'y'. Then, we add the values of 'x' and 'y' and stores their result in 'z' variable.

Output

```
Enter the values of x and y
```

```
8
```

9

Value of z is :17

Let's see another example of integral expression.

```
#include <iostream>

using namespace std;

int main()

{

    int x; // variable declaration

    int y=9; // variable initialization

    x=y+int(10.0); // integral expression

    cout<<"Value of x : "<<x; // displaying the value of x.

    return 0;

}
```

In the above code, we declare two variables, i.e., x and y. We store the value of expression (y+int(10.0)) in a 'x' variable.

Output

Value of x : 19

Float Expressions

A float expression is an expression that produces floating-point value as output after performing all the explicit and implicit conversions.

The following are the examples of float expressions:

- i. $x+y$
- ii. $(x/10) + y$
- iii. 34.5
- iv. $x+\text{float}(10)$

Let's understand through an example.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    float x=8.9;    // variable initialization
```

```
    float y=5.6;    // variable initialization
```

```
    float z;        // variable declaration
```

```
    z=x+y;
```

```
    std::cout <<"value of z is :" << z<<std::endl; // displaying the value of z.
```

```
    return 0;

}
```

Output

```
value of z is :14.5
```

Let's see another example of float expression.

```
#include <iostream>

using namespace std;

int main()

{

    float x=6.7;  // variable initialization

    float y;      // variable declaration

    y=x+float(10); // float expression

    std::cout <<"value of y is :" << y<<std::endl; // displaying the value of y

    return 0;

}
```

In the above code, we have declared two variables, i.e., x and y. After declaration, we store the value of expression `(x+float(10))` in variable 'y'.

Output

```
value of y is :16.7
```

Pointer Expressions

A pointer expression is an expression that produces address value as an output.

The following are the examples of pointer expression:

- i. `&x`
- ii. `ptr`
- iii. `ptr++`
- iv. `ptr--`

Let's understand through an example.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int a[]={1,2,3,4,5}; // array initialization
```

```

int *ptr;    // pointer declaration

ptr=a;    // assigning base address of array to the pointer ptr

ptr=ptr+1;  // incrementing the value of pointer

std::cout <<"value of second element of an array : " << *ptr<<std::endl;

return 0;

}

```

In the above code, we declare the array and a pointer ptr. We assign the base address to the variable 'ptr'. After assigning the address, we increment the value of pointer 'ptr'. When pointer is incremented then 'ptr' will be pointing to the second element of the array.

Output

```

value of second element of an array : 2

```

Relational Expressions

A relational expression is an expression that produces a value of type bool, which can be either true or false. It is also known as a boolean expression. When arithmetic expressions are used on both sides of the relational operator, arithmetic expressions are evaluated first, and then their results are compared.

The following are the examples of the relational expression:

- i. $a > b$
- ii. $a - b \geq x - y$

iii. $a+b>80$

Let's understand through an example

```
#include <iostream>

using namespace std;

int main()

{

    int a=45;  // variable declaration

    int b=78;  // variable declaration

    bool y= a>b;  // relational expression

    cout<<"Value of y is :"<<y;  // displaying the value of y.

    return 0;

}
```

In the above code, we have declared two variables, i.e., 'a' and 'b'. After declaration, we have applied the relational operator between the variables to check whether 'a' is greater than 'b' or not.

Output

Value of y is :0

Let's see another example.

```

#include <iostream>

using namespace std;

int main()

{

    int a=4;    // variable declaration

    int b=5;    // variable declaration

    int x=3;    // variable declaration

    int y=6;    // variable declaration

    cout<<((a+b)>=(x+y)); // relational expression

    return 0;

}

```

In the above code, we have declared four variables, i.e., 'a', 'b', 'x' and 'y'. Then, we apply the relational operator ($\geq$) between these variables.

Output

1

Logical Expressions

A logical expression is an expression that combines two or more relational expressions and produces a bool type value. The logical operators are '&&' and '||' that combines two or more relational expressions.

The following are some examples of logical expressions:

- i. `a > b && x > y`
- ii. `a > 10 || b == 5`

Let's see a simple example of logical expression.

```
#include <iostream>

using namespace std;

int main()
{

    int a=2;

    int b=7;

    int c=4;

    cout<<((a>b)|| (a>c));

    return 0;

}
```

Output

0

Bitwise Expressions

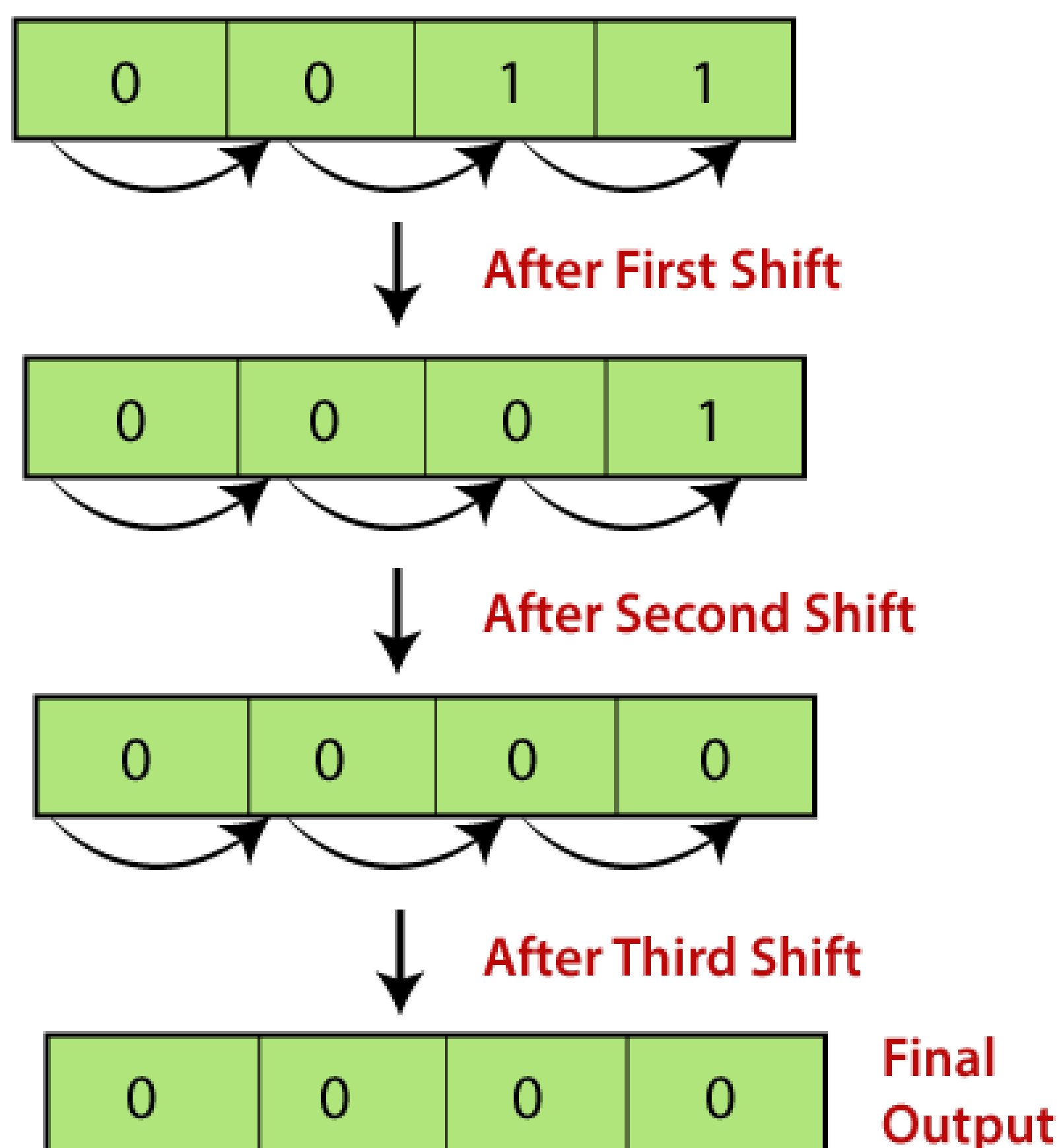
A bitwise expression is an expression which is used to manipulate the data at a bit level. They are basically used to shift the bits.

Forexample:

$x=3$

$x \gg 3$ // This statement means that we are shifting the three-bit position to the right.

In the above example, the value of 'x' is 3 and its binary value is 0011. We are shifting the value of 'x' by three-bit position to the right. Let's understand through the diagrammatic representation.



Let's see a simple example.

```
#include <iostream>

using namespace std;

int main()

{

    int x=5; // variable declaration

    std::cout << (x>>1) << std::endl;

    return 0;

}
```

In the above code, we have declared a variable 'x'. After declaration, we applied the bitwise operator, i.e., right shift operator to shift one-bit position to right.

Output

2

Let's look at another example.

```
#include <iostream>

using namespace std;

int main()

{
```

```
int x=7; // variable declaration
```

```
std::cout << (x<<3) << std::endl;
```

```
return 0;
```

```
}
```

In the above code, we have declared a variable 'x'. After declaration, we applied the left shift operator to variable 'x' to shift the three-bit position to the left.

Output

56

Special Assignment Expressions

Special assignment expressions are the expressions which can be further classified depending upon the value assigned to the variable.

Chained Assignment

Chained assignment expression is an expression in which the same value is assigned to more than one variable by using single statement.

For example:

a=b=20

or

(a=b) = 20

Let's understand through an example.

```
#include <iostream>

using namespace std;

int main()

{
    int a; // variable declaration

    int b; // variable declaration

    a=b=80; // chained assignment

    std::cout << "Values of 'a' and 'b' are : " << a << ", " << b << std::endl;

    return 0;

}
```

In the above code, we have declared two variables, i.e., 'a' and 'b'. Then, we have assigned the same value to both the variables using chained assignment expression.

Output

Values of 'a' and 'b' are : 80,80

Note: Using chained assignment expression, the value cannot be assigned to the variable at the time of declaration. For example, `int a=b=c=90` is an invalid statement.

Embedded Assignment Expression

An embedded assignment expression is an assignment expression in which assignment expression is enclosed within another assignment expression.

Let's understand through an example.

```
#include <iostream>

using namespace std;

int main()

{

    int a; // variable declaration

    int b; // variable declaration

    a=10+(b=90); // embedded assignment expression

    std::cout <<"Values of 'a' is " <<a<< std::endl;

    return 0;

}
```

In the above code, we have declared two variables, i.e., 'a' and 'b'. Then, we applied embedded assignment expression ($a=10+(b=90)$).

Output

```
Values of 'a' is 100
```

Compound Assignment

A compound assignment expression is an expression which is a combination of an assignment operator and binary operator.

For example,

```
a+=10;
```

In the above statement, 'a' is a variable and '+=' is a compound statement.

Let's understand through an example.

```
#include <iostream>

using namespace std;

int main()
{

    int a=10; // variable declaration

    a+=10; // compound assignment

    std::cout << "Value of a is :" <<a<< std::endl; // displaying the value of a.

    return 0;

}
```

In the above code, we have declared a variable 'a' and assigns 10 value to this variable. Then, we applied compound assignment operator (+=) to 'a' variable, i.e., a+=10 which is equal to (a=a+10). This statement increments the value of 'a' by 10.

Output

Value of a is :20

C++ Control Statement

C++ if-else

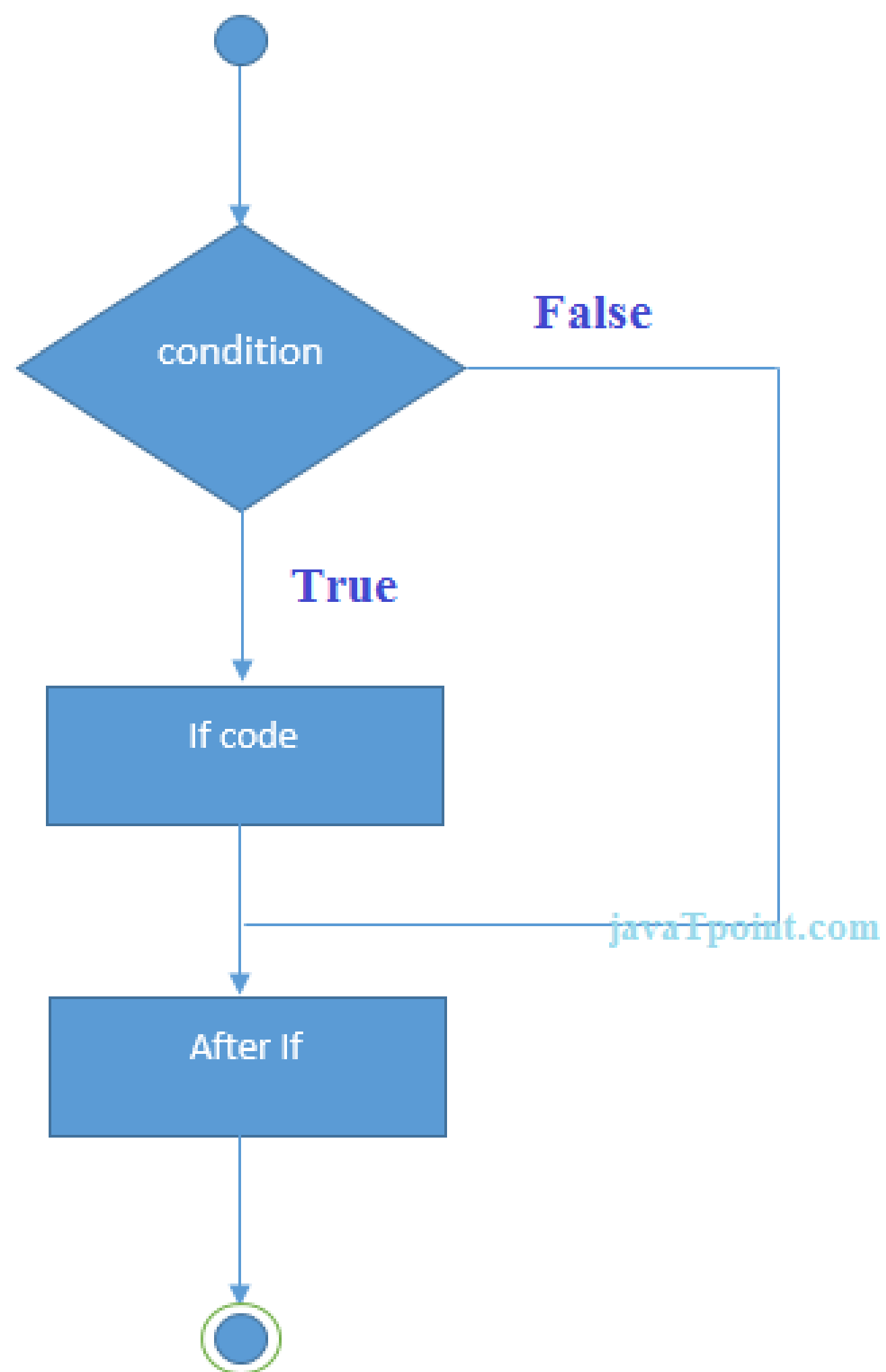
In C++ programming, if statement is used to test the condition. There are various types of if statements in C++.

- if statement
- if-else statement
- nested if statement
- if-else-if ladder

C++ IF Statement

The C++ if statement tests the condition. It is executed if condition is true.

```
if(condition){  
  
    //code to be executed  
  
}
```



C++ If Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int num = 10;
```

```
    if (num % 2 == 0)
```

```

    {

        cout<<"It is even number";

    }

    return 0;

}

```

Output: /p>

It is even number

C++ IF-else Statement

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

```

if(condition){

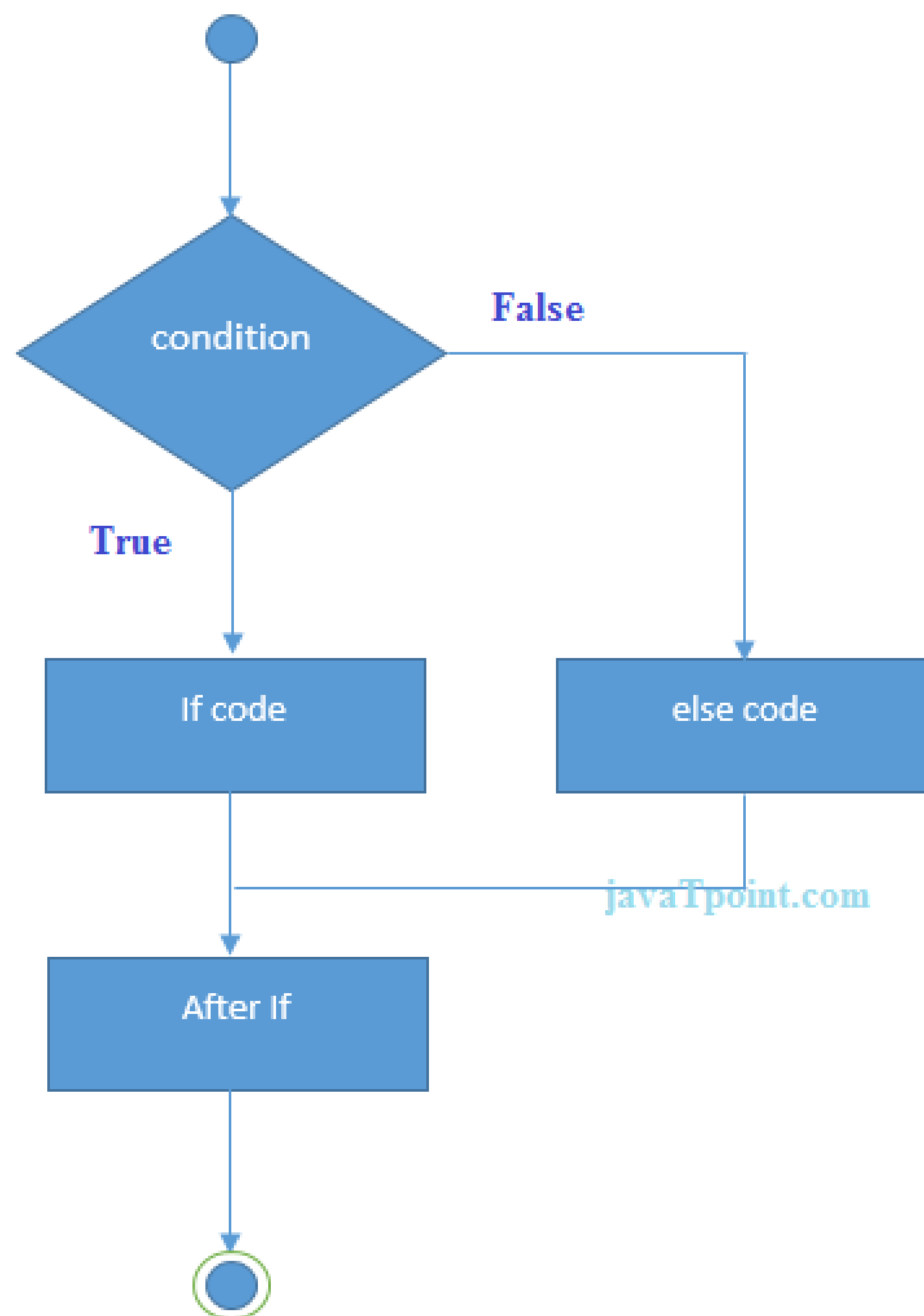
    //code if condition is true

}else{

    //code if condition is false

}

```



C++ If-else Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int num = 11;
```

```
    if (num % 2 == 0)
```

```
{
```

```

        cout<<"It is even number";

    }

    else

    {

        cout<<"It is odd number";

    }

    return 0;

}

```

Output:

It is odd number

C++ If-else Example: with input from user

```

#include <iostream>

using namespace std;

int main () {

    int num;

    cout<<"Enter a Number: ";

    cin>>num;

    if (num % 2 == 0)

```

```

    {

        cout<<"It is even number"<<endl;

    }

    else

    {

        cout<<"It is odd number"<<endl;

    }

    return 0;

}

```

Output:

Enter a number:11

It is odd number

Output:

Enter a number:12

It is even number

C++ IF-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

```

if(condition1){

```

```
//code to be executed if condition1 is true

}else if(condition2){

//code to be executed if condition2 is true

}

else if(condition3){

//code to be executed if condition3 is true

}

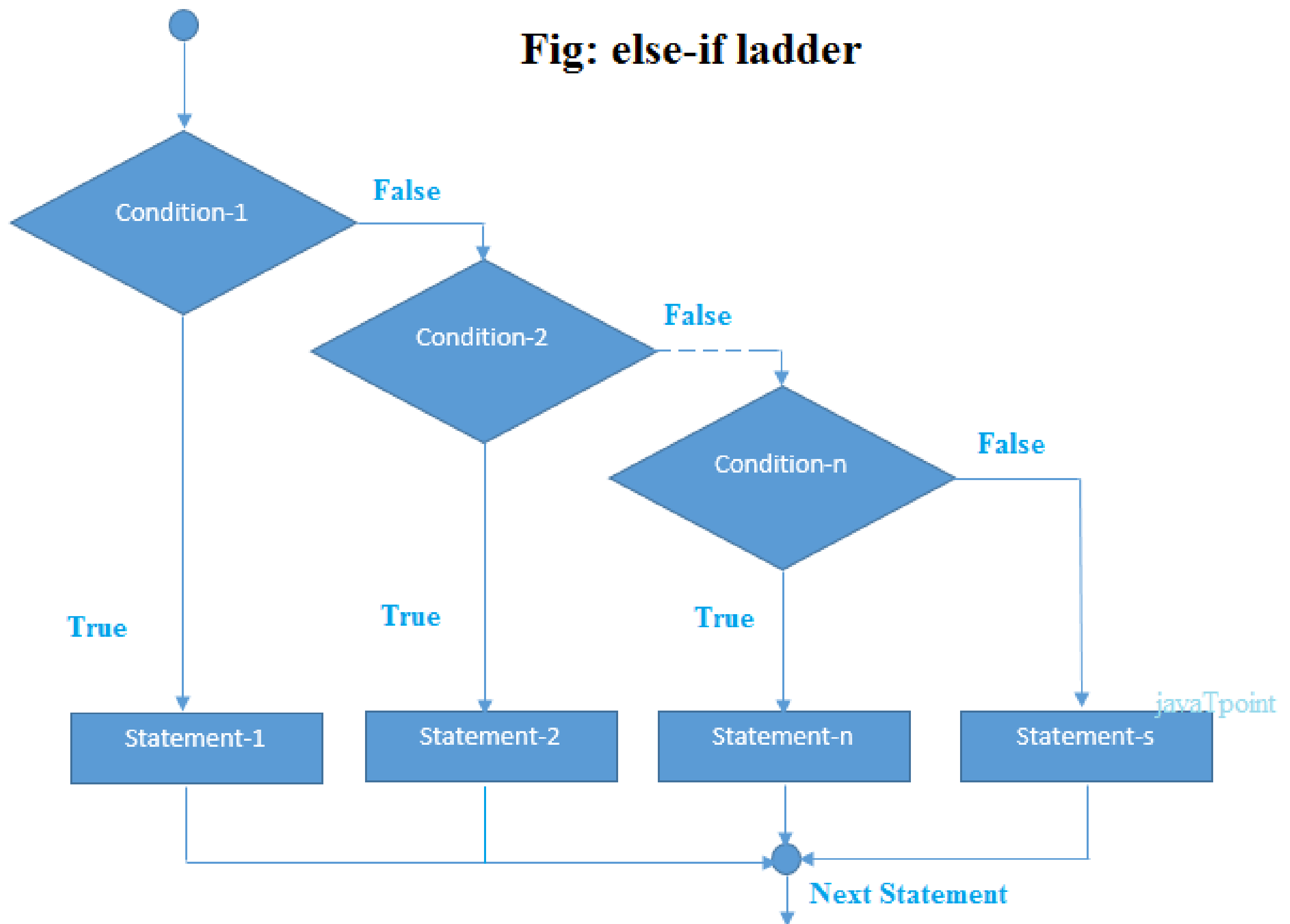
...

else{

//code to be executed if all the conditions are false

}
```

Fig: else-if ladder



C++ If else-if Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int num;
```

```
    cout<<"Enter a number to check grade:";
```

```
    cin>>num;
```

```
    if (num <0 || num >100)
```

```
{

    cout<<"wrong number";

}

else if(num >= 0 && num < 50){

    cout<<"Fail";

}

else if (num >= 50 && num < 60)

{

    cout<<"D Grade";

}

else if (num >= 60 && num < 70)

{

    cout<<"C Grade";

}

else if (num >= 70 && num < 80)

{

    cout<<"B Grade";

}
```

```

else if (num >= 80 && num < 90)

{

    cout<<"A Grade";

}

else if (num >= 90 && num <= 100)

{

    cout<<"A+ Grade";

}

}

```

Output:

Enter a number to check grade:66

C Grade

Output:

Enter a number to check grade:-2

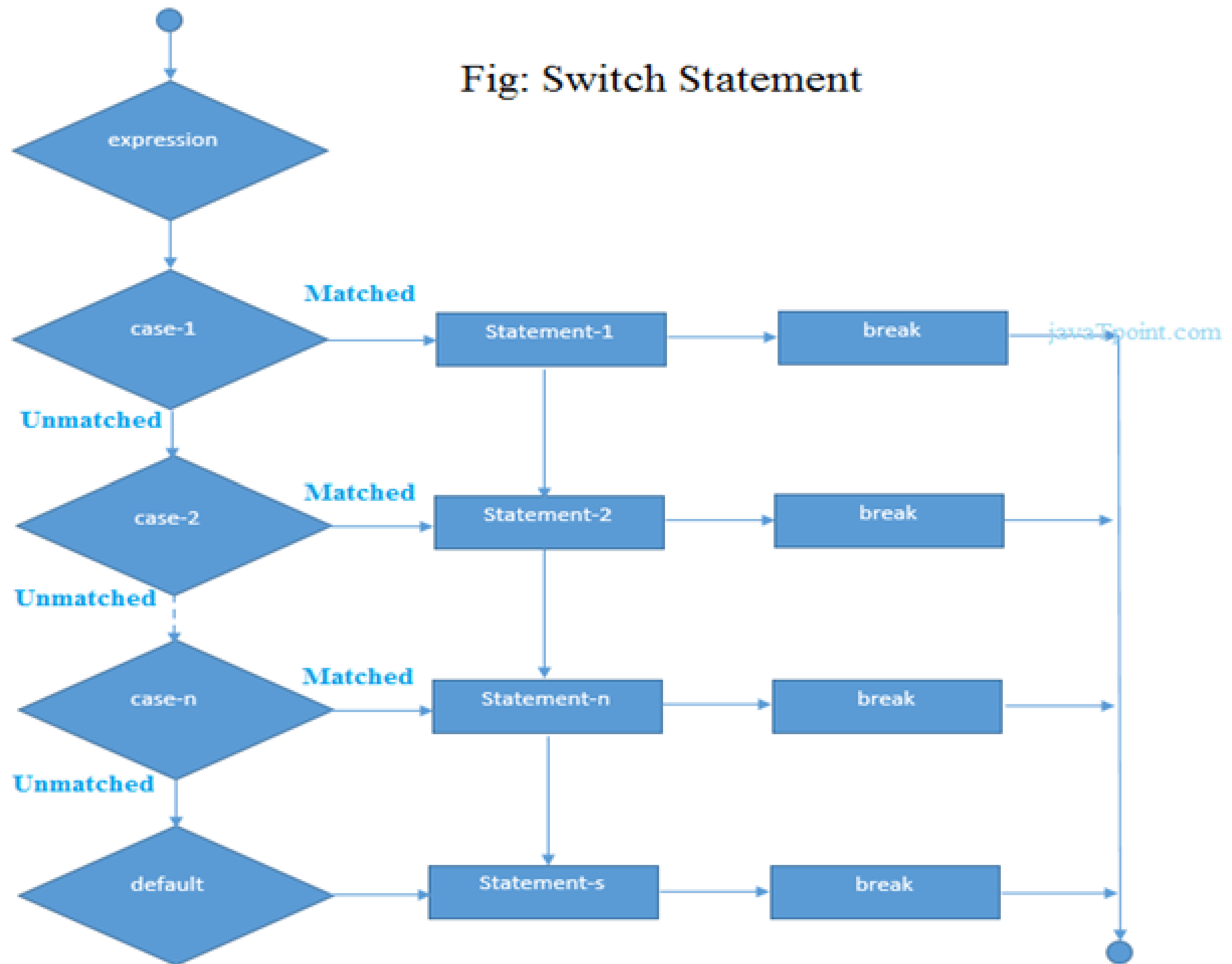
wrong number

C++ switch

The C++ switch statement executes one statement from multiple conditions. It is like if-else-if ladder statement in C++.

```
switch(expression){  
  
    case value1:  
  
        //code to be executed;  
  
        break;  
  
    case value2:  
  
        //code to be executed;  
  
        break;  
  
    .....  
  
    default:  
  
        //code to be executed if all cases are not matched;  
  
        break;  
  
}
```

Fig: Switch Statement



C++ Switch Example

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int num;
```

```
    cout<<"Enter a number to check grade:";
```

```
    cin>>num;
```

```
    switch (num)
```

```
{  
  
    case 10: cout<<"It is 10"; break;  
  
    case 20: cout<<"It is 20"; break;  
  
    case 30: cout<<"It is 30"; break;  
  
    default: cout<<"Not 10, 20 or 30"; break;  
  
}  
  
}
```

Output:

Enter a number:

10

It is 10

Output:

Enter a number:

55

Not 10, 20 or 30

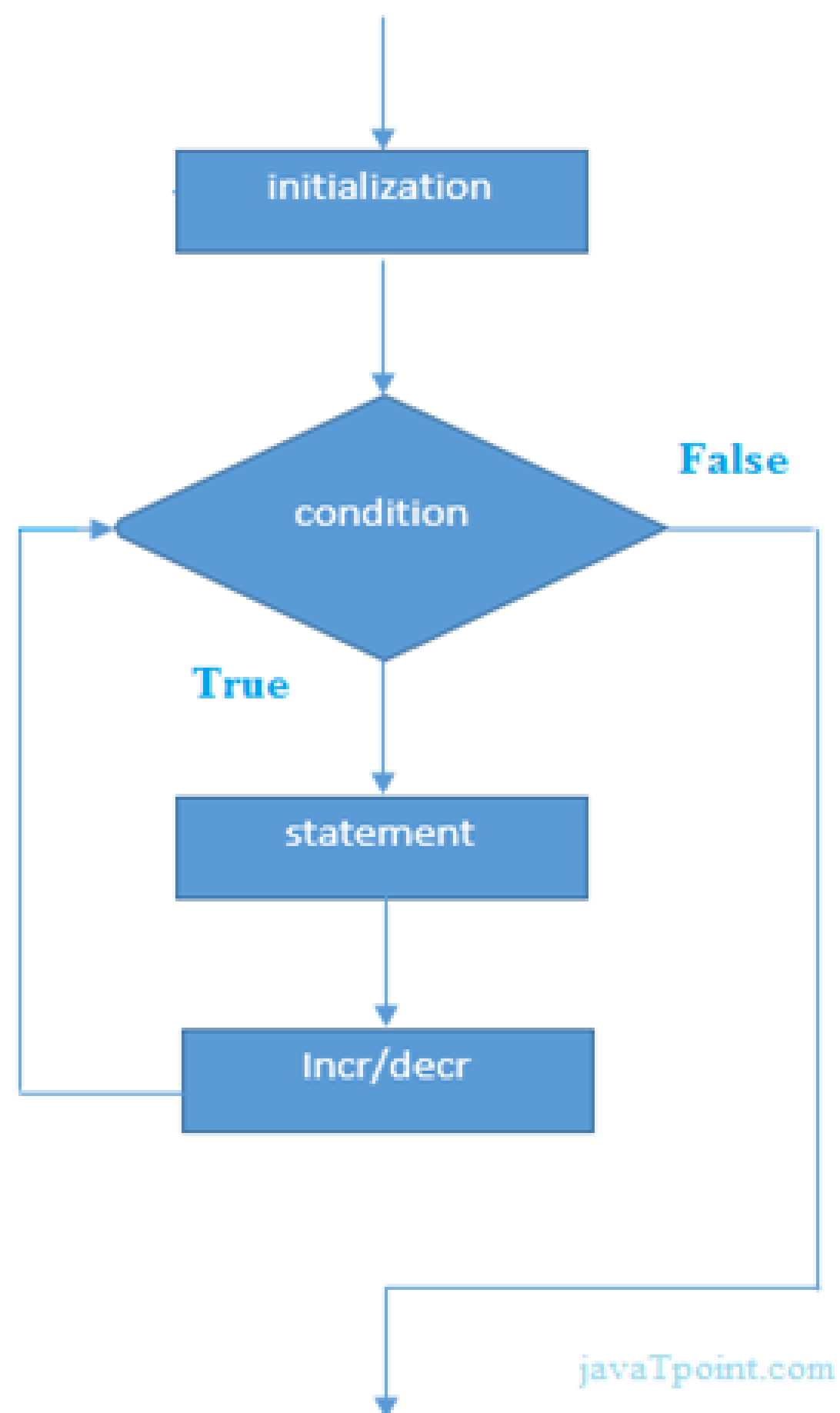
C++ For Loop

The C++ for loop is used to iterate a part of the program several times. If the number of iteration is fixed, it is recommended to use for loop than while or do-while loops.

The C++ for loop is same as C/C#. We can initialize variable, check condition and increment/decrement value.

```
for(initialization; condition; incr/decr){  
  
    //code to be executed  
  
}
```

Flowchart:



C++ For Loop Example

```
#include <iostream>
```

```
using namespace std;

int main() {

    for(int i=1;i<=10;i++){

        cout<<i <<"\n";

    }

}
```

Output:

```
1
2
3
4
5
6
7
8
9
10
```

C++ Nested For Loop

In C++, we can use for loop inside another for loop, it is known as nested for loop. The inner loop is executed fully when outer loop is executed one time. So if outer loop and inner loop

are executed 4 times, inner loop will be executed 4 times for each outer loop i.e. total 16 times.

C++ Nested For Loop Example

Let's see a simple example of nested for loop in C++.

```
#include <iostream>

using namespace std;

int main ( ) {

    for(int i=1;i<=3;i++){

        for(int j=1;j<=3;j++){

            cout<<i<<" "<<j<<"\n";

        }

    }

}
```

Output:

```
1 1
1 2
1 3
2 1
```

2 2

2 3

3 1

3 2

3 3

C++ Infinite For Loop

If we use double semicolon in for loop, it will be executed infinite times. Let's see a simple example of infinite for loop in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main ( ) {
```

```
    for ( ;; )
```

```
    {
```

```
        cout<<"Infinitive For Loop";
```

```
    }
```

```
}
```

Output:

Infinitive For Loop

Infinite For Loop

Infinite For Loop

Infinite For Loop

Infinite For Loop

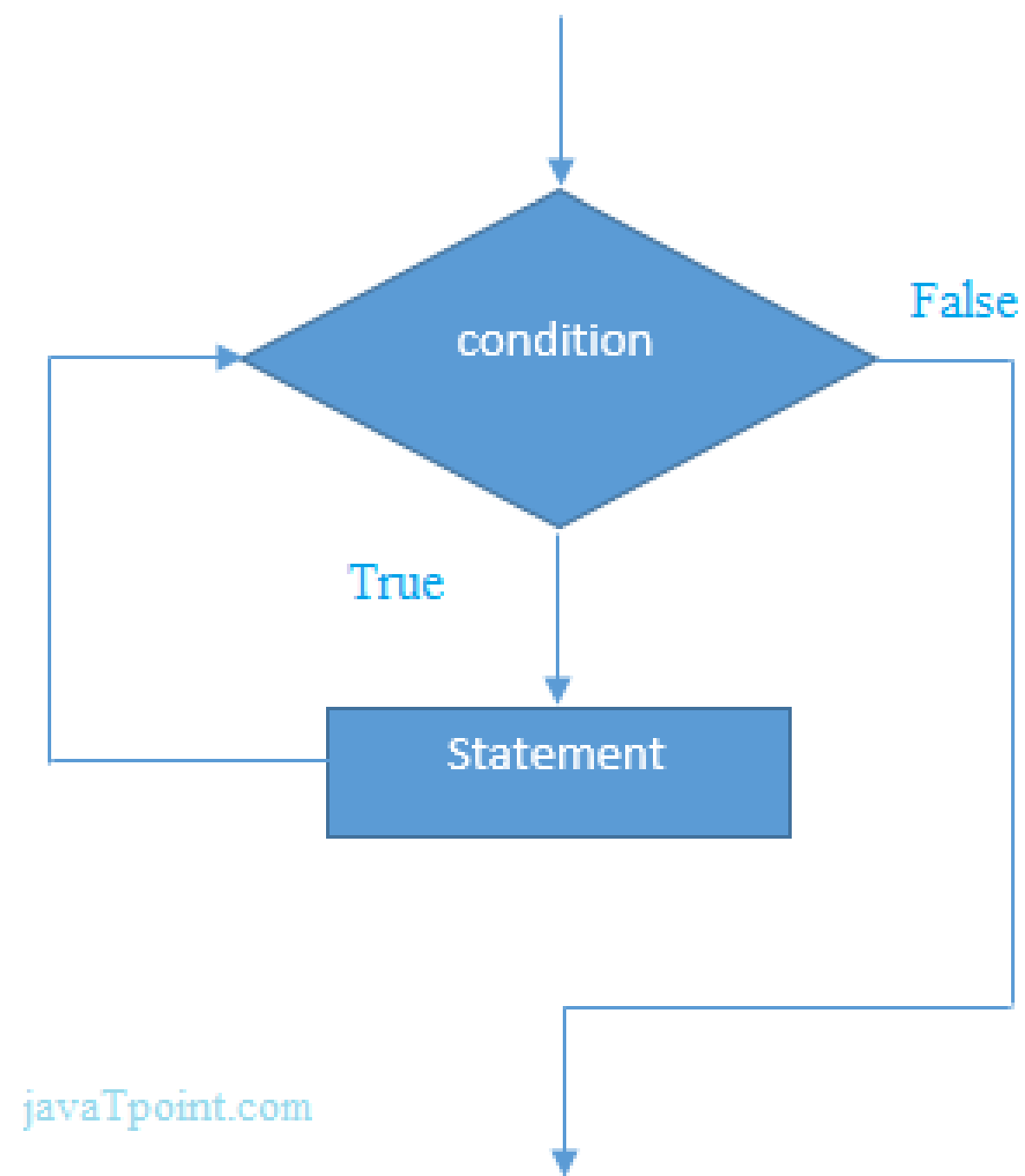
ctrl+c

C++ While loop

In C++, while loop is used to iterate a part of the program several times. If the number of iteration is not fixed, it is recommended to use while loop than for loop.

```
while(condition){  
  
    //code to be executed  
  
}
```

Flowchart:



C++ While Loop Example

Let's see a simple example of while loop to print table of 1.

```
#include <iostream>

using namespace std;

int main() {

    int i=1;

    while(i<=10)

    {

        cout<<i <<"\n";

        i++;

    }
```

```
}  
  
}
```

Output:

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10
```

C++ Nested While Loop Example

In C++, we can use while loop inside another while loop, it is known as nested while loop.

The nested while loop is executed fully when outer loop is executed once.

Let's see a simple example of nested while loop in C++ programming language.

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
int i=1;

while(i<=3)

{

    int j = 1;

    while (j <= 3)

{

    cout<<i<<" "<<j<<"\n";

    j++;

}

    i++;

}

}
```

Output:

1 1

1 2

1 3

2 1

2 2

2 3

3 1

3 2

3 3

C++ Infinitive While Loop Example:

We can also create infinite while loop by passing true as the test condition.

```
#include <iostream>

using namespace std;

int main ( ) {

    while(true)

    {

        cout<<"Infinitive While Loop";

    }

}
```

Output:

Infinitive While Loop

Infinitive While Loop

Infinitive While Loop

Infinitive While Loop

Infinitive While Loop

ctrl+c

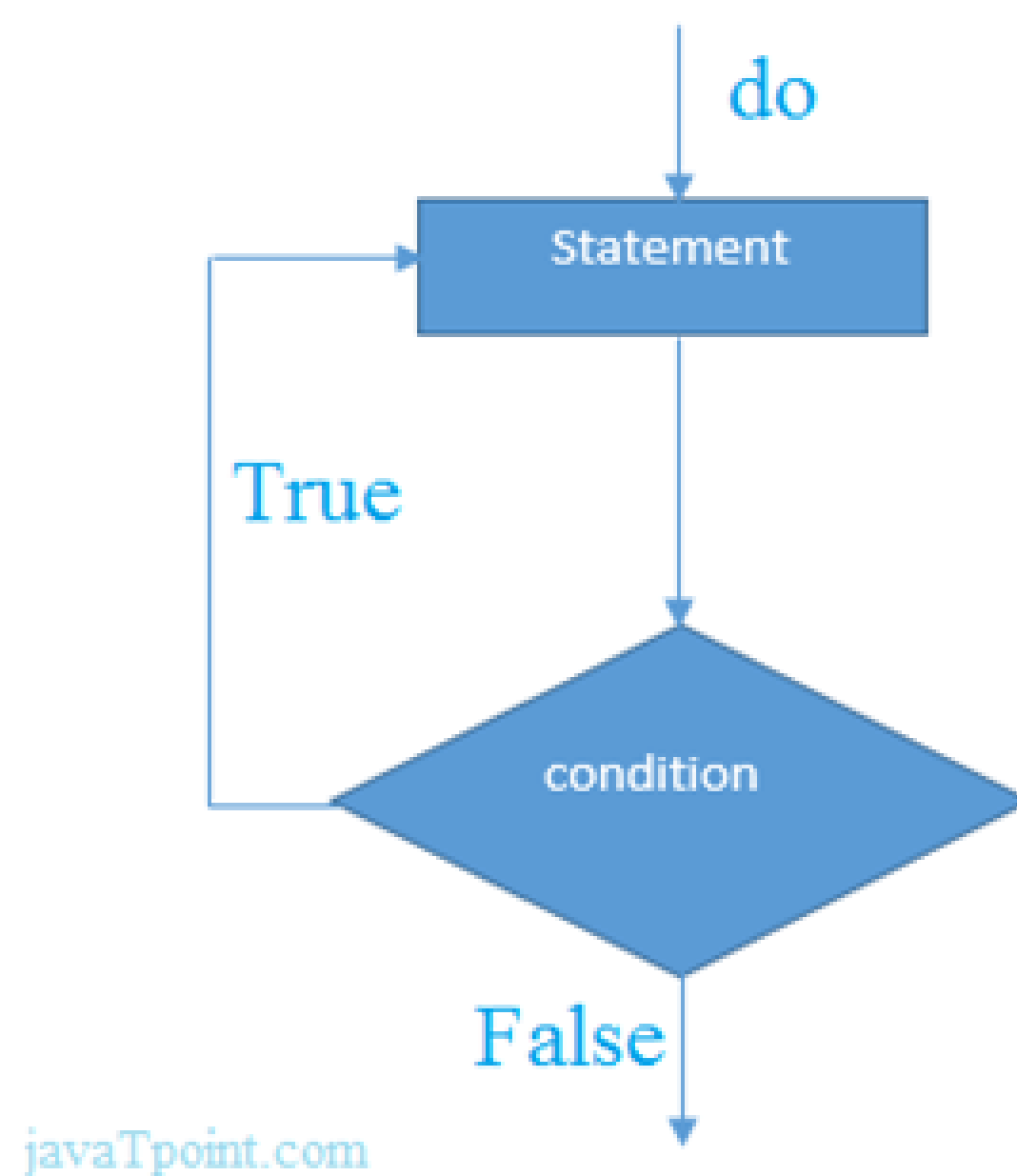
C++ Do-While Loop

The C++ do-while loop is used to iterate a part of the program several times. If the number of iteration is not fixed and you must have to execute the loop at least once, it is recommended to use do-while loop.

The C++ do-while loop is executed at least once because condition is checked after loop body.

```
do{  
  
    //code to be executed  
  
}while(condition);
```

Flowchart:



C++ do-while Loop Example

Let's see a simple example of C++ do-while loop to print the table of 1.

```
#include <iostream>

using namespace std;

int main() {

    int i = 1;

    do{

        cout<<i<<"\n";

        i++;

    } while (i <= 10) ;

}
```

Output:

1
2
3
4
5
6
7
8
9

C++ Nested do-while Loop

In C++, if you use do-while loop inside another do-while loop, it is known as nested do-while loop. The nested do-while loop is executed fully for each outer do-while loop.

Let's see a simple example of nested do-while loop in C++.

```
#include <iostream>

using namespace std;

int main() {

    int i = 1;

    do{

        int j = 1;

        do{

            cout<<i<<"\n";

            j++;

        } while (j <= 3) ;

        i++;

    } while (i <= 3) ;

}
```

Output:

1 1

1 2

1 3

2 1

2 2

2 3

3 1

3 2

3 3

C++ Infinitive do-while Loop

In C++, if you pass **true** in the do-while loop, it will be infinitive do-while loop.

```
do{  
  
    //code to be executed  
  
}while(true);
```

C++ Infinitive do-while Loop Example

```
#include <iostream>  
  
using namespace std;  
  
int main() {
```

```
do{

    cout<<"Infinitive do-while Loop";

} while( true);

}
```

Output:

```
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
Infinitive do-while Loop
ctrl+c
```

C++ Break Statement

The C++ break is used to break loop or switch statement. It breaks the current flow of the program at the given condition. In case of inner loop, it breaks only inner loop.

```
jump-statement;
```

```
break;
```

Flowchart:

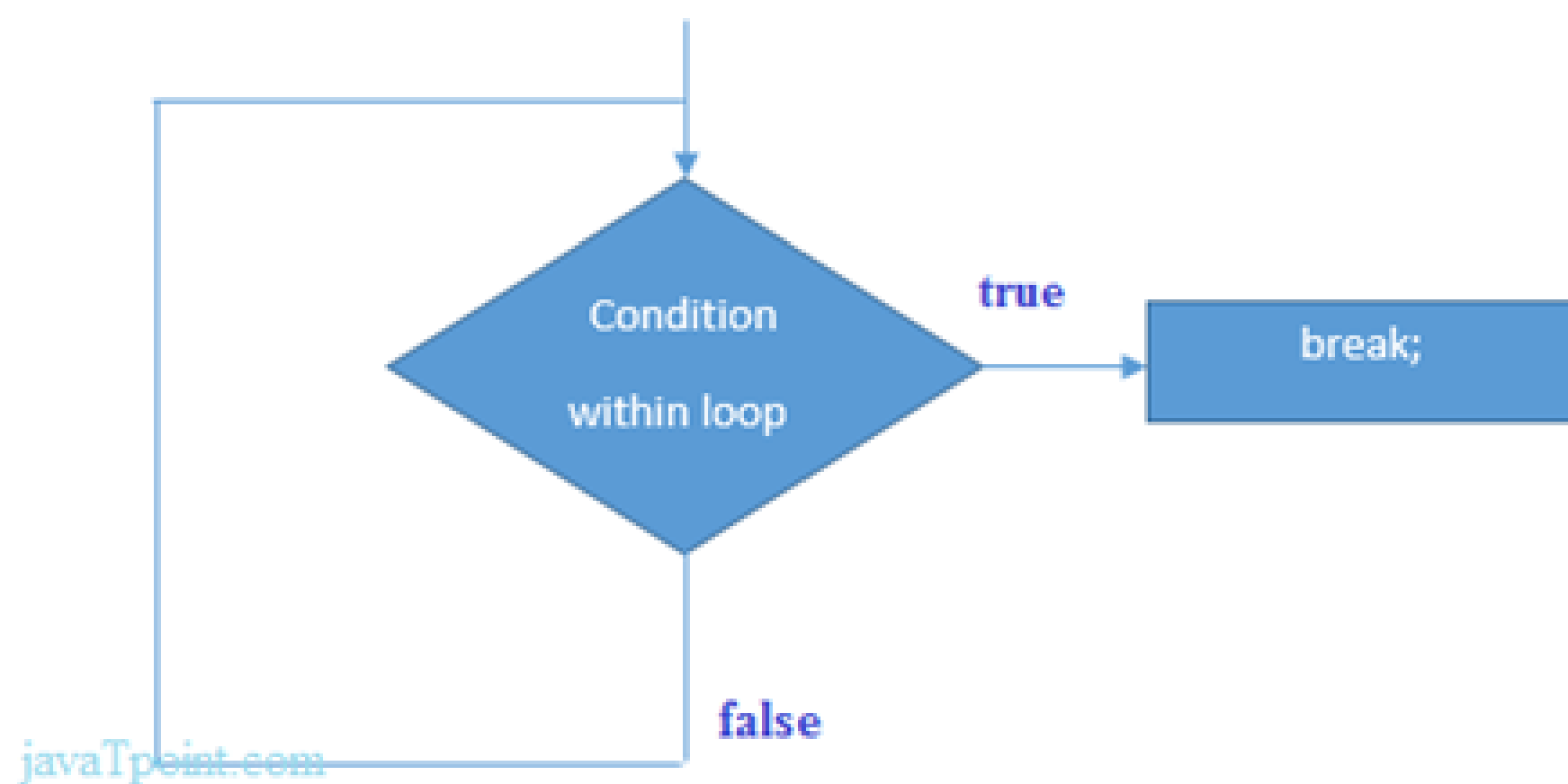


Figure: Flowchart of break statement

C++ Break Statement Example

Let's see a simple example of C++ break statement which is used inside the loop.

```

#include <iostream>

using namespace std;

int main() {

    for (int i = 1; i <= 10; i++)

    {

        if (i == 5)

        {

            break;

        }

        cout<<i<<"\n";
  
```

```
    }  
}
```

Output:

```
1  
2  
3  
4
```

C++ Break Statement with Inner Loop

The C++ break statement breaks inner loop only if you use break statement inside the inner loop.

Let's see the example code:

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
  
    for(int i=1;i<=3;i++){  
  
        for(int j=1;j<=3;j++){  
  
            if(i==2&& j==2){  
  
                break;  
  
            }  
        }  
    }  
}
```

```

        }

        cout<<i<<" "<<j<<"\n";

    }

}

```

Output:

```

1 1
1 2
1 3
2 1
3 1
3 2
3 3

```

C++ Continue Statement

The C++ continue statement is used to continue loop. It continues the current flow of the program and skips the remaining code at specified condition. In case of inner loop, it continues only inner loop.

```

jump-statement;

```

```

continue;

```

C++ Continue Statement Example

```
#include <iostream>

using namespace std;

int main()

{

    for(int i=1;i<=10;i++){

        if(i==5){

            continue;

        }

        cout<<i<<"\n";

    }

}
```

Output:

1
2
3
4
6
7

8

9

10

C++ Continue Statement with Inner Loop

C++ Continue Statement continues inner loop only if you use continue statement inside the inner loop.

```
#include <iostream>

using namespace std;

int main()

{

    for(int i=1;i<=3;i++){

        for(int j=1;j<=3;j++){

            if(i==2&& j==2){

                continue;

            }

            cout<<i<<" "<<j<<"\n";

        }

    }

}
```

Output:

1 1

1 2

1 3

2 1

2 3

3 1

3 2

3 3

C++ Goto Statement

The C++ goto statement is also known as jump statement. It is used to transfer control to the other part of the program. It unconditionally jumps to the specified label.

It can be used to transfer control from deeply nested loop or switch case label.

C++ Goto Statement Example

Let's see the simple example of goto statement in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

ineligible:

```
        cout<<"You are not eligible to vote!\n";

    cout<<"Enter your age:\n";

    int age;

    cin>>age;

    if (age < 18){

        goto ineligible;

    }

    else

    {

        cout<<"You are eligible to vote!";

    }

}
```

Output:

You are not eligible to vote!

Enter your age:

16

You are not eligible to vote!

Enter your age:

7

You are not eligible to vote!

Enter your age:

22

You are eligible to vote!

C++ Comments

The C++ comments are statements that are not executed by the compiler. The comments in C++ programming can be used to provide explanation of the code, variable, method or class. By the help of comments, you can hide the program code also.

There are two types of comments in C++.

- Single Line comment
- Multi Line comment

C++ Single Line Comment

The single line comment starts with // (double slash). Let's see an example of single line comment in C++.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x = 11; // x is a variable

cout<<x<<"\n";

}
```

Output:

11

C++ Multi Line Comment

The C++ multi line comment is used to comment multiple lines of code. It is surrounded by slash and asterisk (/* */). Let's see an example of multi line comment in C++.

```
#include <ostream>

using namespace std;

int main()

{

/* declare and

print variable in C++. */

int x = 35;

cout<<x<<"\n";

}
```

Output:

C++ Functions

C++ Functions

The function in C++ language is also known as procedure or subroutine in other programming languages.

To perform any task, we can create function. A function can be called many times. It provides modularity and code reusability.

Advantage of functions in C

There are many advantages of functions.

1) Code Reusability

By creating functions in C++, you can call it many times. So we don't need to write the same code again and again.

2) Code optimization

It makes the code optimized, we don't need to write much code.

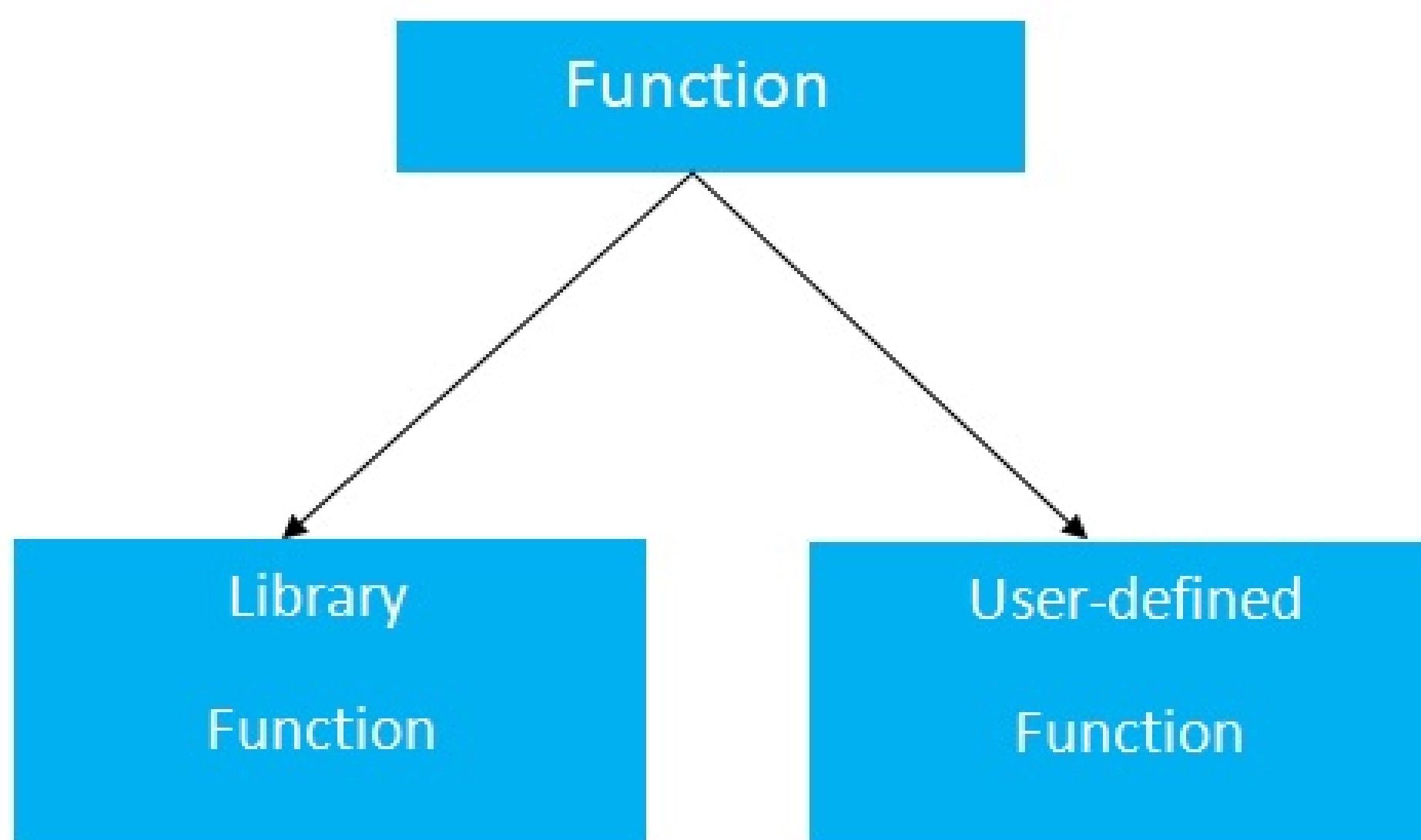
Suppose, you have to check 3 numbers (531, 883 and 781) whether it is prime number or not. Without using function, you need to write the prime number logic 3 times. So, there is repetition of code.

But if you use functions, you need to write the logic only once and you can reuse it several times.

Types of Functions

There are two types of functions in C programming:

- 1. Library Functions:** are the functions which are declared in the C++ header files such as `ceil(x)`, `cos(x)`, `exp(x)`, etc.
- 2. User-defined functions:** are the functions which are created by the C++ programmer, so that he/she can use it many times. It reduces complexity of a big program and optimizes the code.



Declaration of a function

The syntax of creating function in C++ language is given below:

```
return_type function_name(data_type parameter...)

{

    //code to be executed
```

```
}
```

C++ Function Example

Let's see the simple example of C++ function.

```
#include <iostream>

using namespace std;

void func() {

    static int i=0; //static variable

    int j=0; //local variable

    i++;

    j++;

    cout<<"i=" << i<<" and j=" <<j<<endl;

}

int main()

{

    func();

    func();

    func();

}
```

Output:

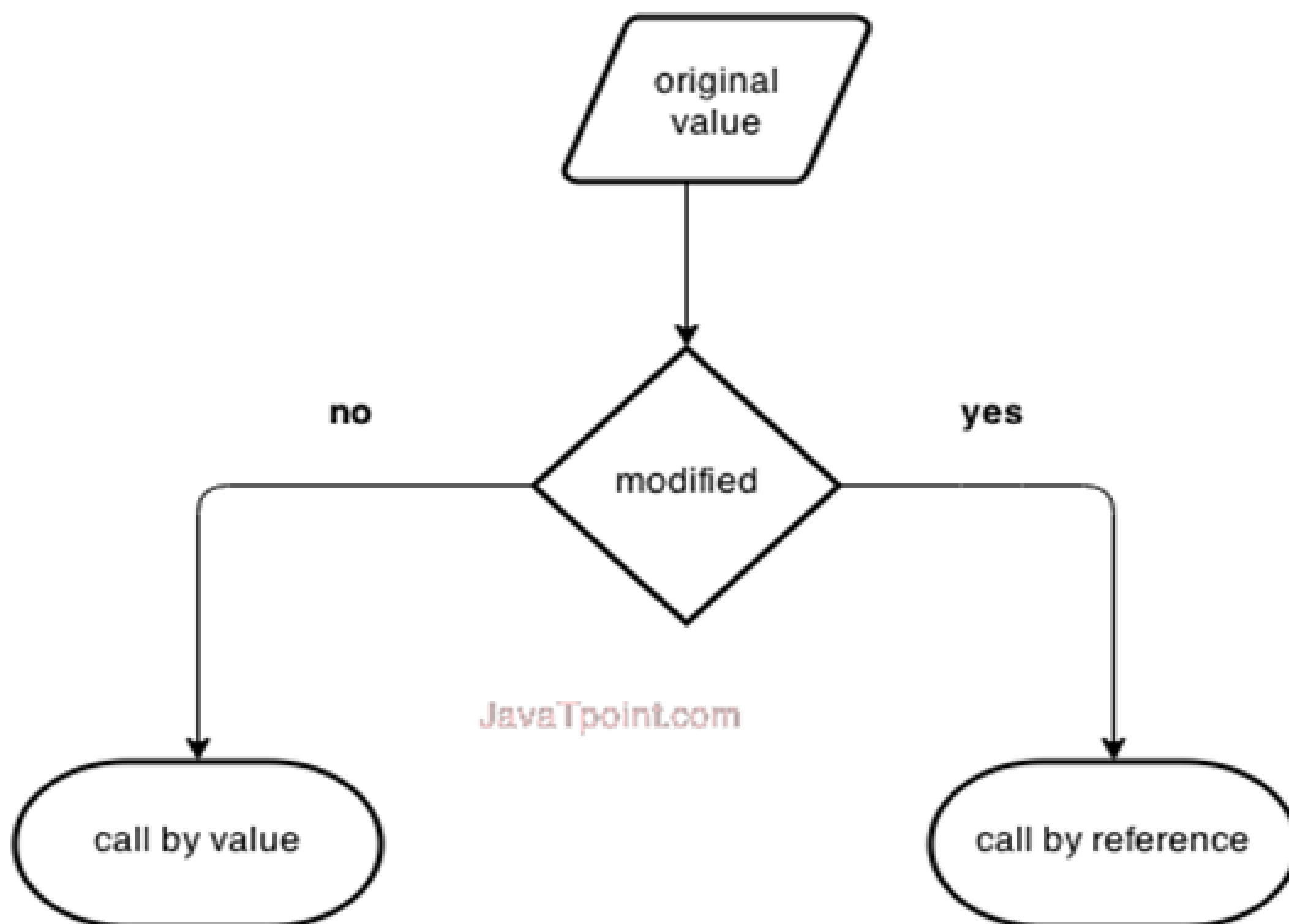
i= 1 and j= 1

i= 2 and j= 1

i= 3 and j= 1

Call by value and call by reference in C++

There are two ways to pass value or data to function in C language: call by value and call by reference. Original value is not modified in call by value but it is modified in call by reference.



Let's understand call by value and call by reference in C++ language one by one.

Call by value in C++

In call by value, **original value is not modified.**

In call by value, value being passed to the function is locally stored by the function parameter in stack memory location. If you change the value of function parameter, it is changed for the current function only. It will not change the value of variable inside the caller method such as main().

Let's try to understand the concept of call by value in C++ language by the example given below:

```
#include <iostream>

using namespace std;

void change( int data);

int main( )

{

    int data = 3;

    change(data);

    cout << "Value of the data is: " << data<< endl;

    return 0;

}

void change( int data)

{

    data = 5;
```

```
}
```

Output:

Value of the data is: 3

Call by reference in C++

In call by reference, original value is modified because we pass reference (address).

Here, address of the value is passed in the function, so actual and formal arguments share the same address space. Hence, value changed inside the function, is reflected inside as well as outside the function.

Note: To understand the call by reference, you must have the basic knowledge of pointers.

Let's try to understand the concept of call by reference in C++ language by the example given below:

```
#include<iostream>

using namespace std;

void swap(int *x, int *y)
{
    int swap;

    swap= *x;

    *x= *y;

    *y=swap;
```

```

}

int main()

{

    int x=500, y=100;

    swap( &x, &y); // passing value to function

    cout<<"Value of x is: "<<x<<endl;

    cout<<"Value of y is: "<<y<<endl;

    return 0;

}

```

Output:

Value of x is: 100

Value of y is: 500

Difference between call by value and call by reference in C++

Nb.	Call byvalue	Call byreference
1	A copy of value is passed to the function	An address of value is passed to the function
2	Changes made inside the function	Changes made inside the function is

	is not reflected on other functions	reflected outside the function also
3	Actual and formal arguments will be created in different memory location	Actual and formal arguments will be created in same memory location

C++ Recursion

When function is called within the same function, it is known as recursion in C++. The function which calls the same function, is known as recursive function.

A function that calls itself, and doesn't perform any task after function call, is known as tail recursion. In tail recursion, we generally call the same function with return statement.

Let's see a simple example of recursion.

```
recursionfunction(){
    recursionfunction(); //calling self function
}
```

C++ Recursion Example

Let's see an example to print factorial number using recursion in C++ language.

```
#include<iostream>

using namespace std;
```

```

int main()

{

int factorial(int);

int fact,value;

cout<<"Enter any number: ";

cin>>value;

fact=factorial(value);

cout<<"Factorial of a number is: "<<fact<<endl;

return 0;

}

int factorial(int n)

{

if(n<0)

return(-1); /*Wrong value*/

elseif(n==0)

return(1); /*Terminating condition*/

else

{

```

```
return(n*factorial(n-1));  
  
}  
  
}
```

Output:

Enter any number: 5

Factorial of a number is: 120

We can understand the above program of recursive method call by the figure given below:

```
return 5 * factorial(4) = 120  
└─ return 4 * factorial(3) = 24  
    └─ return 3 * factorial(2) = 6  
        └─ return 2 * factorial(1) = 2  
            └─ return 1 * factorial(0) = 1
```

javaTpoint.com

$1 * 2 * 3 * 4 * 5 = 120$

Fig: Recursion

C++ Storage Classes

Storage class is used to define the lifetime and visibility of a variable and/or function within a C++ program.

Lifetime refers to the period during which the variable remains active and visibility refers to the module of a program in which the variable is accessible.

There are five types of storage classes, which can be used in a C++ program

- 1. Automatic
- 2. Register
- 3. Static
- 4. External
- 5. Mutable

Storage Class	Keyword	Lifetime	Visibility	Initial Value
Automatic	auto	Function Block	Local	Garbage
Register	register	Function Block	Local	Garbage
Mutable	mutable	Class	Local	Garbage
External	extern	Whole Program	Global	Zero
Static	static	Whole Program	Local	Zero

Automatic Storage Class

It is the default storage class for all local variables. The auto keyword is applied to all local variables automatically.

```
{  
  
    auto int y;  
  
    float y = 3.45;  
  
}
```

The above example defines two variables with a same storage class, auto can only be used within functions.

Register Storage Class

The register variable allocates memory in register than RAM. Its size is same of register size. It has a faster access than other variables.

It is recommended to use register variable only for quick access such as in counter.

Note: We can't get the address of register variable.

```
register int counter=0;
```

Static Storage Class

The static variable is initialized only once and exists till the end of a program. It retains its value between multiple functions call.

The static variable has the default value 0 which is provided by compiler.

```
#include <iostream>
```

```

using namespace std;

void func() {

    static int i=0; //static variable

    int j=0; //local variable

    i++;

    j++;

    cout<<"i=" << i<<" and j=" <<j<<endl;

}

int main()

{

    func();

    func();

    func();

}

```

Output:

i= 1 and j= 1

i= 2 and j= 1

i= 3 and j= 1

External Storage Class

The extern variable is visible to all the programs. It is used if two or more files are sharing same variable or function.

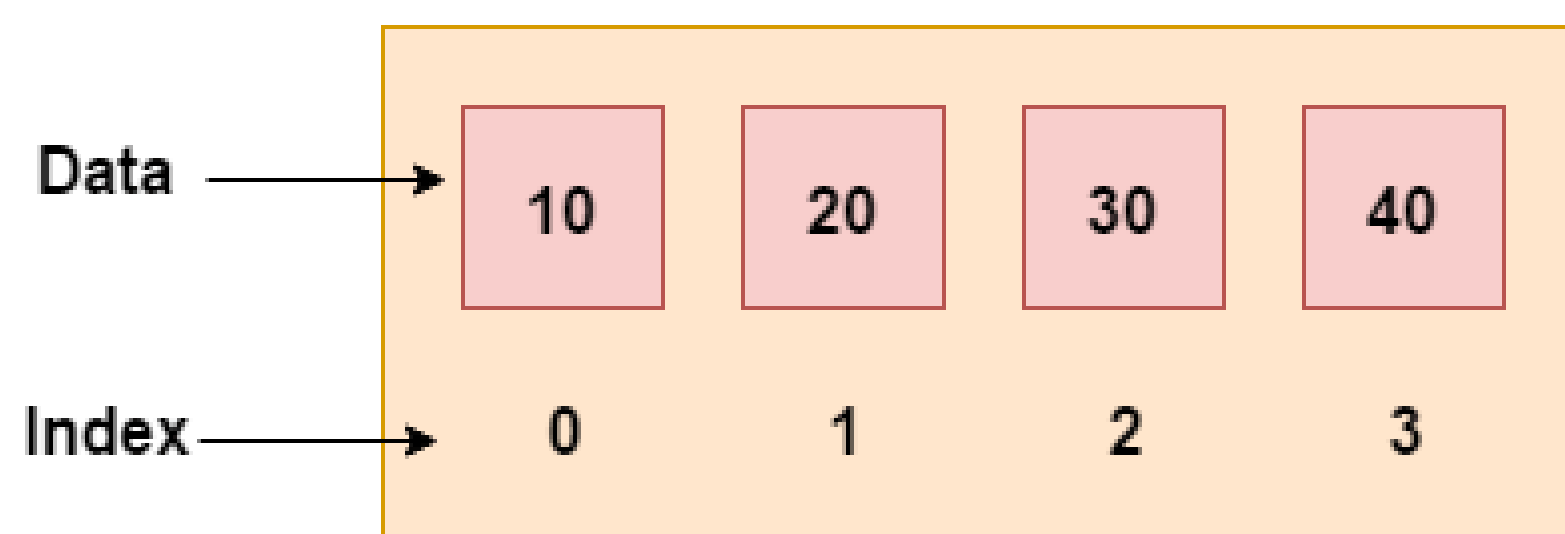
```
extern int counter=0;
```

C++ Arrays

C++ Arrays

Like other programming languages, array in C++ is a group of similar types of elements that have contiguous memory location.

In C++ `std::array` is a container that encapsulates fixed size arrays. In C++, array index starts from 0. We can store only fixed set of elements in C++ array.



Advantages of C++ Array

- Code Optimization (less code)
- Random Access
- Easy to traverse data

- Easy to manipulate data
- Easy to sort data etc.

Disadvantages of C++ Array

- Fixed size

C++ Array Types

There are 2 types of arrays in C++ programming:

1. Single Dimensional Array
2. Multidimensional Array

C++ Single Dimensional Array

Let's see a simple example of C++ array, where we are going to create, initialize and traverse array.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int arr[5]={10, 0, 20, 0, 30}; //creating and initializing array
```

```
//traversing array
```

```

    for (int i = 0; i < 5; i++)
    {
        cout<<arr[i]<<"\n";
    }
}

```

Output: /p>

```

10
0
20
0
30

```

C++ Array Example: Traversal using foreach loop

We can also traverse the array elements using foreach loop. It returns array element one by one.

```

#include <iostream>

using namespace std;

int main()
{
    int arr[5]={10, 0, 20, 0, 30}; //creating and initializing array

```

```

        //traversing array

for (int i: arr)

{

    cout<<i<<"\n";

}

}

```

Output:

```

10
20
30
40
50

```

C++ Passing Array to Function

In C++, to reuse the array logic, we can create function. To pass array to function in C++, we need to provide only array name.

```

functionname(arrayname); //passing array to function

```

C++ Passing Array to Function Example: print array elements

Let's see an example of C++ function which prints the array elements.

```

#include <iostream>

```

```

using namespace std;

void printArray(int arr[5]);

int main()

{

    int arr1[5] = { 10, 20, 30, 40, 50 };

    int arr2[5] = { 5, 15, 25, 35, 45 };

    printArray(arr1); //passing array to function

    printArray(arr2);

}

void printArray(int arr[5])

{

    cout << "Printing array elements:" << endl;

    for (int i = 0; i < 5; i++)

    {

        cout<<arr[i]<<"\n";

    }

}

```

Output:

Printing array elements:

10

20

30

40

50

Printing array elements:

5

15

25

35

45

C++ Passing Array to Function Example: Print minimum number

Let's see an example of C++ array which prints minimum number in an array using function.

```
#include <iostream>
```

```
using namespace std;
```

```
void printMin(int arr[5]);
```

```
int main()
```

```
{
```

```
    int arr1[5] = { 30, 10, 20, 40, 50 };
```

```

    int arr2[5] = { 5, 15, 25, 35, 45 };

    printMin(arr1); // passing array to function

    printMin(arr2);

}

void printMin(int arr[5])

{

    int min = arr[0];

    for (int i = 0; i < 5; i++)

    {

        if (min > arr[i])

        {

            min = arr[i];

        }

    }

    cout<< "Minimum element is: "<< min <<"\n";

}

```

Output:

Minimum element is: 10

Minimum element is: 5

C++ Passing Array to Function Example: Print maximum number

Let's see an example of C++ array which prints maximum number in an array using function.

```
#include <iostream>

using namespace std;

void printMax(int arr[5]);

int main()
{
    int arr1[5] = { 25, 10, 54, 15, 40 };

    int arr2[5] = { 12, 23, 44, 67, 54 };

    printMax(arr1); //Passing array to function

    printMax(arr2);
}

void printMax(int arr[5])
{
    int max = arr[0];

    for (int i = 0; i < 5; i++)
```

```

    {

        if (max < arr[i])

        {

            max = arr[i];

        }

    }

    cout<< "Maximum element is: "<< max <<"\n";

}

```

Output:

Maximum element is: 54

Maximum element is: 67

C++ Multidimensional Arrays

The multidimensional array is also known as rectangular arrays in C++. It can be two dimensional or three dimensional. The data is stored in tabular form (row * column) which is also known as matrix.

C++ Multidimensional Array Example

Let's see a simple example of multidimensional array in C++ which declares, initializes and traverse two dimensional arrays.

```
#include <iostream>
```

```

using namespace std;

int main()

{

    int test[3][3]; //declaration of 2D array

    test[0][0]=5; //initialization

    test[0][1]=10;

    test[1][1]=15;

    test[1][2]=20;

    test[2][0]=30;

    test[2][2]=10;

    //traversal

    for(int i = 0; i < 3; ++i)

    {

        for(int j = 0; j < 3; ++j)

        {

            cout<< test[i][j]<<" ";

        }

        cout<<"\n"; //new line at each row
    }
}

```

```
    }  
  
    return 0;  
  
}
```

Output:

```
5 10 0  
0 15 20  
30 0 10
```

C++ Multidimensional Array Example: Declaration and initialization at same time

Let's see a simple example of multidimensional array which initializes array at the time of declaration.

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
  
    int test[3][3] =  
  
        {  
  
            {2, 5, 5},  
  
            {4, 0, 3},  
  
        }  
}
```

```

        {9, 1, 8} }; //declaration and initialization

//traversal

for(int i = 0; i < 3; ++i)

{

    for(int j = 0; j < 3; ++j)

    {

        cout<< test[i][j]<<" ";

    }

    cout<<"\n"; //new line at each row

}

return 0;

}

```

Output:

2 5 5

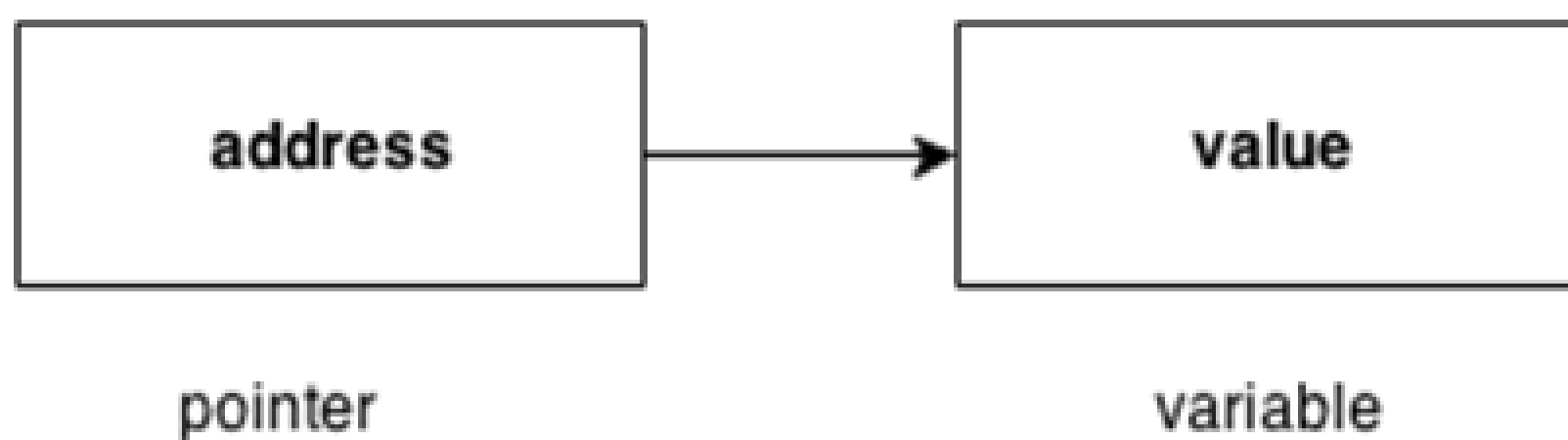
4 0 3

9 1 8

C++ Pointers

C++ Pointers

The pointer in C++ language is a variable, it is also known as locator or indicator that points to an address of a value.



Advantage of pointer

- 1) Pointer reduces the code and improves the performance, it is used to retrieving strings, trees etc. and used with arrays, structures and functions.
- 2) We can return multiple values from function using pointer.
- 3) It makes you able to access any memory location in the computer's memory.

Usage of pointer

There are many usage of pointers in C++ language.

1) Dynamic memory allocation

In c language, we can dynamically allocate memory using `malloc()` and `calloc()` functions where pointer is used.

2) Arrays, Functions and Structures

Pointers in c language are widely used in arrays, functions and structures. It reduces the code and improves the performance.

Symbols used in pointer

Symbol	Name	Description
& (ampersand sign)	Address operator	Determine the address of a variable.
* (asterisk sign)	Indirection operator	Access the value of an address.

Declaring a pointer

The pointer in C++ language can be declared using * (asterisk symbol).

```
int * a; // pointer to int

char * c; // pointer to char
```

Pointer Example

Let's see the simple example of using pointers printing the address and value.

```
#include <iostream>

using namespace std;

int main()
```

```

{

int number=30;

int *   p;

p=&number; //stores the address of number variable

cout<<"Address of number variable is:"<<&number<<endl;

cout<<"Address of p variable is:"<<p<<endl;

cout<<"Value of p variable is:"<<*p<<endl;

    return 0;

}

```

Output:

Address of number variable is:0x7ffccc8724c4

Address of p variable is:0x7ffccc8724c4

Value of p variable is:30

Pointer Program to swap 2 numbers without using 3rd variable

```

#include <iostream>

using namespace std;

int main( )

{

```

```

int a=20,b=10,* p1=&a,* p2=&b;

cout<<"Before swap: * p1="<<* p1<<" * p2="<<* p2<<endl;

* p1=* p1+* p2;

* p2=* p1-* p2;

* p1=* p1-* p2;

cout<<"After swap: * p1="<<* p1<<" * p2="<<* p2<<endl;

return 0;

}

```

Output:

Before swap: * p1=20 * p2=10

After swap: * p1=10 * p2=20

sizeof() operator in C++

The sizeof() is an operator that evaluates the size of data type, constants, variable. It is a compile-time operator as it returns the size of any variable or a constant at the compilation time.

The size, which is calculated by the sizeof() operator, is the amount of RAM occupied in the computer.

Syntax of the sizeof() operator is given below:

```
sizeof(data_type);
```

In the above syntax, the data_type can be the data type of the data, variables, constants, unions, structures, or any other user-defined data type.

The sizeof () operator can be applied to the following operand types:

When an operand is of data type

If the parameter of a **sizeof ()** operator contains the data type of a variable, then the **sizeof ()** operator will return the size of the data type.

Let's understand this scenario through an example.

```
#include <iostream>

using namespace std;

int main()
{
    // Determining the space in bytes occupied by each data type.

    std::cout << "Size of integer data type : " << sizeof(int) << std::endl;

    std::cout << "Size of float data type : " << sizeof(float) << std::endl;

    std::cout << "Size of double data type : " << sizeof(double) << std::endl;

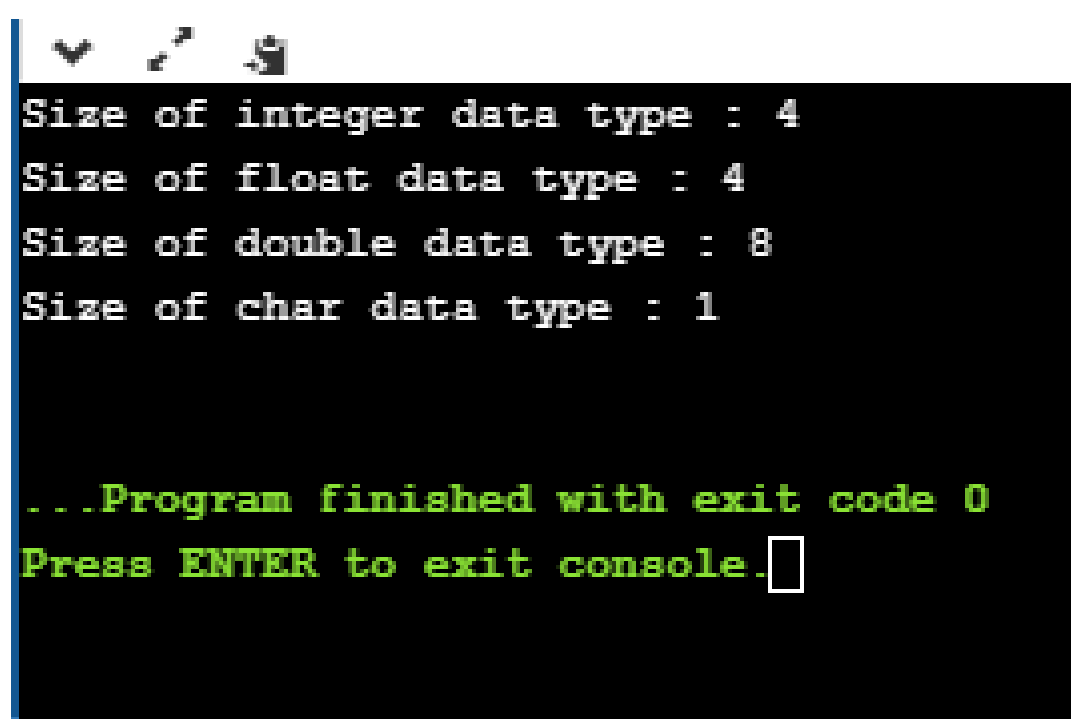
    std::cout << "Size of char data type : " << sizeof(char) << std::endl;

    return 0;
```

```
}
```

In the above program, we have evaluated the size of the in-built data types by using the `sizeof()` operator. As we know that `int` occupies 4 bytes, `float` occupies 4 bytes, `double` occupies 8 bytes, and `char` occupies 1 byte, and the same result is shown by the `sizeof()` operator as we can observe in the following output.

Output



```
Size of integer data type : 4
Size of float data type : 4
Size of double data type : 8
Size of char data type : 1

...Program finished with exit code 0
Press ENTER to exit console.
```

When an operand is of Class type.

```
#include <iostream>
```

```
using namespace std;
```

```
class Base
```

```
{
```

```
int a;
```

```
};
```

```
int main()
```

```

{

Base b;

std::cout << "Size of class base is : "<<sizeof(b) << std::endl;

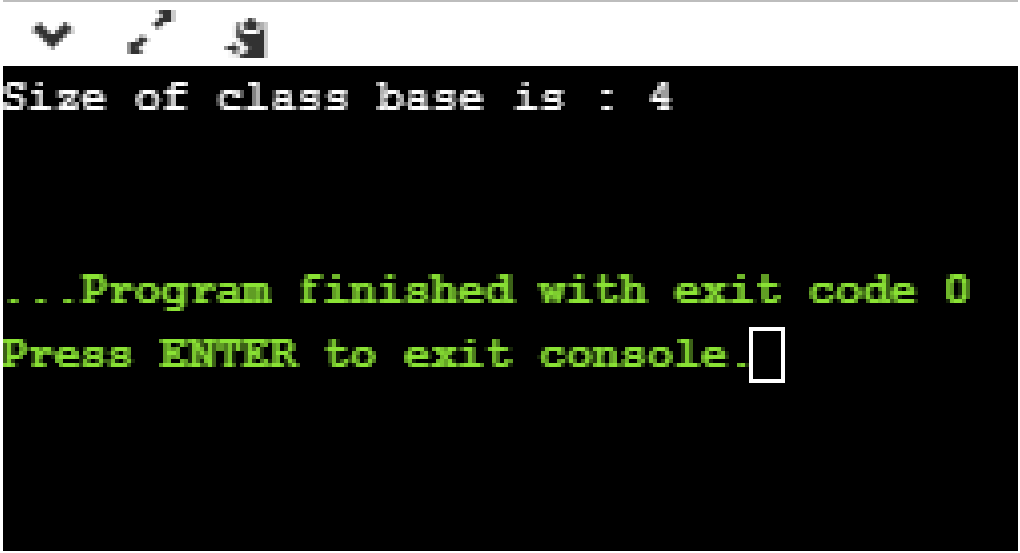
return 0;

}

```

In the above program, we have evaluated the size of the class, which is having a single integer variable. The output would be 4 bytes as int variable occupies 4 bytes.

Output



```

Size of class base is : 4

...Program finished with exit code 0
Press ENTER to exit console.

```

If we add one more integer variable in a class, then the code would look like:

```

#include <iostream>

using namespace std;

class Base

{

    int a;

```

```

    int d;

};

int main()

{

    Base b;

    std::cout << "Size of class base is : "<<sizeof(b) << std::endl;

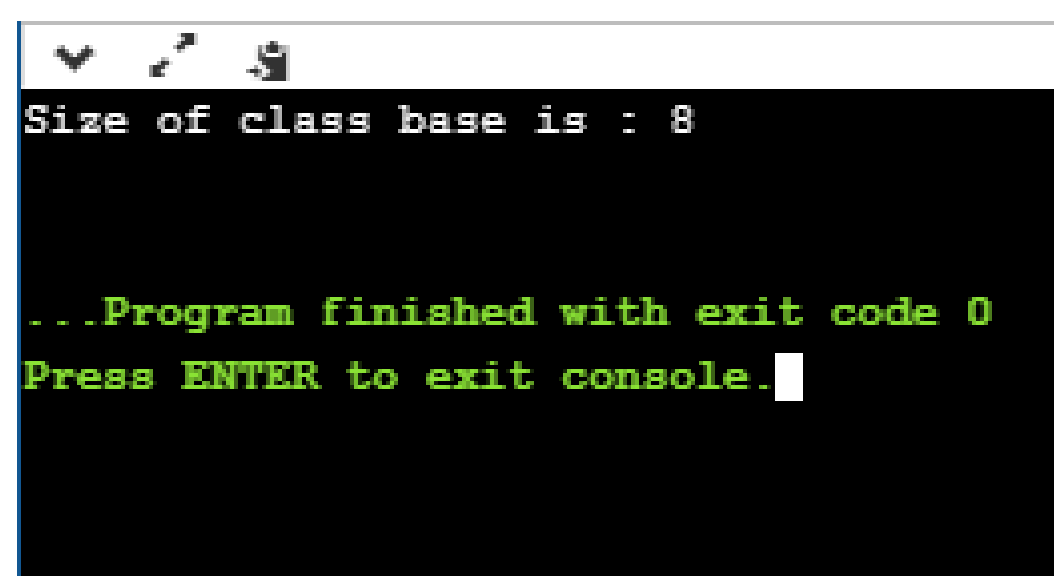
    return 0;

}

```

In the above code, we have added one more integer variable. In this case, the size of the class would be 8 bytes as `int` variable occupies 4 bytes, so two integer variables occupy 8 bytes.

Output



```

Size of class base is : 8

...Program finished with exit code 0
Press ENTER to exit console.

```

If we add a char variable in the above code, then the code would look like:

```

#include <iostream>

```

```
using namespace std;
```

```
class Base
```

```
{
```

```
    int a;
```

```
    int d;
```

```
    char ch;
```

```
};
```

```
int main()
```

```
{
```

```
    Base b;
```

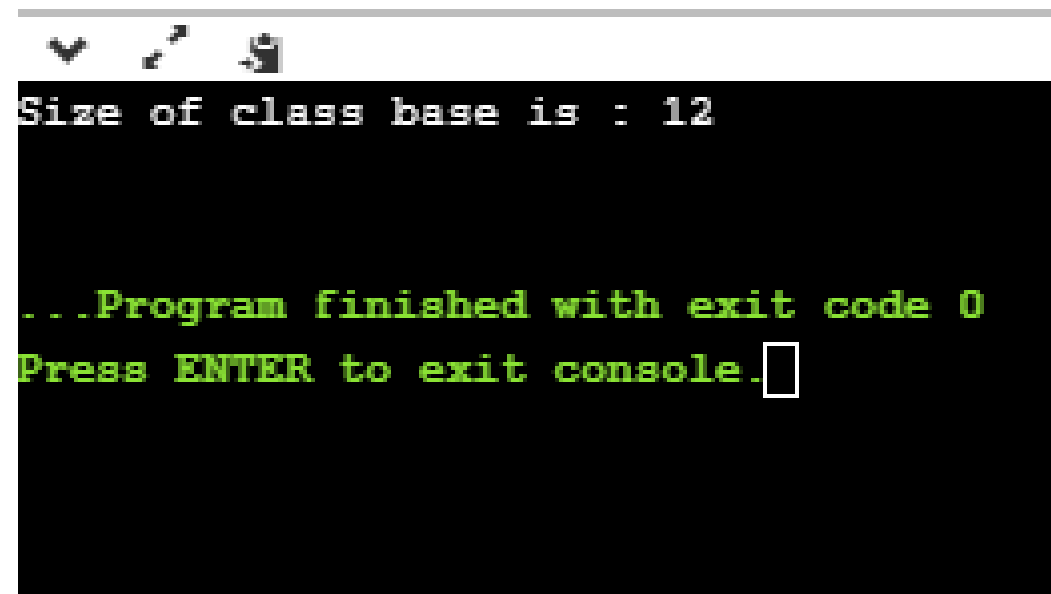
```
    std::cout << "Size of class base is : "<<sizeof(b) << std::endl;
```

```
    return 0;
```

```
}
```

In the above code, the class has two integer variables, and one char variable. According to our calculation, the size of the class would be equal to 9 bytes (int+int+char), but this is wrong due to the concept of structure padding.

Output



```
Size of class base is : 12

...Program finished with exit code 0
Press ENTER to exit console.
```

When an operand is of array type.

```
#include <iostream>

using namespace std;

int main()

{

    int arr[]={10,20,30,40,50};

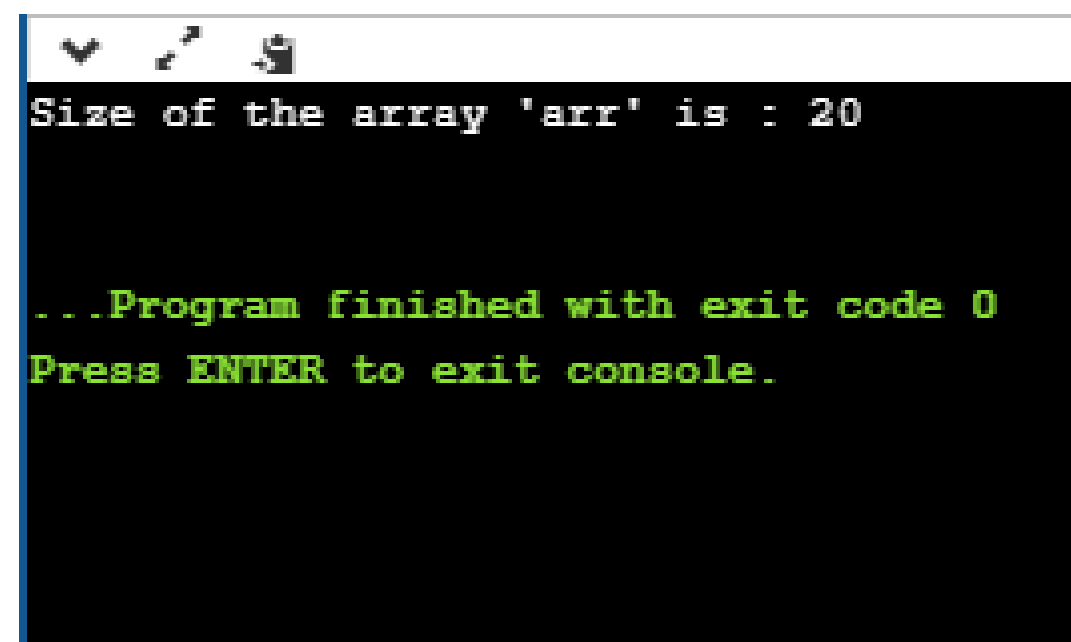
    std::cout << "Size of the array 'arr' is : "<<sizeof(arr) << std::endl;

    return 0;

}
```

In the above program, we have declared an array of integer type which contains five elements. We have evaluated the size of the array by using **sizeof()** operator. According to our calculation, the size of the array should be 20 bytes as int data type occupies 4 bytes, and array contains 5 elements, so total memory space occupied by this array is $5 * 4 = 20$ bytes. The same result has been shown by the **sizeof()** operator as we can observe in the following output.

Output



```
Size of the array 'arr' is : 20

...Program finished with exit code 0
Press ENTER to exit console.
```

Let's consider another scenario of an array.

```
#include <iostream>

using namespace std;

void fun(int arr[])

{

    std::cout << "Size of array is : " << sizeof(arr) << std::endl;

}

int main()

{

    int arr[]={10,20,30,40,50};

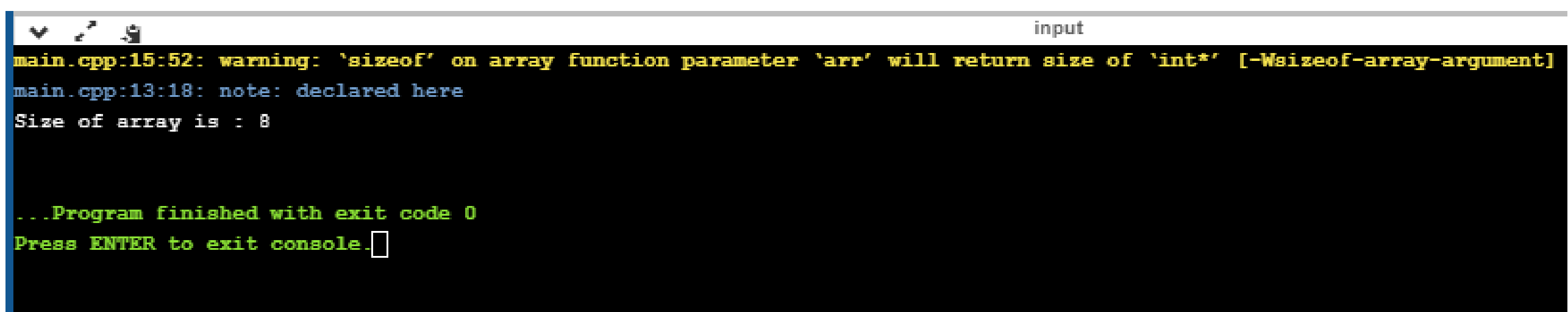
    fun(arr);

    return 0;

}
```

In the above program, we have tried to print the size of the array using the function. In this case, we have created an array of type integer, and we pass the 'arr' to the function fun(). The fun() would return the size of the integer pointer, i.e., int*, and the size of the int* is 8 bytes in the 64-bit operating system.

Output

A screenshot of a terminal window titled 'input'. The terminal shows the output of a C++ program. It starts with two compiler warnings: 'main.cpp:15:52: warning: 'sizeof' on array function parameter 'arr' will return size of 'int*' [-Wsizeof-array-argument]' and 'main.cpp:13:18: note: declared here'. The program output is 'Size of array is : 8'. At the bottom, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor.

```
input
main.cpp:15:52: warning: 'sizeof' on array function parameter 'arr' will return size of 'int*' [-Wsizeof-array-argument]
main.cpp:13:18: note: declared here
Size of array is : 8

...Program finished with exit code 0
Press ENTER to exit console.
```

When an operand is of pointer type.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int *ptr1=new int(10);
```

```
    std::cout << "size of ptr1 : " << sizeof(ptr1)<< std::endl;
```

```
    std::cout << "size of *ptr1 : " << sizeof(*ptr1)<< std::endl;
```

```
    char *ptr2=new char('a');
```

```

std::cout << "size of ptr2 : " << sizeof(ptr2) << std::endl;

std::cout << "size of * ptr2 : " << sizeof(* ptr2) << std::endl;

double * ptr3 = new double(12.78);

std::cout << "size of ptr3 : " << sizeof(ptr3) << std::endl;

std::cout << "size of * ptr3 : " << sizeof(* ptr3) << std::endl;

return 0;

}

```

In the above program, we have determined the size of pointers. The size of pointers would remain same for all the data types. If the computer has 32bit operating system, then the size of the pointer would be 4 bytes. If the computer has 64-bit operating system, then the size of the pointer would be 8 bytes. I am running this program on 64-bit, so the output would be 8 bytes. Now, if we provide the '*' symbol to the pointer, then the output depends on the data type, for example, * ptr1 is of integer type means the sizeof() operator will return 4 bytes as int data type occupies 4 bytes.

Output

```
size of ptr1 : 8
size of *ptr1 : 4
size of ptr2 : 8
size of *ptr2 : 1
size of ptr3 : 8
size of *ptr3 : 8

...Program finished with exit code 0
Press ENTER to exit console.
```

When an operand is an expression.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int num1;
```

```
    double num2;
```

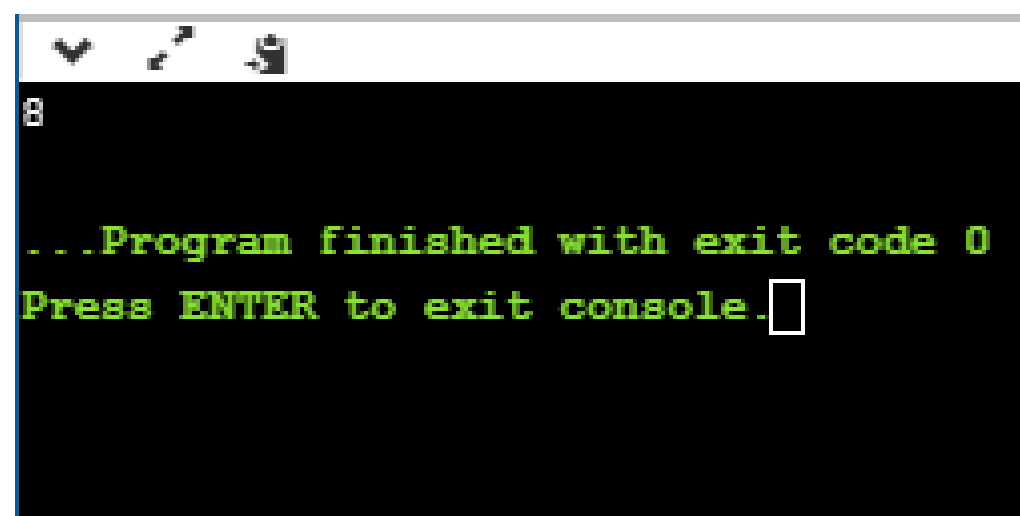
```
    cout << sizeof(num1+num2);
```

```
    return 0;
```

```
}
```

In the above program, we have declared two variables num1 and num2 of type int and double, respectively. The size of the int is 4 bytes, while the size of double is 8 bytes. The result would be the variable, which is of double type occupying 8 bytes.

Output



C++ Array of Pointers

Array and pointers are closely related to each other. In C++, the name of an array is considered as a pointer, i.e., the name of an array contains the address of an element. C++ considers the array name as the address of the first element. For example, if we create an array, i.e., marks which hold the 20 values of integer type, then marks will contain the address of first element, i.e., marks[0]. Therefore, we can say that array name (marks) is a pointer which is holding the address of the first element of an array.

Let's understand this scenario through an example.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```

int *ptr; // integer pointer declaration

int marks[10]; // marks array declaration

std::cout << "Enter the elements of an array :" << std::endl;

for(int i=0;i<10;i++)

{

    cin>>marks[i];

}

ptr=marks; // both marks and ptr pointing to the same element..

std::cout << "The value of * ptr is :" << *ptr<< std::endl;

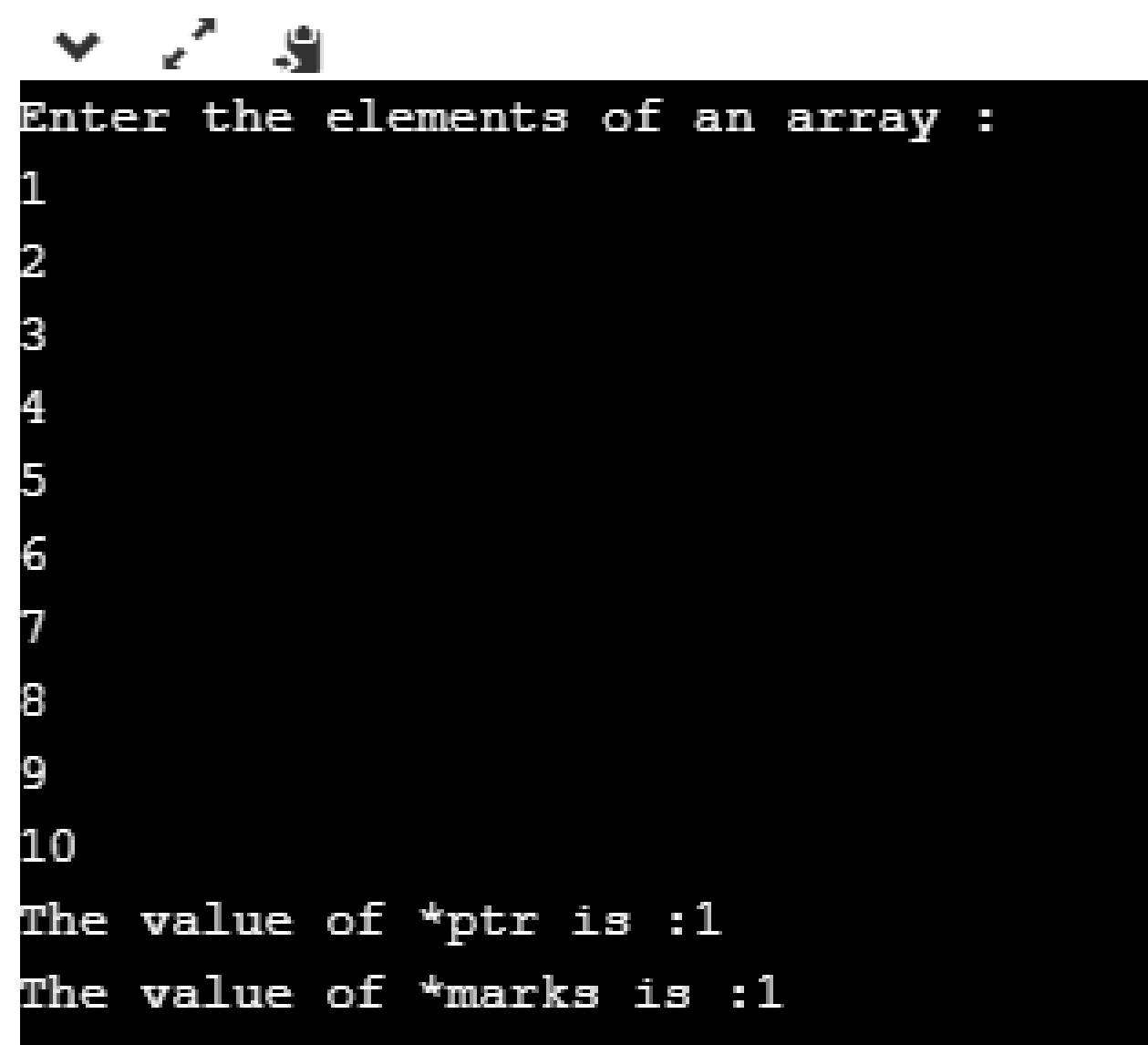
std::cout << "The value of * marks is :" << *marks<<std::endl;

}

```

In the above code, we declare an integer pointer and an array of integer type. We assign the address of marks to the ptr by using the statement ptr=marks; it means that both the variables 'marks' and 'ptr' point to the same element, i.e., marks[0]. When we try to print the values of * ptr and * marks, then it comes out to be same. Hence, it is proved that the array name stores the address of the first element of an array.

Output

A screenshot of a terminal window with a black background and white text. At the top, there are three small icons: a downward arrow, a magnifying glass, and a document. The text in the terminal reads: "Enter the elements of an array :", followed by a list of numbers from 1 to 10, each on a new line. Below the list, it says "The value of *ptr is :1" and "The value of *marks is :1".

```
Enter the elements of an array :
1
2
3
4
5
6
7
8
9
10
The value of *ptr is :1
The value of *marks is :1
```

Array of Pointers

An array of pointers is an array that consists of variables of pointer type, which means that the variable is a pointer addressing to some other element. Suppose we create an array of pointer holding 5 integer pointers; then its declaration would look like:

```
int *ptr[5];    // array of 5 integer pointer.
```

In the above declaration, we declare an array of pointer named as ptr, and it allocates 5 integer pointers in memory.

The element of an array of a pointer can also be initialized by assigning the address of some other element. Let's observe this case through an example.

```
int a; // variable declaration.
```

```
ptr[2] = &a;
```

In the above code, we are assigning the address of 'a' variable to the third element of an array 'ptr'.

We can also retrieve the value of 'a' by dereferencing the pointer.

```
*ptr[2];
```

Let's understand through an example.

```
#include <iostream>

using namespace std;

int main()
{

    int ptr1[5]; // integer array declaration

    int *ptr2[5]; // integer array of pointer declaration

    std::cout << "Enter five numbers : " << std::endl;

    for(int i=0;i<5;i++)
    {

        std::cin >> ptr1[i];

    }

    for(int i=0;i<5;i++)
    {

        ptr2[i]=&ptr1[i];

    }
```

```

// printing the values of ptr1 array

std::cout << "The values are" << std::endl;

for(int i=0;i<5;i++)

{

    std::cout << *ptr2[i] << std::endl;

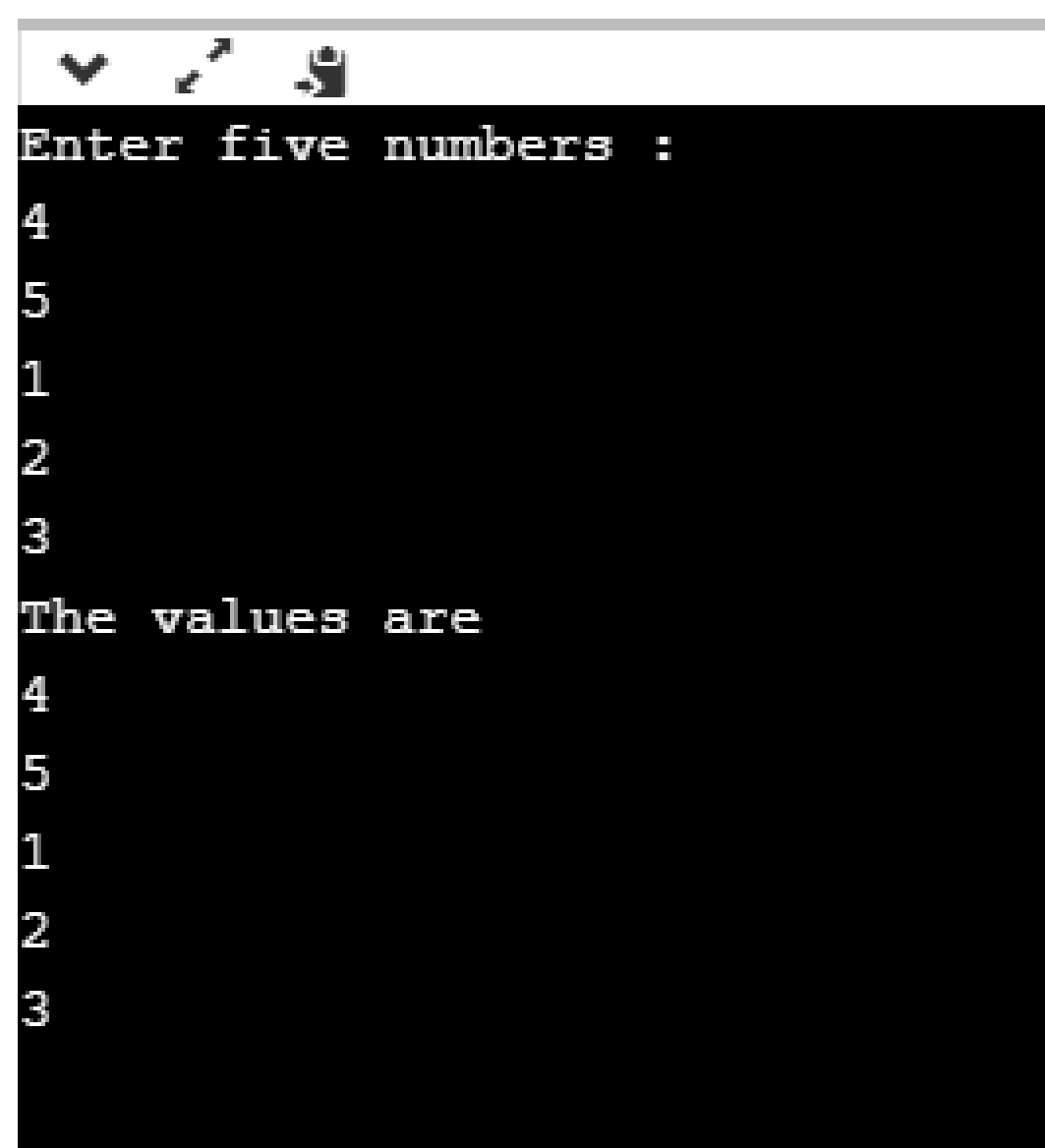
}

}

```

In the above code, we declare an array of integer type and an array of integer pointers. We have defined the 'for' loop, which iterates through the elements of an array 'ptr1', and on each iteration, the address of element of ptr1 at index 'i' gets stored in the ptr2 at index 'i'.

Output



```

Enter five numbers :
4
5
1
2
3
The values are
4
5
1
2
3

```

Till now, we have learnt the array of pointers to an integer. Now, we will see how to create the array of pointers to strings.

Array of Pointer to Strings

An array of pointer to strings is an array of character pointers that holds the address of the first character of a string or we can say the base address of a string.

The following are the differences between an array of pointers to string and two-dimensional array of characters:

- An array of pointers to string is more efficient than the two-dimensional array of characters in case of memory consumption because an array of pointer to strings consumes less memory than the two-dimensional array of characters to store the strings.
- In an array of pointers, the manipulation of strings is comparatively easier than in the case of 2d array. We can also easily change the position of the strings by using the pointers.

Let's see how to declare the array of pointers to string.

First, we declare the array of pointer to string:

```
char *names[5] = {"john",  
                  "Peter",  
                  "Marco",  
                  "Devin",
```

```
"Ronan"};
```

In the above code, we declared an array of pointer names as 'names' of size 5. In the above case, we have done the initialization at the time of declaration, so we do not need to mention the size of the array of a pointer. The above code can be re-written as:

```
char *names[ ] = {"john",  
                  "Peter",  
                  "Marco",  
                  "Devin",  
                  "Ronan"};
```

In the above case, each element of the 'names' array is a string literal, and each string literal would hold the base address of the first character of a string. For example, names[0] contains the base address of "john", names[1] contains the base address of "Peter", and so on. It is not guaranteed that all the string literals will be stored in the contiguous memory location, but the characters of a string literal are stored in a contiguous memory location.

Let's create a simple example.

```
#include <iostream>  
  
using namespace std;  
  
int main()  
{  
  
    char *names[5] = {"john",
```

```

        "Peter",

        "Marco",

        "Devin",

        "Ronan"};

for(int i=0;i<5;i++)

{

    std::cout << names[i] << std::endl;

}

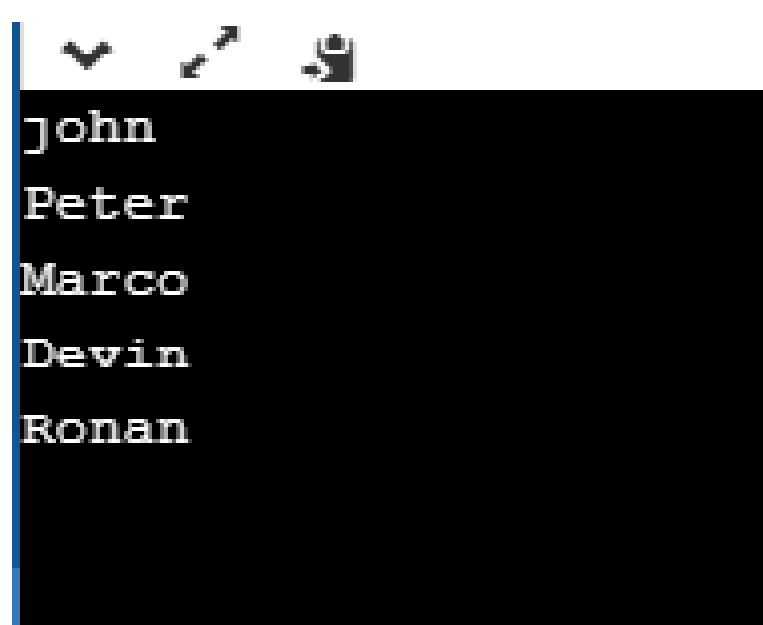
return 0;

}

```

In the above code, we have declared an array of char pointer holding 5 string literals, and the first character of each string is holding the base address of the string.

Output



```

john
Peter
Marco
Devin
Ronan

```

C++ Void Pointer

A void pointer is a general-purpose pointer that can hold the address of any data type, but it is not associated with any data type.

Syntax of void pointer

```
void * ptr;
```

In C++, we cannot assign the address of a variable to the variable of a different data type. Consider the following example:

```
int * ptr; // integer pointer declaration
```

```
float a=10.2; // floating variable initialization
```

```
ptr= &a; // This statement throws an error.
```

In the above example, we declare a pointer of type integer, i.e., ptr and a float variable, i.e., 'a'. After declaration, we try to store the address of 'a' variable in 'ptr', but this is not possible in C++ as the variable cannot hold the address of different data types.

Let's understand through a simple example.

```
#include <iostream.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```

int *ptr;

float f=10.3;

ptr = &f; // error

std::cout << "The value of *ptr is : " <<*ptr<< std::endl;

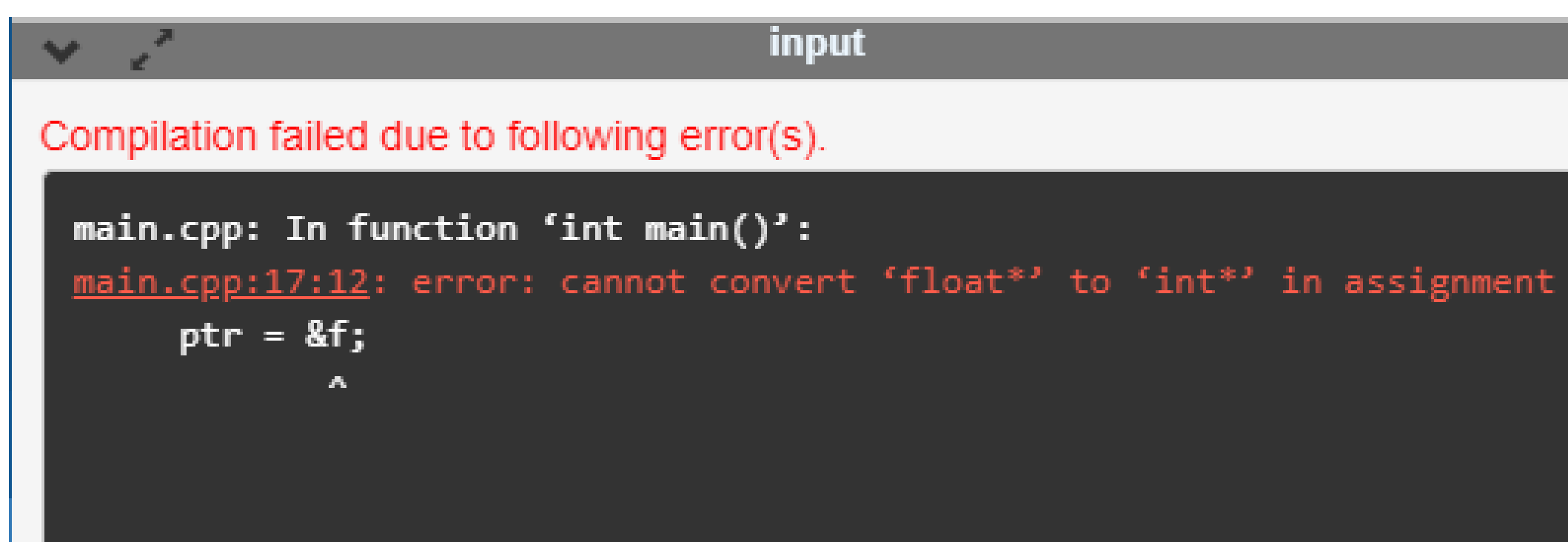
return 0;

}

```

In the above program, we declare a pointer of integer type and variable of float type. An integer pointer variable cannot point to the float variable, but it can point to an only integer variable.

Output



The screenshot shows a window titled "input" with a dark background. It displays a compilation error message in red text: "Compilation failed due to following error(s)." Below this, the error details are shown in white text on a dark background: "main.cpp: In function 'int main()':" followed by "main.cpp:17:12: error: cannot convert 'float*' to 'int*' in assignment". The code snippet "ptr = &f;" is shown with a caret (^) pointing to the ampersand (&) character.

C++ has overcome the above problem by using the C++ void pointer as a void pointer can hold the address of any data type.

Let's look at a simple example of void pointer.

```
#include <iostream>
```

```

using namespace std;

int main()

{

    void * ptr; // void pointer declaration

    int a=9; // integer variable initialization

    ptr=&a; // storing the address of 'a' variable in a void pointer variable.

    std::cout << &a << std::endl;

    std::cout << ptr << std::endl;

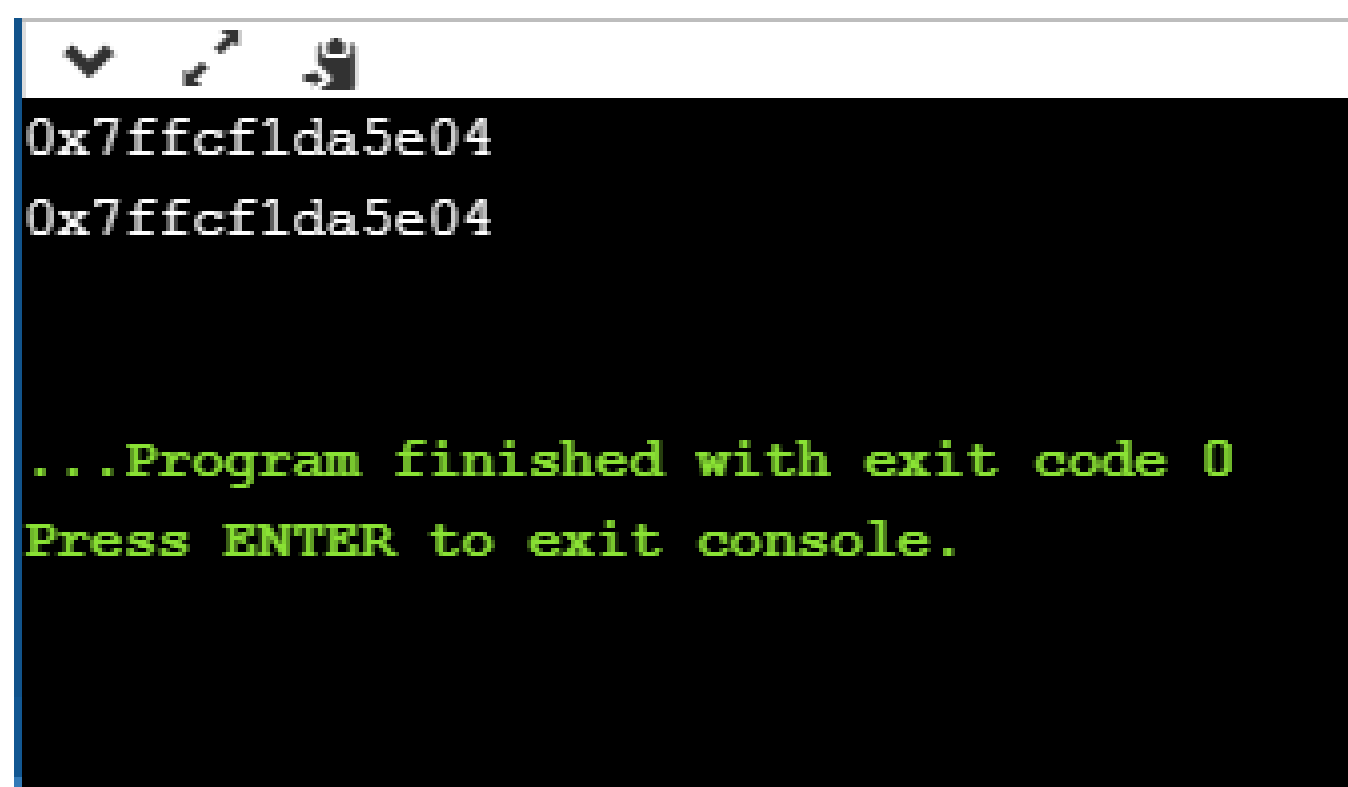
    return 0;

}

```

In the above program, we declare a void pointer variable and an integer variable where the void pointer contains the address of an integer variable.

Output



```

0x7ffcf1da5e04
0x7ffcf1da5e04

...Program finished with exit code 0
Press ENTER to exit console.

```

Difference between void pointer in C and C++

In C, we can assign the void pointer to any other pointer type without any typecasting, whereas in C++, we need to typecast when we assign the void pointer type to any other pointer type.

Let's understand through a simple example.

In C,

```
#include <stdio.h>

int main()

{

    void *ptr; // void pointer declaration

    int *ptr1; // integer pointer declaration

    int a = 90; // integer variable initialization

    ptr = &a; // storing the address of 'a' in ptr

    ptr1 = ptr; // assigning void pointer to integer pointer type.

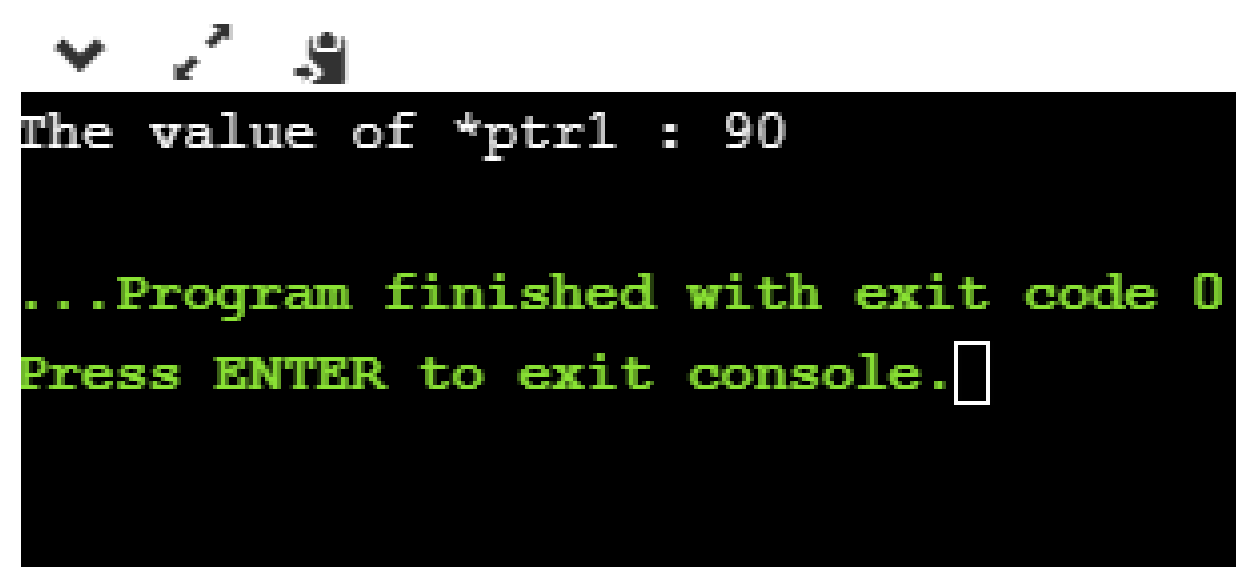
    printf("The value of *ptr1 : %d", *ptr1);

    return 0;

}
```

In the above program, we declare two pointers 'ptr' and 'ptr1' of type void and integer, respectively. We also declare the integer type variable, i.e., 'a'. After declaration, we assign the address of 'a' variable to the pointer 'ptr'. Then, we assign the void pointer to the integer pointer, i.e., ptr1 without any typecasting because in C, we do not need to typecast while assigning the void pointer to any other type of pointer.

Output



```
The value of *ptr1 : 90
...Program finished with exit code 0
Press ENTER to exit console.
```

InC++,

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
void * ptr; // void pointer declaration
```

```
int * ptr1; // integer pointer declaration
```

```
int data=10; // integer variable initialization
```

```
ptr=&data; // storing the address of data variable in void pointer variable
```

```

ptr1=(int *)ptr; // assigning void pointer to integer pointer

std::cout << "The value of *ptr1 is : " << *ptr1 << std::endl;

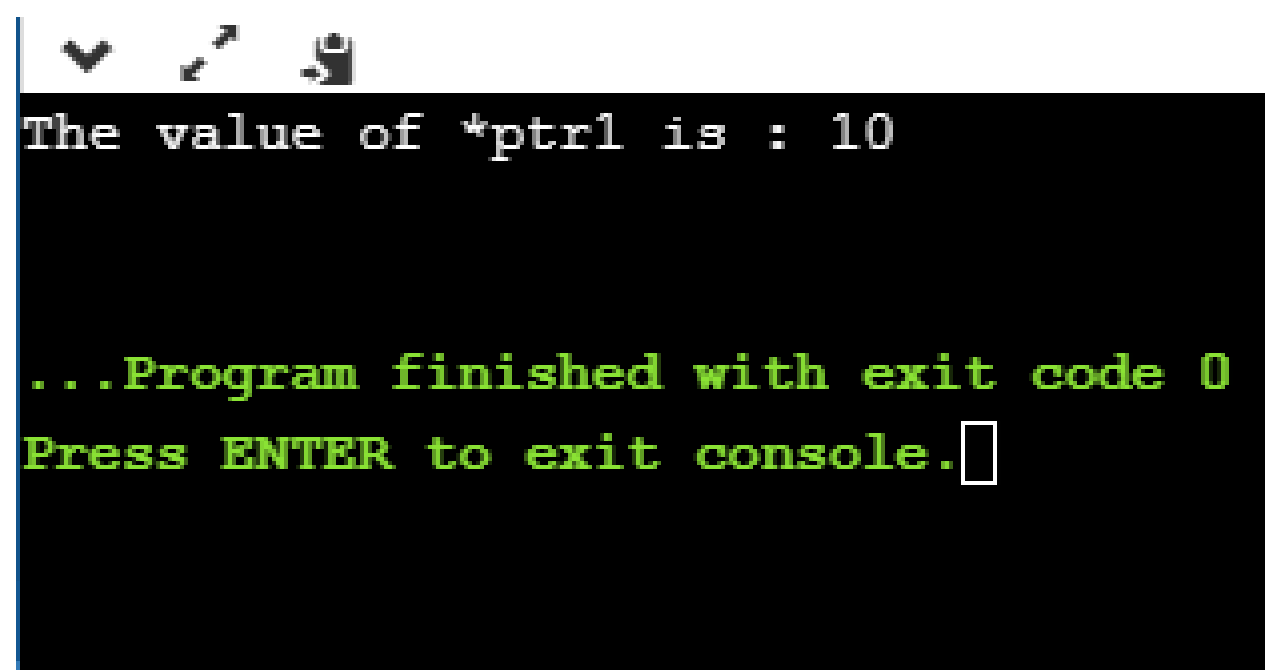
return 0;

}

```

In the above program, we declare two pointer variables of type void and int type respectively. We also create another integer type variable, i.e., 'data'. After declaration, we store the address of variable 'data' in a void pointer variable, i.e., ptr. Now, we want to assign the void pointer to integer pointer, in order to do this, we need to apply the cast operator, i.e., (int *) to the void pointer variable. This cast operator tells the compiler which type of value void pointer is holding. For casting, we have to type the data type and * in a bracket like (char *) or (int *).

Output



```

The value of *ptr1 is : 10

...Program finished with exit code 0
Press ENTER to exit console.

```

C++ References

Till now, we have read that C++ supports two types of variables:

- An ordinary variable is a variable that contains the value of some type. For example, we create a variable of type `int`, which means that the variable can hold the value of type integer.
- A pointer is a variable that stores the address of another variable. It can be dereferenced to retrieve the value to which this pointer points to.
- There is another variable that C++ supports, i.e., references. It is a variable that behaves as an alias for another variable.

How to create a reference?

Reference can be created by simply using an ampersand (`&`) operator. When we create a variable, then it occupies some memory location. We can create a reference of the variable; therefore, we can access the original variable by using either name of the variable or reference. For example,

```
int a=10;
```

Now, we create the reference variable of the above variable.

```
int &ref=a;
```

The above statement means that 'ref' is a reference variable of 'a', i.e., we can use the 'ref' variable in place of 'a' variable.

C++ provides two types of references:

- References to non-const values
- References as aliases

References to non-const values

It can be declared by using & operator with the reference type variable.

```
#include <iostream>

using namespace std;

int main()

{

    int a=10;

    int &value=a;

    std::cout << value << std::endl;

    return 0;

}
```

Output

10

References as aliases

References as aliases is another name of the variable which is being referenced.

Forexample,

- i. `int a=10; // 'a' is a variable.`

ii. `int &b=a; // 'b' reference to a.`

iii. `int &c=a; // 'c' reference to a.`

Let's look at a simple example.

```
#include <iostream>

using namespace std;

int main()
{

    int a=70; // variable initialization

    int &b=a;

    int &c=a;

    std::cout << "Value of a is :" <<a<< std::endl;

    std::cout << "Value of b is :" <<b<< std::endl;

    std::cout << "Value of c is :" <<c<< std::endl;

    return 0;}
```

In the above code, we create a variable 'a' which contains a value '70'. We have declared two reference variables, i.e., b and c, and both are referring to the same variable 'a'. Therefore, we can say that 'a' variable can be accessed by 'b' and 'c' variable.

Output

Value of a is :70

Value of b is :70

Value of c is :70

Properties of References

The following are the properties of references:

Initialization

It must be initialized at the time of the declaration.

```
#include <iostream>

using namespace std;

int main()

{

    int a=10; // variable initialization

    int &b=a; // b reference to a

    std::cout << "value of a is " <<b<< std::endl;

    return 0;

}
```

In the above code, we have created a reference variable, i.e., 'b'. At the time of declaration, 'a' variable is assigned to 'b'. If we do not assign at the time of declaration, then the code would look like:

```
int &b;
```

```
&b=a;
```

The above code will throw a compile-time error as 'a' is not assigned at the time of declaration.

Output

```
value of a is 10
```

Reassignment

It cannot be reassigned means that the reference variable cannot be modified.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
int x=11; // variable initialization
```

```
int z=67;
```

```
int &y=x; // y reference to x
```

```
int &y=z; // y reference to z, but throws a compile-time error.
```

```
return 0;}
```

In the above code, 'y' reference variable is referring to 'x' variable, and then 'z' is assigned to 'y'. But this reassignment is not possible with the reference variable, so it throws a compile-time error.

Compile-time error

```
main.cpp: In function 'int main()':
```

```
main.cpp:18:9: error: redeclaration of 'int& y'
```

```
int &y=z; // y reference to z, but throws a compile-time error.
```

```
^
```

```
main.cpp:17:9: note: 'int& y' previously declared here
```

```
int &y=x; // y reference to x
```

```
^
```

Function Parameters

References can also be passed as a function parameter. It does not create a copy of the argument and behaves as an alias for a parameter. It enhances the performance as it does not create a copy of the argument.

Let's understand through a simple example.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```

{

int a=9; // variable initialization

int b=10; // variable initialization

swap(a, b); // function calling

std::cout << "value of a is :" <<a<< std::endl;

std::cout << "value of b is :" <<b<< std::endl;

return 0;

}

void swap(int &p, int &q) // function definition

{

int temp; // variable declaration

temp=p;

p=q;

q=temp;

}

```

In the above code, we are swapping the values of 'a' and 'b'. We have passed the variables 'a' and 'b' to the swap() function. In swap() function, 'p' is referring to 'a' and 'q' is referring to 'b'. When we swap the values of 'p' and 'q' means that the values of 'a' and 'b' are also swapped.

Output

```
value of a is :10
```

```
value of b is :9
```

References as shortcuts

With the help of references, we can easily access the nested data.

```
#include <iostream>
```

```
using namespace std;
```

```
struct profile
```

```
{
```

```
int id;
```

```
};
```

```
struct employee
```

```
{
```

```
profile p;
```

```
};
```

```
int main()
```

```
{
```

```
employee e;
```

```
int &ref=e.p.id;  
  
ref=34;  
  
std::cout << e.p.id << std::endl;  
  
}
```

In the above code, we are trying to access the 'id' of the profile struct of the employee. We generally access this member by using the statement e.p.id, but this would be a tedious task if we have multiple access to this member. To avoid this situation, we create a reference variable, i.e., ref, which is another name of 'e.p.id'.

Output

34

C++ Reference vs Pointer

C++ reference and pointer seem to be similar, but there are some differences that exist between them. A reference is a variable which is another name of the existing variable, while the pointer is variable that stores the address of another variable.

What is Reference?

A reference is a variable that is referred to as another name for an already existing variable.

The reference of a variable is created by storing the address of another variable.

A reference variable can be considered as a constant pointer with automatic indirection.

Here, automatic indirection means that the compiler automatically applies the indirection operator (*).

Example of reference:

i. `int &a = i;`

In the above declaration, 'a' is an alias name for 'i' variable. We can also refer to the 'i' variable through 'a' variable also.

Let's understand through an example.

```
#include <iostream>

using namespace std;

int main()

{

    int i=8;    // variable initialization

    int &a=i; // creating a reference variable

    cout<<"The value of 'i' variable is :"<<a;

    return 0;

}
```

In the above code, we have created a reference variable, i.e., 'a' for 'i' variable. After creating a reference variable, we can access the value of 'i' with the help of 'a' variable.

What is Pointer?

A pointer is a variable that contains the address of another variable. It can be dereferenced with the help of (*) operator to access the memory location to which the pointer points.

Differences between Reference and Pointer

Definition

A reference variable is another name for an already existing variable. It is mainly used in 'pass by reference' where the reference variable is passed as a parameter to the function and the function to which this variable is passed works on the original copy of the variable.

Let's understand through a simple example.

```
#include <iostream>

using namespace std;

void func(int &);

int main()
{

    int a=10;

    std::cout << "Value of 'a' is :" << a << std::endl;

    func(a);

    std::cout << "Now value of 'a' is :" << a << std::endl;

    return 0;
```

```

    }

    void func(int &m)

    {

        m=8;

    }

```

Output:

Value of 'a' is :10

Now value of 'a' is :8

Whereas, **Pointer** is a variable that stores the address of another variable. It makes the programming easier as it holds the memory address of some variable.

Declaration

We can declare a reference variable by adding a '&' symbol before a variable. If this symbol is used in the expression, then it will be treated as an address operator.

Before using a pointer variable, we should declare a pointer variable, and this variable is created by adding a '*' operator before a variable.

Reassignment

We cannot reassign the reference variable. Now, we take a simple example as given below:

```

#include <iostream>

using namespace std;

```

```

void func(int &);

int main()

{

    int i;  // variable declaration

    int k;  // variable declaration

    int &a=i;

    int &a=k; // error

    return 0;

}

```

The above code shows the error that multiple declarations of `int &a` are not allowed. Therefore, the above program concludes that reassignment operation is not valid for the reference variable.

Whereas, the pointers can be re-assigned. This reassignment is useful when we are working with the data structures such as linked list, trees, etc.

Memory Address

In the case of reference, both the reference and actual variable refer to the same address. The new variable will not be assigned to the reference variable until the actual variable is either deleted or goes out of the scope.

Let's understand this scenario through an example.

```

#include <iostream>

using namespace std;

void func(int &);

int main()
{

    int i;

    int &a=i;

    std::cout << "The address of 'a' variable is : " <<&a<< std::endl;

    std::cout << "The address of 'i' variable is : " <<&i<< std::endl;

    return 0;

}

```

Output:

The address of 'a' variable is : 0x7fff078e7e44

The above output shows that both the reference variable and the actual variable have the same address.

In the case of pointers, both the pointer variable and the actual variable will have different memory addresses. Let's understand this through an example.

```

#include <iostream>

using namespace std;

```

```

int main()

{

    int k;

    int *p;

    p=&k;

    cout<<"The memory address of p variable is :"<<&p;

    cout<<"\nThe memory address of k variable is :"<<&k;

    return 0;

}

```

Output:

The memory address of p variable is :0x7ffcc5c164b8

NULL value

We cannot assign the NULL value to the reference variable, but the pointer variable can be assigned with a NULL value.

Indirection

Pointers can have pointer to pointer offering more than one level of indirection.

```

#include <iostream>

using namespace std;

```

```

int main( )

{

    int *p;

    int a=8;

    int **q;

    p=&a;

    q=&p;

    std::cout << "The value of q is : " <<*q<< std::endl;

    return 0;

}

```

In the above code, the pointer 'p' is pointing to variable 'a' while 'q' is a double pointer which is pointing to 'p'. Therefore, we can say that the value of 'p' would be the address of 'a' variable and the value of 'q' variable would be the address of 'p' variable.

Output:

The value of q is : 0x7ffd104891dc

In the case of References, reference to reference is not possible. If we try to do c++ program will throw a compile-time error

Let's understand this scenario through an example.

```

#include <iostream>

```

```

using namespace std;

int main()

{

    int a=8; // variable initialization

    int &p=a; // creating a reference variable for 'a' variable.

    int &&q=p; // reference to reference is not valid, it throws an error.

    return 0;

}

```

Output:

```

main.cpp: In function 'int main()':
main.cpp:18:10: error: cannot bind 'int' lvalue to 'int&&'
int &&q=p;

```

Arithmetic Operations

As we know that arithmetic operations can be applied to the pointers named as "**Pointer Arithmetic**", but arithmetic operations cannot be applied on the references. There is no word, i.e., Reference Arithmetic exists in C++.

Let's see a simple example of Pointers.

```

#include <iostream>

using namespace std;

```

```

int main()

{

    int a[]={1,2,3,4,5}; // array initialization

    int *ptr; // pointer declaration

    ptr=a; assigning base address to pointer ptr.

    cout<<"The value of *ptr is :"<<*ptr;

    ptr=ptr+1; // incrementing the value of ptr by 1.

    std::cout << "\nThe value of *ptr is: " <<*ptr<< std::endl;

    return 0;

}

```

Output:

The value of *ptr is :1

The value of *ptr is: 2

Let's understand the references through an example.

```

#include <iostream>

```

```

using namespace std;

```

```

int main()

```

```

{

```

```
int value=90; // variable declaration

int &a=value; // assigning value to the reference

&a=&a+5 // arithmetic operation is not possible with reference variable, it thro
ws an error.

return 0;

}
```

The above code will throw a compile-time error as arithmetic operations are not allowed with references.

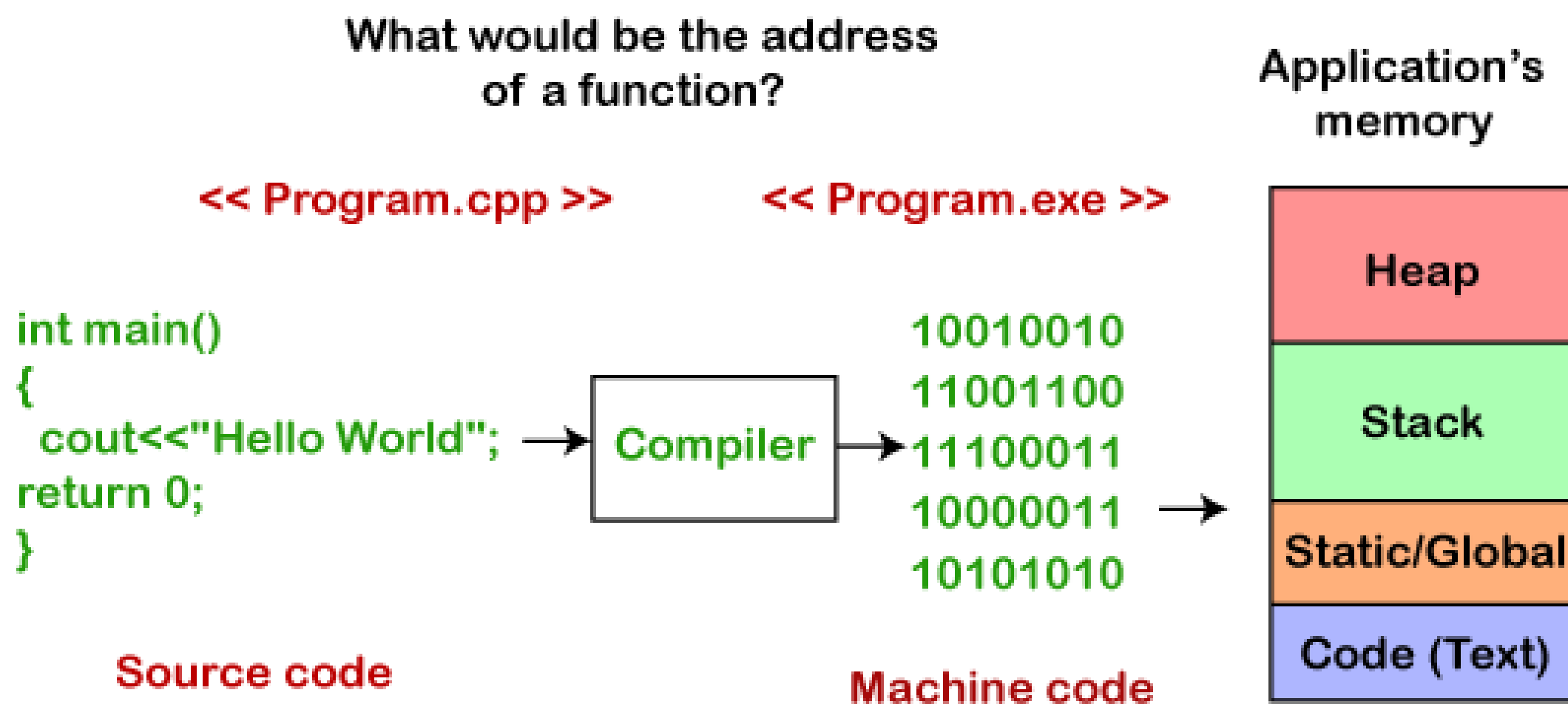
Function Pointer in C++

As we know that pointers are used to point some variables; similarly, the function pointer is a pointer used to point functions. It is basically used to store the address of a function. We can call the function by using the function pointer, or we can also pass the pointer to another function as a parameter.

They are mainly useful for event-driven applications, callbacks, and even for storing the functions in arrays.

What is the address of a function?

Function Pointers



Computer only understands the low-level language, i.e., binary form. The program we write in C++ is always in high-level language, so to convert the program into binary form, we use compiler. Compiler is a program that converts source code into an executable file. This executable file gets stored in RAM. The CPU starts the execution from the `main()` method, and it reads the copy in RAM but not the original file.

All the functions and machine code instructions are data. This data is a bunch of bytes, and all these bytes have some address in RAM. The function pointer contains RAM address of the first instruction of a function.

Syntax for Declaration

The following is the syntax for the declaration of a function pointer:

```
int (*FuncPtr) (int,int);
```

The above syntax is the function declaration. As functions are not simple as variables, but C++ is a type safe, so function pointers have return type and parameter list. In the above

syntax, we first supply the return type, and then the name of the pointer, i.e., FuncPtr which is surrounded by the brackets and preceded by the pointer symbol, i.e., (*). After this, we have supplied the parameter list (int,int). The above function pointer can point to any function which takes two integer parameters and returns integer type value.

Address of a function

We can get the address of a function very easily. We just need to mention the name of the function, we do not need to call the function.

Let's illustrate through an example.

```
#include <iostream>

using namespace std;

int main()

{

    std::cout << "Address of a main() function is : " << &main << std::endl;

    return 0;

}
```

In the above program, we are displaying the address of a main() function. To print the address of a main() function, we have just mentioned the name of the function, there is no bracket not parameters. Therefore, the name of the function by itself without any brackets or parameters means the address of a function.

We can use the alternate way to print the address of a function, i.e., &main.

Calling a function indirectly

We can call the function with the help of a function pointer by simply using the name of the function pointer. The syntax of calling the function through the function pointer would be similar as we do the calling of the function normally.

Let's understand this scenario through an example.

```
#include <iostream>

using namespace std;

int add(int a , int b)

{

    return a+b;

}

int main()

{

    int (*funcptr)(int,int); // function pointer declaration

    funcptr=add; // funcptr is pointing to the add function

    int sum=funcptr(5,5);

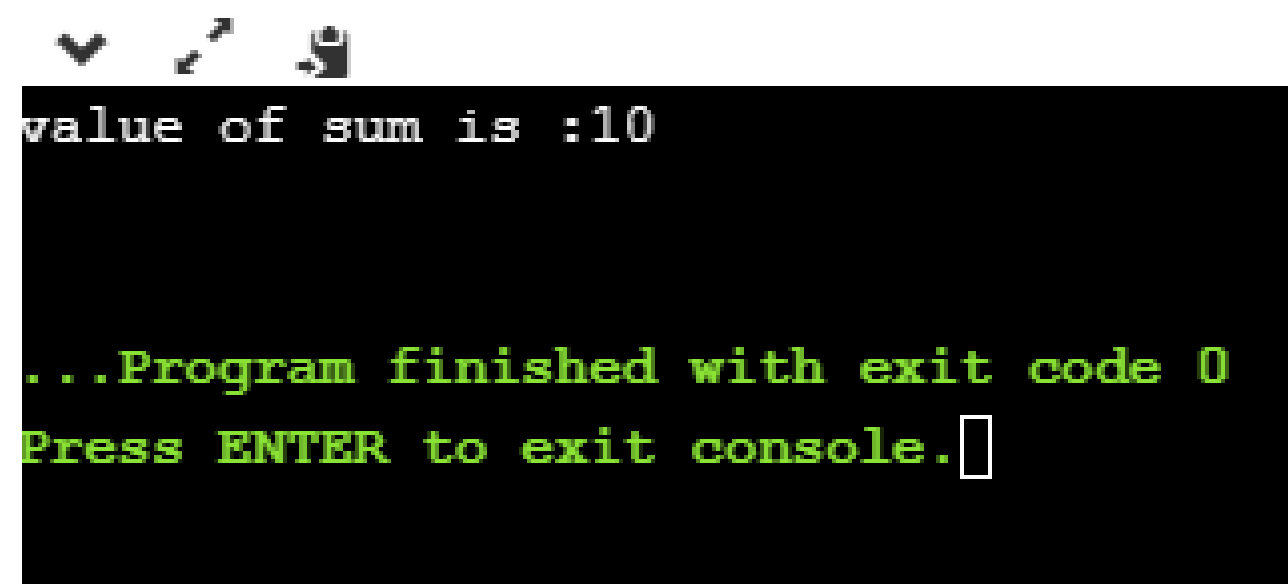
    std::cout << "value of sum is :" << sum << std::endl;

    return 0;
```

```
}
```

In the above program, we declare the function pointer, i.e., `int (*funcptr)(int,int)` and then we store the address of `add()` function in `funcptr`. This implies that `funcptr` contains the address of `add()` function. Now, we can call the `add()` function by using `funcptr`. The statement `funcptr(5,5)` calls the `add()` function, and the result of `add()` function gets stored in `sum` variable.

Output:



```
value of sum is :10

...Program finished with exit code 0
Press ENTER to exit console.
```

Let's look at another example of function pointer.

```
#include <iostream>
```

```
using namespace std;
```

```
void printname(char *name)
```

```
{
```

```
    std::cout << "Name is :" << name << std::endl;
```

```
}
```

```

int main()

{

    char s[20]; // array declaration

    void (*ptr)(char*); // function pointer declaration

    ptr=printname; // storing the address of printname in ptr.

    std::cout << "Enter the name of the person: " << std::endl;

    cin>>s;

    cout<<s;

    ptr(s); // calling printname() function

    return 0;

}

```

In the above program, we define the function `printname()` which contains the char pointer as a parameter. We declare the function pointer, i.e., `void (*ptr)(char*)`. The statement `ptr=printname` means that we are assigning the address of `printname()` function to `ptr`. Now, we can call the `printname()` function by using the statement `ptr(s)`.

Output:

```
Enter the name of the person:
john
john
Name is :john

...Program finished with exit code 0
Press ENTER to exit console.
```

Passing a function pointer as a parameter

The function pointer can be passed as a parameter to another function.

Let's understand through an example.

```
#include <iostream>

using namespace std;

void func1()

{

    cout<<"func1 is called";

}

void func2(void (*funcptr)())

{

    funcptr();

}
```

```

}

int main()

{

    func2(func1);

    return 0;

}

```

In the above code, the `func2()` function takes the function pointer as a parameter. The `main()` method calls the `func2()` function in which the address of `func1()` is passed. In this way, the `func2()` function is calling the `func1()` indirectly.

Output:

```

func1 is called

...Program finished with exit code 0
Press ENTER to exit console.

```

What is Memory Management?

Memory management is a process of managing computer memory, assigning the memory space to the programs to improve the overall system performance.

Why is memory management required?

As we know that arrays store the homogeneous data, so most of the time, memory is allocated to the array at the declaration time. Sometimes the situation arises when the exact memory is not determined until runtime. To avoid such a situation, we declare an array with a maximum size, but some memory will be unused. To avoid the wastage of memory, we use the new operator to allocate the memory dynamically at the run time.

Memory Management Operators

In C language, we use the `malloc()` or `calloc()` functions to allocate the memory dynamically at runtime, and `free()` function is used to deallocate the dynamically allocated memory. C++ also supports these functions, but C++ also defines unary operators such as `new` and `delete` to perform the same tasks, i.e., allocating and freeing the memory.

New operator

A `new` operator is used to create the object while a `delete` operator is used to delete the object. When the object is created by using the new operator, then the object will exist until we explicitly use the delete operator to delete the object. Therefore, we can say that the lifetime of the object is not related to the block structure of the program.

Syntax

```
pointer_variable = new data-type
```

The above syntax is used to create the object using the new operator. In the above syntax, '`pointer_variable`' is the name of the pointer variable, '`new`' is the operator, and '`data-type`' defines the type of the data.

Example 1:

```
int *p;
```

```
p = new int;
```

In the above example, 'p' is a pointer of type int.

Example 2:

```
float *q;
```

```
q = new float;
```

In the above example, 'q' is a pointer of type float.

In the above case, the declaration of pointers and their assignments are done separately.

We can also combine these two statements as follows:

```
int *p = new int;
```

```
float *q = new float;
```

Assigning a value to the newly created object

Two ways of assigning values to the newly created object:

- We can assign the value to the newly created object by simply using the assignment operator. In the above case, we have created two pointers 'p' and 'q' of type int and float, respectively. Now, we assign the values as follows:

```
*p = 45;
```

```
*q = 9.8;
```

We assign 45 to the newly created int object and 9.8 to the newly created float object.

- We can also assign the values by using new operator which can be done as follows:

```
pointer_variable = new data-type(value);
```

Let's look at some examples.

i. `int *p = new int(45);`

ii. `float *p = new float(9.8);`

How to create a single dimensional array

As we know that new operator is used to create memory space for any data-type or even user-defined data type such as an array, structures, unions, etc., so the syntax for creating a one-dimensional array is given below:

```
pointer-variable = new data-type[size];
```

Examples:

```
int *a1 = new int[8];
```

In the above statement, we have created an array of type int having a size equal to 8 where `p[0]` refers first element, `p[1]` refers the first element, and so on.

Delete operator

When memory is no longer required, then it needs to be deallocated so that the memory can be used for another purpose. This can be achieved by using the delete operator, as shown below:

```
delete pointer_variable;
```

In the above statement, 'delete' is the operator used to delete the existing object, and 'pointer_variable' is the name of the pointer variable.

In the previous case, we have created two pointers 'p' and 'q' by using the new operator, and can be deleted by using the following statements:

```
delete p;
```

```
delete q;
```

The dynamically allocated array can also be removed from the memory space by using the following syntax:

```
delete [size] pointer_variable;
```

In the above statement, we need to specify the size that defines the number of elements that are required to be freed. The drawback of this syntax is that we need to remember the size of the array. But, in recent versions of C++, we do not need to mention the size as follows:

```
delete [ ] pointer_variable;
```

Let's understand through a simple example:

```
#include <iostream>
```

```
using namespace std
```

```
int main()
```

```
{
```

```

int size; // variable declaration

int *arr = new int[size]; // creating an array

cout<<"Enter the size of the array : ";

std::cin >> size; //

cout<<"\nEnter the element : ";

for(int i=0;i<size;i++) // for loop

{

cin>>arr[i];

}

cout<<"\nThe elements that you have entered are :";

for(int i=0;i<size;i++) // for loop

{

cout<<arr[i]<<",";

}

delete arr; // deleting an existing array.

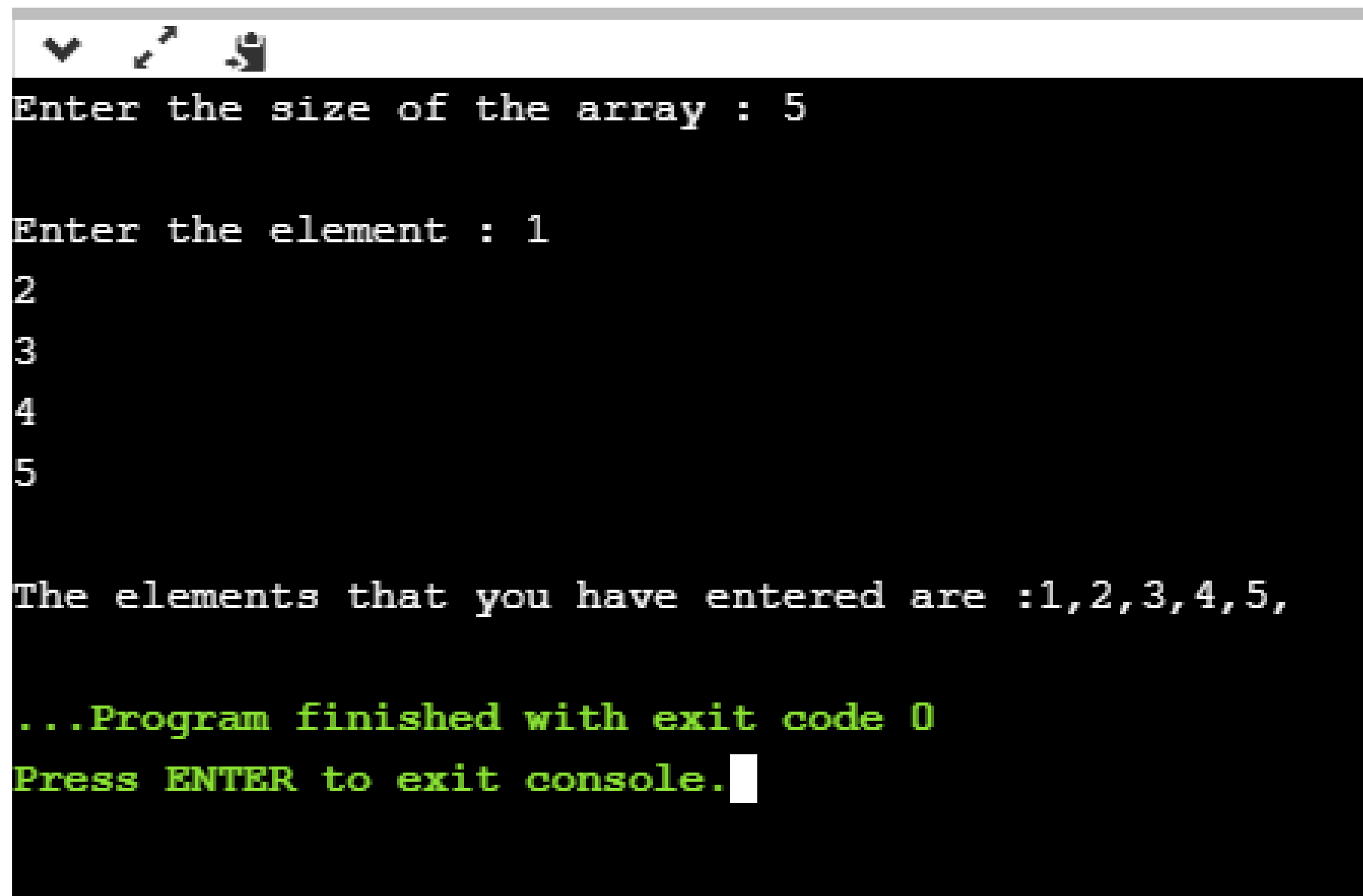
return 0;

}

```

In the above code, we have created an array using the new operator. The above program will take the user input for the size of an array at the run time. When the program completes all the operations, then it deletes the object by using the statement `delete arr`.

Output



```
Enter the size of the array : 5

Enter the element : 1
2
3
4
5

The elements that you have entered are :1,2,3,4,5,

...Program finished with exit code 0
Press ENTER to exit console.
```

Advantages of the new operator

The following are the advantages of the new operator over `malloc()` function:

- It does not use the `sizeof()` operator as it automatically computes the size of the data object.
- It automatically returns the correct data type pointer, so it does not need to use the typecasting.
- Like other operators, the new and delete operator can also be overloaded.

- It also allows you to initialize the data object while creating the memory space for the object.

malloc() vs new in C++

Both the `malloc()` and `new` in C++ are used for the same purpose. They are used for allocating memory at the runtime. But, `malloc()` and `new` have different syntax. The main difference between the `malloc()` and `new` is that the `new` is an operator while `malloc()` is a standard library function that is predefined in a `stdlib.h` header file.

What is new?

The `new` is a memory allocation operator, which is used to allocate the memory at the runtime. The memory initialized by the `new` operator is allocated in a heap. It returns the starting address of the memory, which gets assigned to the variable. The functionality of the `new` operator in C++ is similar to the `malloc()` function, which was used in the C programming language. C++ is compatible with the `malloc()` function also, but the `new` operator is mostly used because of its advantages.

Syntax of new operator

```
type variable = new type(parameter_list);
```

In the above syntax

type: It defines the datatype of the variable for which the memory is allocated by the `new` operator.

variable: It is the name of the variable that points to the memory.

parameter_list: It is the list of values that are initialized to a variable.

The new operator does not use the sizeof() operator to allocate the memory. It also does not use the resize as the new operator allocates sufficient memory for an object. It is a construct that calls the constructor at the time of declaration to initialize an object.

As we know that the new operator allocates the memory in a heap; if the memory is not available in a heap and the new operator tries to allocate the memory, then the exception is thrown. If our code is not able to handle the exception, then the program will be terminated abnormally.

Let's understand the new operator through an example.

```
#include <iostream>

using namespace std;

int main()
{

    int *ptr; // integer pointer variable declaration

    ptr=new int; // allocating memory to the pointer variable ptr.

    std::cout << "Enter the number : " << std::endl;

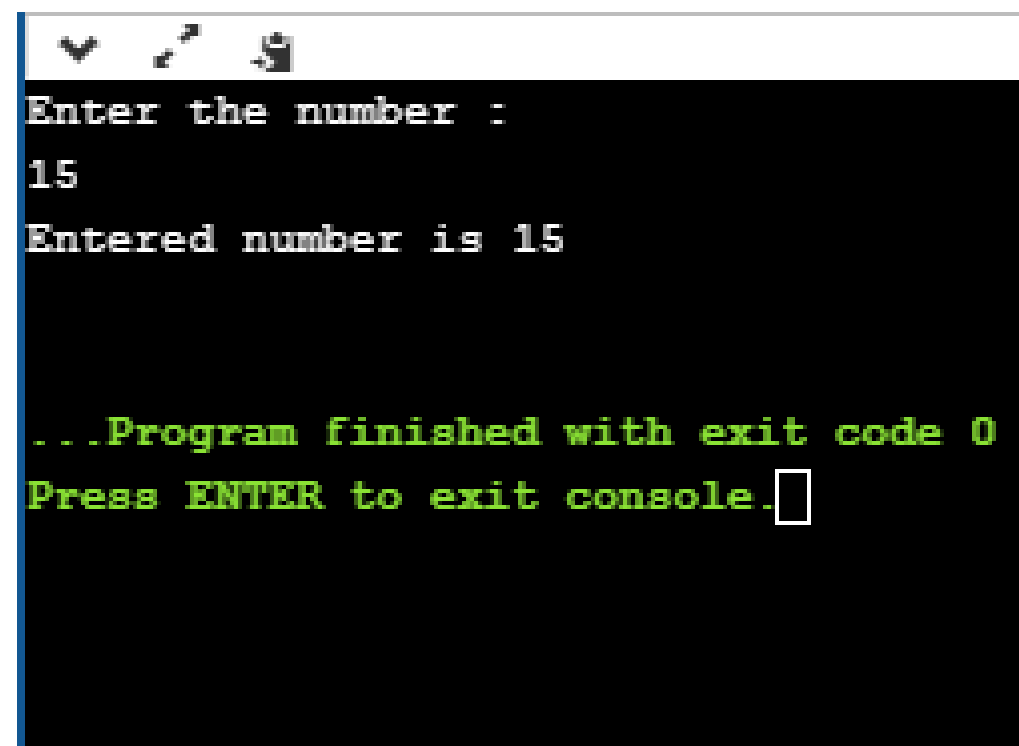
    std::cin >> *ptr;

    std::cout << "Entered number is " << *ptr<< std::endl;

    return 0;
```

}

Output:

A screenshot of a console window with a black background and white text. The text shows a program prompt 'Enter the number :', the user input '15', and the program output 'Entered number is 15'. At the bottom, it says '...Program finished with exit code 0' and 'Press ENTER to exit console.' with a cursor. The window has standard OS icons at the top.

```
Enter the number :  
15  
Entered number is 15  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

What is malloc()?

A `malloc()` is a function that allocates memory at the runtime. This function returns the void pointer, which means that it can be assigned to any pointer type. This void pointer can be further typecast to get the pointer that points to the memory of a specified type.

The syntax of the `malloc()` function is given below:

```
type variable_name = (type *)malloc(sizeof(type));
```

where,

type: it is the datatype of the variable for which the memory has to be allocated.

variable_name: It defines the name of the variable that points to the memory.

(type*): It is used for typecasting so that we can get the pointer of a specified type that points to the memory.

sizeof(): The sizeof() operator is used in the malloc() function to obtain the memory size required for the allocation.

Note: The malloc() function returns the void pointer, so typecasting is required to assign a different type to the pointer. The sizeof() operator is required in the malloc() function as the malloc() function returns the raw memory, so the sizeof() operator will tell the malloc() function how much memory is required for the allocation.

If the sufficient memory is not available, then the memory can be resized using realloc() function. As we know that all the dynamic memory requirements are fulfilled using heap memory, so malloc() function also allocates the memory in a heap and returns the pointer to it. The heap memory is very limited, so when our code starts execution, it marks the memory in use, and when our code completes its task, then it frees the memory by using the free() function. If the sufficient memory is not available, and our code tries to access the memory, then the malloc() function returns the NULL pointer. The memory which is allocated by the malloc() function can be deallocated by using the free() function.

Let's understand through an example.

```
#include <iostream>
```

```
#include <stdlib.h>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```

int len; // variable declaration

std::cout << "Enter the count of numbers : " << std::endl;

std::cin >> len;

int *ptr; // pointer variable declaration

ptr=(int*) malloc(sizeof(int)*len); // allocating memory to the pointer variable

for(int i=0;i<len;i++)

{

    std::cout << "Enter a number : " << std::endl;

    std::cin >> *(ptr+i);

}

std::cout << "Entered elements are : " << std::endl;

for(int i=0;i<len;i++)

{

    std::cout << *(ptr+i) << std::endl;

}

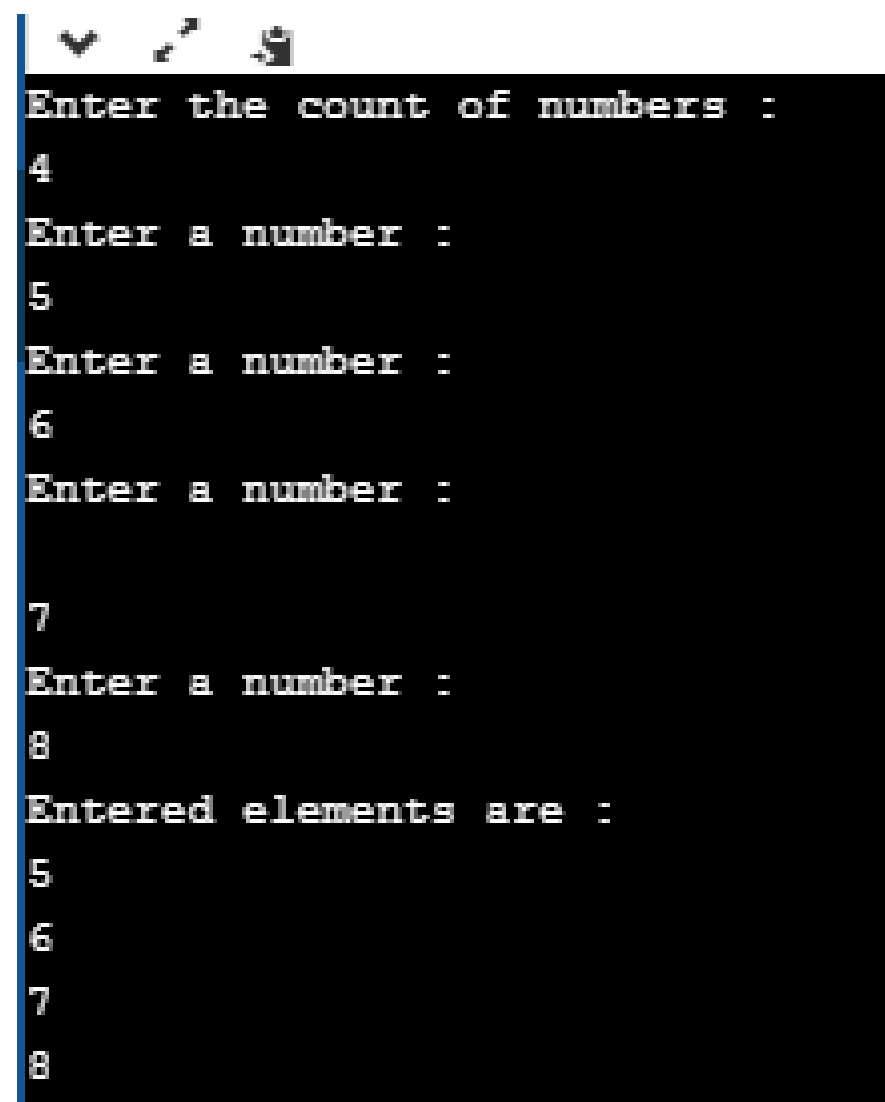
free(ptr);

```

```
    return 0;

}
```

Output:



```
Enter the count of numbers :
4
Enter a number :
5
Enter a number :
6
Enter a number :
7
Enter a number :
8
Entered elements are :
5
6
7
8
```

If we do not use the `free()` function at the correct place, then it can lead to the cause of the dangling pointer. Let's understand this scenario through an example.

```
#include <iostream>
```

```
#include<stdlib.h>
```

```
using namespace std;
```

```
int *func()
```

```
{
```

```
    int *p;
```

```
    p=(int*) malloc(sizeof(int));
```

```

        free(p);

        return p;

    }

    int main()

    {

        int *ptr;

        ptr=func( );

        free(ptr);

        return 0;

    }

```

In the above code, we are calling the func() function. The func() function returns the integer pointer. Inside the func() function, we have declared a *p pointer, and the memory is allocated to this pointer variable using malloc() function. In this case, we are returning the pointer whose memory is already released. The ptr is a dangling pointer as it is pointing to the released memory location. Or we can say ptr is referring to that memory which is not pointed by the pointer.

Till now, we get to know about the new operator and the malloc() function. Now, we will see the differences between the new operator and the malloc() function.

Differences between the malloc() and new

- The new operator constructs an object, i.e., it calls the constructor to initialize an object while `malloc()` function does not call the constructor. The new operator invokes the constructor, and the delete operator invokes the destructor to destroy the object. This is the biggest difference between the `malloc()` and `new`.
- The `new` is an operator, while `malloc()` is a predefined function in the `stdlib` header file.
- The operator `new` can be overloaded while the `malloc()` function cannot be overloaded.
- If the sufficient memory is not available in a heap, then the new operator will throw an exception while the `malloc()` function returns a `NULL` pointer.
- In the new operator, we need to specify the number of objects to be allocated while in `malloc()` function, we need to specify the number of bytes to be allocated.
- In the case of a new operator, we have to use the delete operator to deallocate the memory. But in the case of `malloc()` function, we have to use the `free()` function to deallocate the memory.

Syntax of new operator

`type reference_variable = new type name;`

where,

type: It defines the data type of the reference variable.

reference_variable: It is the name of the pointer variable.

new: It is an operator used for allocating the memory.

typename: It can be any basic data type.

For example,

```
int *p;  
  
p = new int;
```

In the above statements, we are declaring an integer pointer variable. The statement **p = new int;** allocates the memory space for an integer variable.

Syntax of malloc() is given below:

```
int *ptr = (data_type*) malloc(sizeof(data_type));
```

ptr: It is a pointer variable.

data_type: It can be any basic data type.

For example,

```
int *p;  
  
p = (int *) malloc(sizeof(int))
```

The above statement will allocate the memory for an integer variable in a heap, and then stores the address of the reserved memory in 'p' variable.

- On the other hand, the memory allocated using malloc() function can be deallocated using the free() function.

- Once the memory is allocated using the new operator, then it cannot be resized. On the other hand, the memory is allocated using malloc() function; then, it can be reallocated using realloc() function.
- The execution time of new is less than the malloc() function as new is a construct, and malloc is a function.
- The new operator does not return the separate pointer variable; it returns the address of the newly created object. On the other hand, the malloc() function returns the void pointer which can be further typecast in a specified type.

free vs delete in C++

In this topic, we are going to learn about the free() function and delete operator in C++.

free() function

The free() function is used in C++ to de-allocate the memory dynamically. It is basically a library function used in C++, and it is defined in **stdlib.h** header file. This library function is used when the pointers either pointing to the memory allocated using malloc() function or Null pointer.

Syntax of free() function

Suppose we have declared a pointer 'ptr', and now, we want to de-allocate its memory:

```
free(ptr);
```

The above syntax would de-allocate the memory of the pointer variable 'ptr'.

free() parameters

In the above syntax, ptr is a parameter inside the free() function. The ptr is a pointer pointing to the memory block allocated using malloc(), calloc() or realloc function. This pointer can also be null or a pointer allocated using malloc but not pointing to any other memory block.

- If the pointer is null, then the free() function will not do anything.
- If the pointer is allocated using malloc, calloc, or realloc, but not pointing to any memory block then this function will cause undefined behavior.

free() Return Value

The free() function does not return any value. Its main function is to free the memory.

Let's understand through an example.

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int *ptr;
```

```
    ptr = (int*) malloc(5 * sizeof(int));
```

```
cout << "Enter 5 integer" << endl;
```

```
for (int i=0; i<5; i++)
```

```
{
```

```
// *(ptr+i) can be replaced by ptr[i]
```

```
    cin >> ptr[i];
```

```
}
```

```
cout << endl << "User entered value" << endl;
```

```
for (int i=0; i<5; i++)
```

```
{
```

```
    cout << *(ptr+i) << " ";
```

```
}
```

```
free(ptr);
```

```
/* prints a garbage value after ptr is free */
```

```
cout << "Garbage Value" << endl;
```

```

for (int i=0; i<5; i++)

{

    cout << *(ptr+i)<< " ";

}

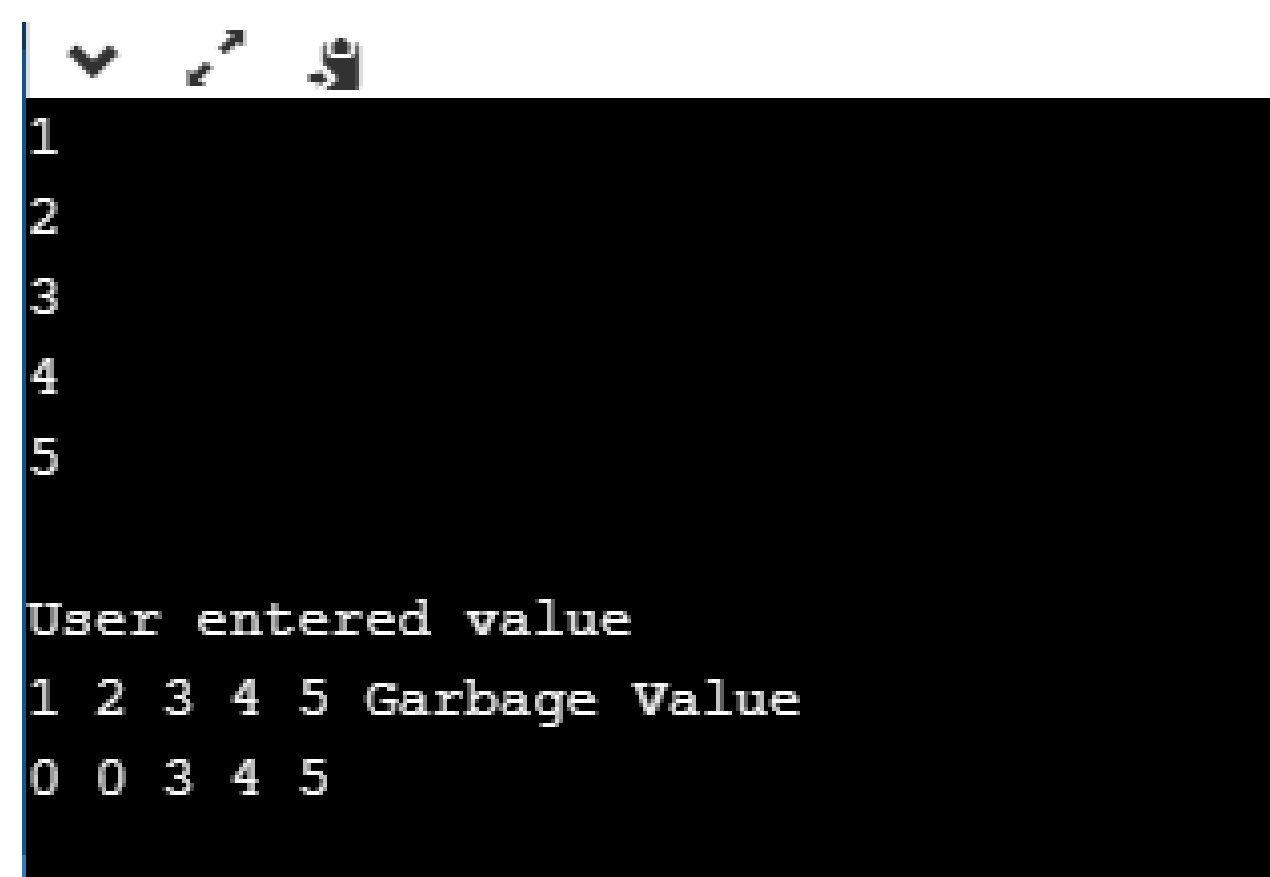
return 0;

}

```

The above code shows how free() function works with malloc(). First, we declare integer pointer *ptr, and then we allocate the memory to this pointer variable by using malloc() function. Now, ptr is pointing to the uninitialized memory block of 5 integers. After allocating the memory, we use the free() function to destroy this allocated memory. When we try to print the value, which is pointed by the ptr, we get a garbage value, which means that memory is de-allocated.

Output



```

1
2
3
4
5

User entered value
1 2 3 4 5 Garbage Value
0 0 3 4 5

```

Let's see how free() function works with a calloc.

```

#include <iostream>

#include <cstdlib>

using namespace std;

int main()

{

    float *ptr; // float pointer declaration

    ptr=(float*)calloc(1,sizeof(float));

    *ptr=6.7;

    std::cout << "The value of *ptr before applying the free() function : " <<*ptr<
< std::endl;

    free(ptr);

    std::cout << "The value of *ptr after applying the free() function : " <<*ptr<< s
td::endl;

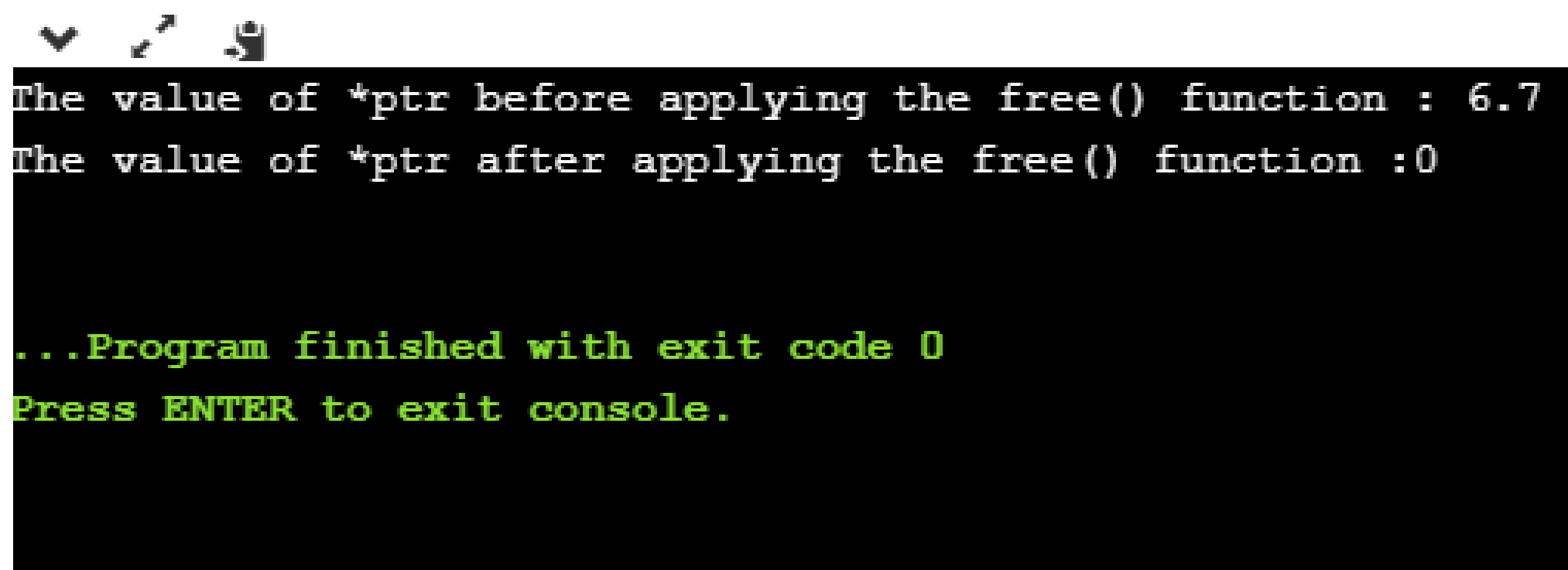
    return 0;

}

```

In the above example, we can observe that free() function works with a calloc(). We use the calloc() function to allocate the memory block to the float pointer ptr. We have assigned a memory block to the ptr that can have a single float type value.

Output:



```
The value of *ptr before applying the free() function : 6.7
The value of *ptr after applying the free() function :0

...Program finished with exit code 0
Press ENTER to exit console.
```

Let's look at another example.

```
#include <iostream>
```

```
#include <cstdlib>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int *ptr1=NULL;
```

```
    int *ptr2;
```

```
    int x=9;
```

```
    ptr2=&x;
```

```
    if(ptr1)
```

```
{
```

```
        std::cout << "Pointer is not Null" << std::endl;
```

```
}
```

```

else

{

    cout<<"Ponter is NULL";

}

free(ptr1);

//free(ptr2); // If this statement is executed, then it gives a runtime error.

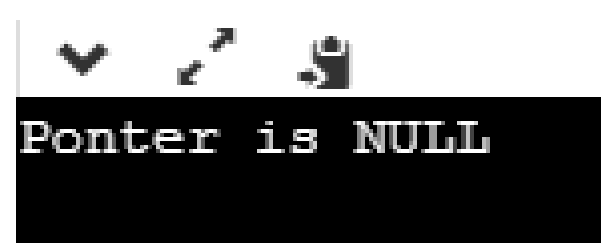
return 0;

}

```

The above code shows how free() function works with a NULL pointer. We have declared two pointers, i.e., ptr1 and ptr2. We assign a NULL value to the pointer ptr1 and the address of x variable to pointer ptr2. When we apply the free(ptr1) function to the ptr1, then the memory block assigned to the ptr is successfully freed. The statement free(ptr2) shows a runtime error as the memory block assigned to the ptr2 is not allocated using malloc or calloc function.

Output



```

Ponter is NULL

```

Delete operator

It is an operator used in C++ programming language, and it is used to de-allocate the memory dynamically. This operator is mainly used either for those pointers which are allocated using a new operator or NULL pointer.

Syntax

```
delete pointer_name
```

For example, if we allocate the memory to the pointer using the new operator, and now we want to delete it. To delete the pointer, we use the following statement:

```
delete p;
```

To delete the array, we use the statement as given below:

```
delete [] p;
```

Some important points related to delete operator are:

- It is either used to delete the array or non-array objects which are allocated by using the new keyword.
- To delete the array or non-array object, we use delete[] and delete operator, respectively.
- The new keyword allocated the memory in a heap; therefore, we can say that the delete operator always de-allocates the memory from the heap
- It does not destroy the pointer, but the value or the memory block, which is pointed by the pointer is destroyed.

Let's look at the simple example of a delete operator.

```
#include <iostream>

#include <cstdlib>

using namespace std;

int main()

{

    int *ptr;

    ptr=new int;

    *ptr=68;

    std::cout << "The value of p is : " << *ptr<< std::endl;

    delete ptr;

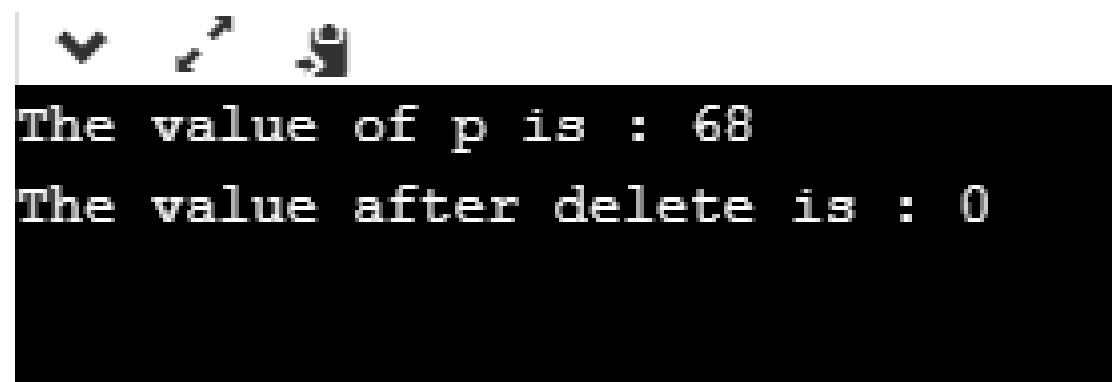
    std::cout << "The value after delete is : " << *ptr<< std::endl;

    return 0;

}
```

In the above code, we use the new operator to allocate the memory, so we use the delete ptr operator to destroy the memory block, which is pointed by the pointer ptr.

Output

A terminal window with a black background and white text. It shows two lines of output: "The value of p is : 68" and "The value after delete is : 0".

```
The value of p is : 68
The value after delete is : 0
```

Let's see how delete works with an array of objects.

```
#include <iostream>

using namespace std;

int main()
{

    int *ptr=new int[5]; // memory allocation using new operator.

    std::cout << "Enter 5 integers:" << std::endl;

    for(int i=1;i<=5;i++)

    {

        cin>>ptr[i];

    }

    std::cout << "Entered values are:" << std::endl;

    for(int i=1;i<=5;i++)

    {

        cout<<*(ptr+i)<<endl;
```

```
}
```

```
delete[] ptr; // deleting the memory block pointed by the ptr.
```

```
std::cout << "After delete, the garbage value:" << std::endl;
```

```
for(int i=1;i<=5;i++)
```

```
{
```

```
cout<<*(ptr+i)<<endl;
```

```
}
```

```
return 0;
```

```
}
```

Output

```
Enter 5 integers:
4
5
3
1
2
Entered values are:
4
5
3
1
2
After delete, the garbage value:
0
5
3
1
2
```

Differences between delete and free()

The following are the differences between delete and free() in C++ are:

- The delete is an operator that de-allocates the memory dynamically while the free() is a function that destroys the memory at the runtime.
- The delete operator is used to delete the pointer, which is either allocated using new operator or a NULL pointer, whereas the free() function is used to delete the pointer that is either allocated using malloc(), calloc() or realloc() function or NULL pointer.
- When the delete operator destroys the allocated memory, then it calls the destructor of the class in C++, whereas the free() function does not call the destructor; it only frees the memory from the heap.

- The `delete()` operator is faster than the `free()` function.

C++ Object Class

C++ OOPs Concepts

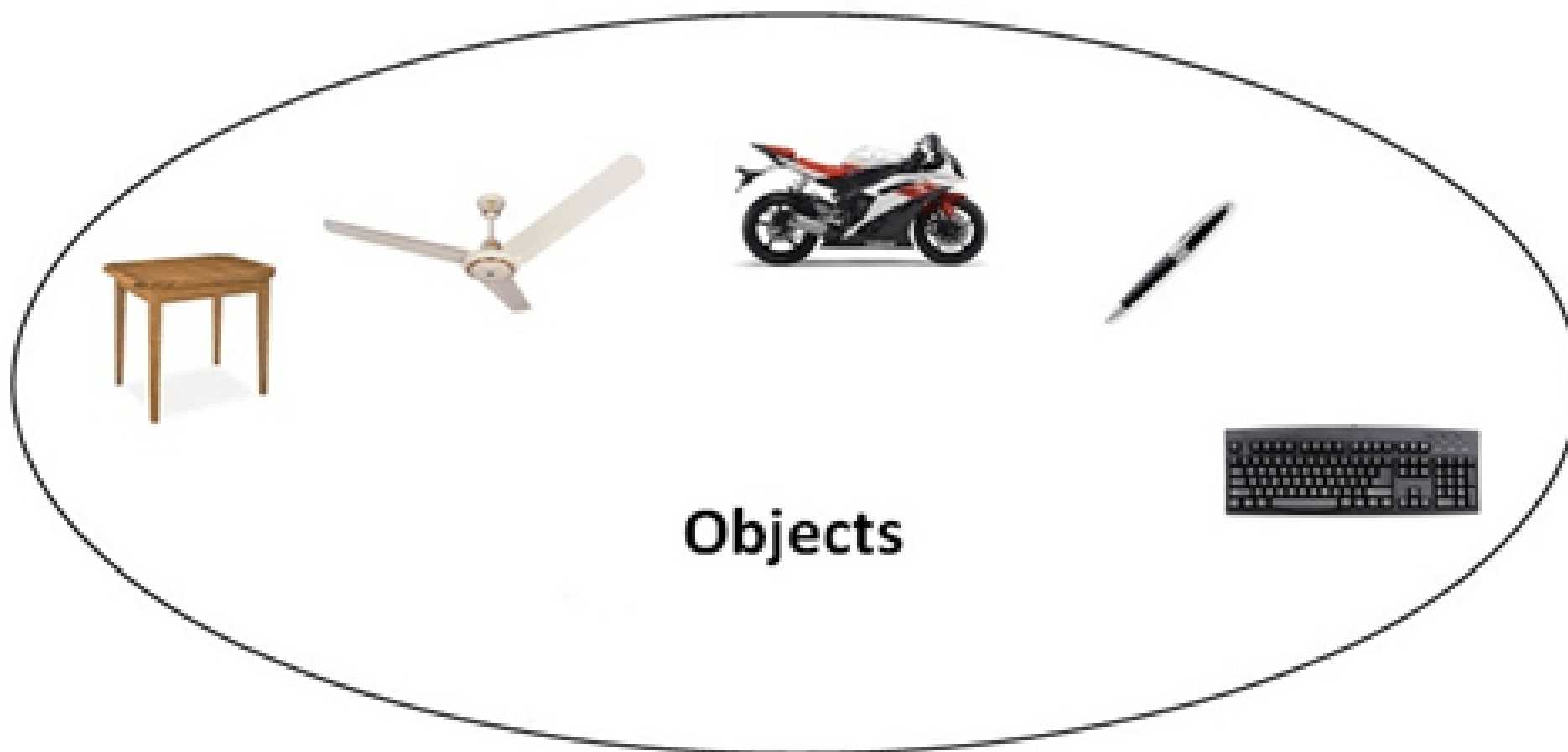
The major purpose of C++ programming is to introduce the concept of object orientation to the C programming language.

Object Oriented Programming is a paradigm that provides many concepts such as inheritance, data binding, polymorphism etc.

The programming paradigm where everything is represented as an object is known as truly object-oriented programming language. **Smalltalk** is considered as the first truly object-oriented programming language.

OOPs (Object Oriented Programming System)

Object means a real world entity such as pen, chair, table etc. **Object-Oriented Programming** is a methodology or paradigm to design a program using classes and objects. It simplifies the software development and maintenance by providing some concepts:



- Object
- Class
- Inheritance
- Polymorphism
- Abstraction
- Encapsulation

Object

Any entity that has state and behavior is known as an object. For example: chair, pen, table, keyboard, bike etc. It can be physical and logical.

Class

Collection of **objects** is called class. It is a logical entity.

Inheritance

When one object acquires all the properties and behaviours of parent object i.e. known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.

Polymorphism

When one task is performed by different ways i.e. known as polymorphism. For example: to convince the customer differently, to draw something e.g. shape or rectangle etc.

In C++, we use Function overloading and Function overriding to achieve polymorphism.

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example: phone call, we don't know the internal processing.

In C++, we use abstract class and interface to achieve abstraction.

Encapsulation

Binding (or wrapping) code and data together into a single unit is known as encapsulation. For example: capsule, it is wrapped with different medicines.

Advantage of OOPs over Procedure-oriented programming language

1. OOPs makes development and maintenance easier where as in Procedure-oriented programming language it is not easy to manage if code grows as project size grows.
2. OOPs provide data hiding whereas in Procedure-oriented programming language a global data can be accessed from anywhere.

3. OOPs provide ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

C++ Object and Class

Since C++ is an object-oriented language, program is designed using objects and classes in C++.

C++ Object

In C++, Object is a real world entity, for example, chair, car, pen, mobile, laptop etc.

In other words, object is an entity that has state and behavior. Here, state means data and behavior means functionality.

Object is a runtime entity, it is created at runtime.

Object is an instance of a class. All the members of the class can be accessed through object.

Let's see an example to create object of student class using s1 as the reference variable.

```
Student s1; //creating an object of Student
```

In this example, Student is the type and s1 is the reference variable that refers to the instance of Student class.

C++ Class

In C++, class is a group of similar objects. It is a template from which objects are created. It can have fields, methods, constructors etc.

Let's see an example of C++ class that has three fields only.

```
class Student
{
    public:

    int id; //field or data member

    float salary; //field or data member

    String name; //field or data member
}
```

C++ Object and Class Example

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

```
#include <iostream>

using namespace std;

class Student {

    public:

    int id; //data member (also instance variable)
```

```

        string name; //data member(also instance variable)

};

int main() {

    Student s1; //creating an object of Student

    s1.id = 201;

    s1.name = "Sonoo Jaiswal";

    cout<<s1.id<<endl;

    cout<<s1.name<<endl;

    return 0;

}

```

Output:

201

Sonoo Jaiswal

C++ Class Example: Initialize and Display data through method

Let's see another example of C++ class where we are initializing and displaying object through method.

```

#include <iostream>

using namespace std;

```

```

class Student {

public:

    int id; //data member (also instance variable)

    string name; //data member (also instance variable)

    void insert(int i, string n)

    {

        id = i;

        name = n;

    }

    void display()

    {

        cout<<id<<" "<<name<<endl;

    }

};

int main(void) {

    Student s1; //creating an object of Student

    Student s2; //creating an object of Student

    s1.insert(201, "Sonoo");

```

```

s2.insert(202, "Nakul");

s1.display();

s2.display();

return 0;

}

```

Output:

```

201  Sonoo

202  Nakul

```

C++ Class Example: Store and Display Employee Information

Let's see another example of C++ class where we are storing and displaying employee information using method.

```

#include <iostream>

using namespace std;

class Employee {

public:

    int id; // data member (also instance variable)

    string name; // data member (also instance variable)

    float salary;

```

```

void insert(int i, string n, float s)

{

    id = i;

    name = n;

    salary = s;

}

void display()

{

    cout<<id<<" "<<name<<" "<<salary<<endl;

}

};

int main(void) {

    Employee e1; //creating an object of Employee

    Employee e2; //creating an object of Employee

    e1.insert(201, "Sonoo",990000);

    e2.insert(202, "Nakul", 29000);

    e1.display();

    e2.display();

```

```
    return 0;

}
```

Output:

```
201  Sonoo  990000
202  Nakul  29000
```

C++ Constructor

In C++, constructor is a special method which is invoked automatically at the time of object creation. It is used to initialize the data members of new object generally. The constructor in C++ has the same name as class or structure.

There can be two types of constructors in C++.

Default constructor

Parameterized constructor

C++ Default Constructor

A constructor which has no argument is known as default constructor. It is invoked at the time of creating object.

Let's see the simple example of C++ default Constructor.

```
#include <iostream>

using namespace std;

class Employee
```

```

{

    public:

        Employee()

        {

            cout<<"Default Constructor Invoked"<<endl;

        }

};

int main(void)

{

    Employee e1; //creating an object of Employee

    Employee e2;

    return 0;

}

```

Output:

```

Default Constructor Invoked

Default Constructor Invoked

```

C++ Parameterized Constructor

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

Let's see the simple example of C++ Parameterized Constructor.

```
#include <iostream>

using namespace std;

class Employee{

public:

    int id; //data member (also instance variable)

    string name; //data member (also instance variable)

    float salary;

    Employee(int i, string n, float s)

    {

        id = i;

        name = n;

        salary = s;

    }

    void display()

    {

        cout<<id<<" "<<name<<" "<<salary<<endl;

    }

}
```

```
};

int main(void) {

    Employee e1 = Employee(101, "Sonoo", 890000); // creating an object of
    Employee

    Employee e2 = Employee(102, "Nakul", 59000);

    e1.display();

    e2.display();

    return 0;

}
```

Output:

```
101  Sonoo  890000

102  Nakul  59000
```

C++ Destructor

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined like constructor. It must have same name as class. But it is prefixed with a tilde sign (~).

Note: C++ destructor cannot have parameters. Moreover, modifiers can't be applied on destructors.

C++ Constructor and Destructor Example

Let's see an example of constructor and destructor in C++ which is called automatically.

```
#include <iostream>

using namespace std;

class Employee

{

public:

    Employee()

    {

        cout<<"Constructor Invoked"<<endl;

    }

    ~Employee()

    {

        cout<<"Destructor Invoked"<<endl;

    }

};

int main(void)

{

    Employee e1; //creating an object of Employee
```

```

Employee e2; //creating an object of Employee

return 0;

}

```

Output:

```

Constructor Invoked

Constructor Invoked

Destructor Invoked

Destructor Invoked

```

C++ this Pointer

In C++ programming, **this** is a keyword that refers to the current instance of the class.

There can be 3 main usage of this keyword in C++.

- It can be used to pass current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.

C++ this Pointer Example

Let's see the example of this keyword in C++ that refers to the fields of current class.

```

#include <iostream>

using namespace std;

```

```

class Employee {

public:

    int id; //data member (also instance variable)

    string name; //data member(also instance variable)

    float salary;

    Employee(int id, string name, float salary)

    {

        this->id = id;

        this->name = name;

        this->salary = salary;

    }

    void display()

    {

        cout<<id<<" "<<name<<" "<<salary<<endl;

    }

};

int main(void) {

```

```

Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of E
mployee

Employee e2=Employee(102, "Nakul", 59000); //creating an object of Em
ployee

e1.display();

e2.display();

return 0;

}

```

Output:

```

101  Sonoo  890000

102  Nakul  59000

```

C++ static

In C++, static is a keyword or modifier that belongs to the type not instance. So instance is not required to access the static members. In C++, static can be field, method, constructor, class, properties, operator and event.

Advantage of C++ static keyword

Memory efficient: Now we don't need to create instance for accessing the static members, so it saves memory. Moreover, it belongs to the type, so it will not get memory each time when instance is created.

C++ Static Field

A field which is declared as static is called static field. Unlike instance field which gets memory each time whenever you create object, there is only one copy of static field created in the memory. It is shared to all the objects.

It is used to refer the common property of all objects such as `rateOfInterest` in case of `Account`, `companyName` in case of `Employee` etc.

C++ static field example

Let's see the simple example of static field in C++.

```
#include <iostream>

using namespace std;

class Account {

public:

    int accno; //data member (also instance variable)

    string name; //data member(also instance variable)

    static float rateOfInterest;

    Account(int accno, string name)

    {

        this->accno = accno;
```

```

        this->name = name;

    }

    void display()

    {

        cout<<accno<< " "<<name<< " "<<rateOfInterest<<endl;

    }

};

float Account::rateOfInterest=6.5;

int main(void) {

    Account a1 =Account(201, "Sanjay"); //creating an object of Employee

    Account a2=Account(202, "Nakul"); //creating an object of Employee

    a1.display();

    a2.display();

    return 0;

}

```

Output:

201 Sanjay 6.5

202 Nakul 6.5

C++ static field example: Counting Objects

Let's see another example of static keyword in C++ which counts the objects.

```
#include <iostream>

using namespace std;

class Account {

public:

    int accno; //data member (also instance variable)

    string name;

    static int count;

    Account(int accno, string name)

    {

        this->accno = accno;

        this->name = name;

        count++;

    }

    void display()

    {

        cout<<accno<<" "<<name<<endl;
```

```

    }

};

int Account::count=0;

int main(void) {

    Account a1 =Account(201, "Sanjay"); //creating an object of Account

    Account a2=Account(202, "Nakul");

    Account a3=Account(203, "Ranjana");

    a1.display();

    a2.display();

    a3.display();

    cout<<"Total Objects are: "<<Account::count;

    return 0;

}

```

Output:

201 Sanjay

202 Nakul

203 Ranjana

Total Objects are: 3

C++ Structs

In C++, classes and structs are blueprints that are used to create the instance of a class.

Structs are used for lightweight objects such as Rectangle, color, Point, etc.

Unlike class, structs in C++ are value type than reference type. It is useful if you have data that is not intended to be modified after creation of struct.

C++ Structure is a collection of different data types. It is similar to the class that holds different types of data.

The Syntax Of Structure

```
struct structure_name  
  
{  
  
    // member declarations.  
  
}
```

In the above declaration, a structure is declared by preceding the **struct** keyword followed by the identifier (structure name). Inside the curly braces, we can declare the member variables of different types. **Consider the following situation:**

```
struct Student  
  
{  
  
    char name[20];  
  
    int id;
```

```
    int age;  
  
}
```

In the above case, Student is a structure contains three variables name, id, and age. When the structure is declared, no memory is allocated. When the variable of a structure is created, then the memory is allocated. Let's understand this scenario.

How to create the instance of Structure?

Structure variable can be defined as:

```
Student s;
```

Here, s is a structure variable of type **Student**. When the structure variable is created, the memory will be allocated. Student structure contains one char variable and two integer variable. Therefore, the memory for one char variable is 1 byte and two ints will be $2 * 4 = 8$. The total memory occupied by the s variable is 9 byte.

How to access the variable of Structure:

The variable of the structure can be accessed by simply using the instance of the structure followed by the dot (.) operator and then the field of the structure.

For example:

```
s.id = 4;
```

In the above statement, we are accessing the id field of the structure Student by using the **dot(.)** operator and assigns the value 4 to the id field.

C++ Struct Example

Let's see a simple example of struct Rectangle which has two data members width and height.

```
#include <iostream>

using namespace std;

struct Rectangle
{

    int width, height;

};

int main(void) {

    struct Rectangle rec;

    rec.width=8;

    rec.height=5;

    cout<<"Area of Rectangle is: "<<(rec.width * rec.height)<<endl;

    return 0;

}
```

Output:

Area of Rectangle is: 40

C++ Struct Example: Using Constructor and Method

Let's see another example of struct where we are using the constructor to initialize data and method to calculate the area of rectangle.

```
#include <iostream>

using namespace std;

struct Rectangle {

    int width, height;

    Rectangle(int w, int h)

    {

        width = w;

        height = h;

    }

    void areaOfRectangle() {

        cout<<"Area of Rectangle is: "<<(width*height); }

};

int main(void) {

    struct Rectangle rec=Rectangle(4,6);
```

```
rec.areaOfRectangle();

return 0;

}
```

Output:

Area of Rectangle is: 24

Structure v/s Class

Structure	Class
If access specifier is not declared explicitly, then by default access specifier will be public.	If access specifier is not declared explicitly, then by default access specifier will be private.
Syntax of Structure: struct structure_name { // body of the structure. }	Syntax of Class: class class_name { // body of the class. }
The instance of the structure is known as "Structure variable".	The instance of the class is known as "Object of the class".

C++ Enumeration

Enum in C++ is a data type that contains fixed set of constants.

It can be used for days of the week (SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY and SATURDAY) , directions (NORTH, SOUTH, EAST and WEST) etc. The C++ enum constants are static and final implicitly.

C++ Enums can be thought of as classes that have fixed set of constants.

Points to remember for C++ Enum

- enum improves type safety
- enum can be easily used in switch
- enum can be traversed
- enum can have fields, constructors and methods
- enum may implement many interfaces but cannot extend any class because it internally extends Enum class

C++ Enumeration Example

Let's see the simple example of enum data type used in C++ program.

```
#include <iostream>
```

```
using namespace std;
```

```
enum week { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };

int main()

{

    week day;

    day = Friday;

    cout << "Day: " << day+1<<endl;

    return 0;

}
```

Output:

Day: 5

C++ Friend function

If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function.

By using the keyword friend compiler knows the given function is a friend function.

For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

1. `class` class_name
2. {
3. `friend` data_type function_name(argument/s); // syntax of friend function.
4. };

In the above declaration, the friend function is preceded by the keyword `friend`. The function can be defined anywhere in the program like a normal C++ function. The function definition does not use either the keyword `friend` or `scope resolution operator`.

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

C++ friend function Example

Let's see the simple example of C++ friend function used to print the length of a box.

```
#include <iostream>
```

```

using namespace std;

class Box

{

    private:

        int length;

    public:

        Box(): length(0) { }

        friend int printLength(Box); //friend function

};

int printLength(Box b)

{

    b.length += 10;

    return b.length;

}

int main()

{

    Box b;

    cout<<"Length of box: "<< printLength(b)<<endl;

```

```
    return 0;

}
```

Output:

Length of box: 10

Let's see a simple example when the function is friendly to two classes.

```
#include <iostream>

using namespace std;

class B;    // forward declaration.

class A
{
    int x;

    public:

    void setdata(int i)
    {
        x=i;
    }

    friend void min(A,B);    // friend function.
};
```

```

class B

{

    int y;

    public:

    void setdata(int i)

    {

        y=i;

    }

    friend void min(A,B);           // friend function

};

void min(A a,B b)

{

    if(a.x<=b.y)

        std::cout << a.x << std::endl;

    else

        std::cout << b.y << std::endl;

}

int main()

```

```

{

    A a;

    B b;

    a.setdata(10);

    b.setdata(20);

    min(a,b);

    return 0;

}

```

Output:

10

In the above example, min() function is friendly to two classes, i.e., the min() function can access the private members of both the classes A and B.

C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

Let's see a simple example of a friend class.

```

#include <iostream>

using namespace std;

```

```

class A

{

    int x=5;

    friend class B;    // friend class.

};

class B

{

    public:

    void display(A &a)

    {

        cout<<"value of x is : "<<a.x;

    }

};

int main()

{

    A a;

    B b;

    b.display(a);

```

```
return 0;  
  
}
```

Output:

```
value of x is : 5
```

In the above example, class B is declared as a friend inside the class A. Therefore, B is a friend of class A. Class B can access the private members of class A.